

无线电干扰源的快速自动定位与清除策略研究

摘 要

本文研究四足机器狗自主搜索、定位并清除未知无线电干扰源的策略。考虑到机器狗运动速度受限、测向存在有界误差，且需抵近方可实施有效清除等现实物理约束，本文将空间几何分析与在线启发式调度深度融合，依次求解了多站交会定位、测点选址优化、多全向源协同巡航，以及全向与定向混合源的包围搜索与清除问题。

针对问题一，研究测向误差下的交会定位多边形直径计算与圆盘覆盖判定。针对 $\pm 1^\circ$ 的测向误差，本文将各测站的单次示向度严格等价于半平面不等式系统，利用半平面交算法精确提取定位凸多边形的所有极点；进一步利用区域直径必在极点对处取得的性质，引入旋转卡壳算法高效锁定最远对踵点。基准算例解得定位多边形几何直径为**48.523 m**，此时以该直径构造的闭圆盘能够完全覆盖定位区域。然而，通过构造正三角形反例，本文从理论上证明了该圆盘在一般情形下并不能实现绝对覆盖（1000组随机测试的完全覆盖率仅为**34.7%**）。上述结果经全顶点穷举与8443次单调性测试交叉验证，算法一致性达**100%**。

针对问题二，以第二检测点坐标为决策变量，建立以最小化“后验最劣几何直径”为目标的优化选点模型。在保证测点距初测可行域最远不超过995 m（预留5 m安全裕量）的前提下，通过离散网格搜索与三阶段寻优，得到离散搜索下的推荐第二检测点坐标为**(839.64, 538.75)**，机动距离为**997.62 m**。二次交会后，最差后验直径由单站的1500.23 m显著缩小至**111.90 m**，不确定区域压缩比高达**92.5%**，从几何机理上印证了该策略能有效拉大交会视角并抑制径向发散。

针对问题三，面向多个全向干扰源的自主搜索与清除任务，建立基准拓扑覆盖与在线状态调度模型。本文设计由中心点与半径**1140 m**的六个外环测站组成七点搜索网，最坏测距为**992.69 m**，实现了对1800 m目标圆域的无缝覆盖，并确保角楔交会凸外包始终包含目标真值。在此基础上，制定了严格的分层清除判据：定位多边形最小外接圆半径收缩至**19.5 m**时提前清除；测向预算耗尽且半径不超过**20 m**时直接清除；若仍未收敛，则启动边长**26 m**的正方形网格遍历进行兜底。在官方模拟器三次正式测试中，清除率均稳居**100%**，虚拟任务平均总耗时为**3162.4 s**（单源加权耗时**243.3 s**），程序实际运行时间平均仅需**4.215 s**。

针对问题四，研究全向与半平面定向混合干扰源的搜索与清除问题。半平面定向源存在背向探测盲区，为此，本文基于凸集分离定理严格论证了“目标落入测站闭凸包内”是保证任意未知朝向至少可被一个测站发现的充要条件。据此设计了由中心站与内外双环共25个基准测站构成的三角剖分网（最大边长**983.60 m**，内切圆半径**1806.28 m**），从物理层面杜绝了全域漏检可能。在主流程序外，引入了对称镜像与自适应尾部探测两项效率支路。100组本地配对样本中的观察结果表明，该支路使平均耗时缩减**109.8 s**，无效清除尝试锐减**40.2%**。官方测试结果表明，混合源清除率依然保持**100%**，虚拟任务平均耗时**6759.9 s**（单源加权**533.7 s**），程序实际运行仅需**5.897 s**。

总体而言，本文模型建立在严密的几何分析与有界误差推导基础之上，算法设计紧密契合机器狗运动学与传感器物理约束。在官方模拟器测试中，全向源与混合源清除率均保持100%，程序实际响应仅需数秒（远低于1200 s比赛上限），具备良好的适应性与高效的在线求解效能。

关键词：交会定位 半平面交 旋转卡壳 七点基准覆盖 双环三角网 在线协同调度

1 问题重述

1.1 问题背景

无线电频谱资源是国家重要的战略资源。近年来，各类突发性不明无线电干扰源严重压制关键频段，给公共安全与基础设施运行带来威胁，针对复杂环境下的干扰源监测与清除已成为电磁安全防护的核心任务。

传统的人工徒步排查效率低下且面临高危风险。为提升响应效率，研制搭载测向与清除载荷的机动巡航平台（如四足机器狗），在未知区域自主机动定位并物理清除干扰源，已成为当前的重要发展方向。本任务的核心目标即是为该机动平台规划最优的搜索航线与定位清除策略，以极小化任务总时间。

1.2 问题提出

目标干扰源分布在一个半径为 1800 米的圆形区域内。机器狗初始位于坐标原点 $(0, 0)$ ，初始监听频道为 1，以 5 m/s 的固定速度在二维平面内直线移动，且机器狗的运动范围不受上述 1800 米目标分布边界的限制，可自由机动以获得更优的观测视角。目标区域内存在若干独占 20 个离散频道的干扰源，其最大有效信号覆盖半径在 1000 ~ 1500 m 之间。

机器狗在执行任务时的客观物理动作约束如下：频道切换耗时 1 秒；停靠调整天线并获取稳定示向度耗时 5 秒；测向机全局角度误差严格有界于 $[-1^\circ, 1^\circ]$ ，且同一地点重复检测时环境测向误差保持恒定；当距干扰源中心不超过 20 米时，方可启动耗时 3 秒的光学精确定位与耗时 2 秒的激光清除（光学成功仅取决于距离，与发射角无关）；此外，若距干扰源不超过 5 米且处于其有效发射角内，将因信号过强而无法获取示向度，但可直接实施精确定位与清除。

基于上述物理机制与客观约束，要求建立数学模型与算法依次解决以下四个核心问题：

- 问题一** 已知若干检测点的空间坐标及目标干扰源在此处的实测示向度。要求基于交会定位法原理，给出计算多边形定位区域直径（即区域内任意两点之间距离的最大值）的具体算法；并判定以该定位区域直径为直径的圆是否能够完全覆盖此定位区域。
- 问题二** 已知某全向干扰源在单一检测点处测得的初始示向度。要求制定第二个检测点的选择策略，以期获得针对该干扰源较好的定位效果，并给出第二个检测点的合理候选区域。
- 问题三** 已知目标区域内的干扰源均为全向干扰源，总数在 10 ~ 16 个之间（具体数目未知）。要求制定机器狗自动搜索定位及清除干扰源的策略与算法，在确保所有干扰源被清除的前提下，使完成任务的总时间越短越好；最后，需通过环境模拟器进行充分演练与三次正式测试，以检验策略有效性并输出规范结果。
- 问题四** 目标区域中同时存在全向干扰源与半平面定向干扰源（即仅在特定 180° 扇区内辐射信号，包含边界），两者混合分布且干扰源总数依然在 10 ~ 16 个之间。其中，定向干扰源的具体数目及各自的辐射方向均完全未知，其余物理条件与时间约束均与问题三相同。要求在此非对称信号辐射遮蔽的复杂对抗条件下，制定针对混

合干扰源的高效检测与清除策略及算法，同样通过环境模拟器完成充分的演练测试与三次正式测试，并按规范要求输出最终测试结果。

2 问题分析

本文研究四足机器狗对未知无线电干扰源的自主搜索定位与物理清除问题。任务涵盖从被动几何解算到单站主动决策、多目标在线协同及非对称定向对抗的完整递进链条。为兼顾 100% 清除率与极小化总耗时，确立“有界误差集合估计 → 几何拓扑覆盖保证 → 主动在线决策优化 → 可验证清除退出闭环”的主线。

2.1 问题一分析：有界测向误差下的集合交会、直径解算与覆盖判定

问题一要求根据给定测站数据求解交会多边形几何直径并判定直径圆盘的覆盖性，属于有界不确定性下的几何集合估计与空间包含性判定问题：

1. **角域代数转化**：测向仪存在 $\pm 1^\circ$ 误差，单站观测形成张角 2° 的平面角楔。利用向量叉积行列式将其转化为两个闭半平面的交集，避免正切奇异角与极角跨界发散，多站交会区域严格表现为紧致凸多边形。
2. **直径降维解算**：由凸分析极值原理，多边形几何直径必在极点（对踵点对）处取得。通过增量半平面求交 ($O(n \log n)$) 获取有序极点后，应用旋转卡壳算法在 $O(h)$ 线性时间内精确提取最大直径及对踵点对。
3. **覆盖矛盾与判据**：由荣格定理 (Jung's Theorem)，紧集最小外接圆半径上界约为 $0.5774D$ ，而直径圆盘半径仅为 $0.5D$ 。当多边形横向展宽较大时（如正三角形或钝角三角形构型），必有顶点外突。为此构建向量点积代数判据 $(\mathbf{v}_j - \mathbf{a}^*) \cdot (\mathbf{v}_j - \mathbf{b}^*) \leq 0$ 实现无开方快速判定。

2.2 问题二分析：单测向约束下的主动感知选点与极小极大不确定性压缩

问题二探讨单站初测下的第二测点优选策略与候选区域划定，属于区间观测条件下的主动感知决策与空间极小化极大 (Minimax) 优化问题：

1. **空间基线机理**：单站初测使目标受限于径向跨度达 1500 m 的狭长角锥，原地重复观测信息增益为零。机器狗必须通过空间位移构建有效物理基线，利用空间视差引入正交约束以收缩可行域。
2. **保底安全域构建**：针对 [1000, 1500] m 未知覆盖半径，基于保底下限 $R_{\min} = 1000$ m 构建保守安全条件。由欧氏距离凸性，第二测点位于以初测多边形各顶点为圆心、1000 m 为半径的闭圆盘交集 $\mathcal{U}$ （排除初测点自身），严格保证信号可接收性。
3. **Minimax 优化与候选域**：以二次交会后最坏几何直径为指标构建 Minimax 选点模型，通过三阶段算法（粗筛 → 局部折半 → 精细评估）锁定最优测点，并量化不同容差（1%, 3%, 5%）下的近优候选域，揭示抑制径向拉伸的几何机理。

2.3 问题三分析：全域多全向源的主动协同搜索、在线追踪与闭环清除

问题三面向 1800 m 圆域内 10 ~ 16 个全向源的清除任务，属于多目标在线协同调度与时间优化问题：

1. **四层保证体系**：针对目标数与频段完全未知的挑战，构筑“拓扑覆盖保证 → 目标真值包含 → 安全清除判据 → 自主完备退出”理论闭环。设计外环半径 1140 m 的

正六边形七点蜂窝拓扑 Q_7 ，严格证明其在保底 1000 m 感知半径下最大边缘测距仅 $992.69\text{ m} < 1000\text{ m}$ ，实现全域无盲区覆盖。

2. **精准清除与网格兜底**：捕获信号后序贯交会裁剪可行域；当最小外接圆（MEB）半径满足 $\rho \leq 19.5\text{ m}$ 时直奔圆心一击清除；对未收敛死角自适应切入边长 $\Delta g = 26\text{ m}$ 正方形网格遍历实施绝对安全兜底。
3. **动态账本与自主退出**：为 20 个独立频段建立“未探测、存活、已清除、确证空置”四态转移账本，在线动态穿插巡航、交会与清除；全频道结案后触发全局安全退出。

2.4 问题四分析：非对称定向遮蔽对抗下的多视角凸包覆盖与双轨协同决策

问题四引入 180° 扇区半平面定向源混合分布，属于非对称电磁遮蔽与不完全信息条件下的多视角几何包围与鲁棒序贯决策问题：

1. **背向盲区多义性**：定向天线单侧发射导致测站处于背向盲区时无法探测信号，无信号反馈涵盖无源、已清、超距与盲侧四类互斥成因，单向搜索彻底失效。
2. **多视角凸包充要性**：由严格分离超平面定理，目标落入有效测站闭凸包是其任意未知朝向均可探测的充要条件。据此设计 25 点双环三角剖分网（中心 1 站 + 内环 12 站 950 m + 外环 12 站 1870 m ），最大边长 $983.60\text{ m} < 1000\text{ m}$ ，内切圆 $1806.28\text{ m} > 1800\text{ m}$ ，拓扑杜绝目标漏检。
3. **双轨协同在线决策**：以拓扑覆盖和网格遍历为“保证主链”，以距离安全选点（ $d_{\max} \leq 995\text{ m}$ ）、条件对称镜像探测与尾部前瞻收益评估为“效率支路”，在守住 100% 清除底线的同时最大化缩减巡航与无效清除时间。

2.5 整体求解思路与论文逻辑架构

四个核心问题层层递进、环环相扣：**问题一为基石**（提供底层半平面交会与旋转卡壳几何计算引擎）；**问题二为纽带**（攻克 Minimax 主动选点与不确定性压缩技术）；**问题三为骨架**（建立蜂窝无盲区覆盖、MEB 精准清除与多频段在线调度框架）；**问题四为升华**（建立多视角凸包覆盖定理与双轨协同策略，攻克非对称定向遮蔽对抗难题）。系统整体求解思路与递进逻辑如图 1 所示。

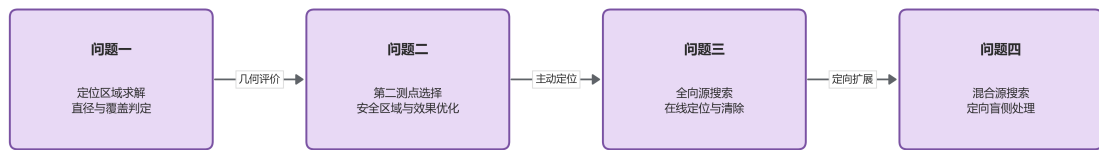


图 1 无线电干扰源定位与清除策略整体求解思路逻辑架构

3 模型假设与符号说明

3.1 模型假设

为使数学模型切合实际物理过程并保持数理严谨，本文提出如下合理假设：

1. **目标位置静止与基准坐标无漂移**：目标干扰源在作业全过程中保持二维空间物理位置静止；机器狗自身定位与方位基准准确，坐标与航向角不存在系统漂移偏差。

2. **信道频段相互正交与有效观测准入**：20 个离散频段相互独立无跨频互调干扰，同一信道在全域至多对应单一客观目标；各测站独立获取测向或无信号状态，仅当测站处于目标有效辐射半平面且距离不超过覆盖半径时产生有效示向角并参与多边形交会解算，无信号测站不作为正向测向约束。
3. **测向角度物理误差严格有界**：测向仪的角度测量误差严格限制在闭区间 $[-\varepsilon, \varepsilon]$ 之内（误差半宽 $\varepsilon = 1^\circ$ ），不依赖先验概率分布假设，按极端最坏情况进行几何包络分析。
4. **同点重复观测误差恒定**：在同一物理地点重复测向时，受固定的局部环境影响，测向系统误差保持不变，无法通过原地多次采样取平均来减小测向偏差。
5. **分段匀速直线机动与离散动作执行**：机器狗转向、起步与制动过程极为迅速，近似为以 5 m/s 固定速度运动的质点分段匀速直线机动；天线调谐、停靠测向、光学定位与激光清除等动作按离散事件串行执行并严格依规程累加时间；物理清除条件仅取决于与目标的真实欧氏间距（ $\leq 20 \text{ m}$ ），不受目标辐射角度限制。
6. **平坦地面视距传播与辐射源状态稳定**：电磁波在平坦二维地面内沿视距直线传播；目标干扰源在任务期间发射功率稳定且工作频率固定，全向源向全域各向同性辐射，定向源辐射主瓣朝向在任务期间固定不变，不存在动态旋转、跳频或间歇关机。

3.2 符号说明

本文使用的主要数学符号及其物理意义如表 1 所示。

表 1 主要数学符号物理意义与数学定义说明

符号	数学定义与物理意义	量纲 / 类型
n	参与协同交会定位的已知检测基站（观测点）总数	无量纲 (个)
$\mathbf{p}_i$	第 i 次观测时检测站的二维平面坐标， $\mathbf{p}_i = (x_i, y_i)^\top$	m
$\hat{\theta}_i$	第 i 次观测所得实测示向度（平面直角坐标系下逆时针度量的极角方向）	rad / 度
ε	测向仪的客观物理误差确界半宽， $\varepsilon = 1^\circ = \pi/180 \text{ rad}$	rad / 度
$\mathbf{s}$	未知的真实辐射源二维平面位置坐标， $\mathbf{s} = (s_x, s_y)^\top$	m
$\mathcal{W}_i$	第 i 次测向在空间中形成的前向有界角域（双向半平面的交集）	平面闭子集
$\mathcal{R}$	全体有效协同观测共同约束下的联合交会定位区域（紧致凸多边形）	紧致凸多边形
$\mathcal{V}$	定位凸多边形 $\mathcal{R}$ 逆时针有序排列的极点序列， $\mathcal{V} = \{\mathbf{v}_0, \dots, \mathbf{v}_{h-1}\}$	极点序列
D	定位多边形区域 $\mathcal{R}$ 内任意两点间欧氏距离的上确界（几何直径）	m
$(\mathbf{a}^*, \mathbf{b}^*)$	凸多边形上距离最远的一组极点对（直径端点对）	极点坐标对
C^*	以最远对踵极点对 $(\mathbf{a}^*, \mathbf{b}^*)$ 连线为直径构造的闭圆盘	平面闭圆盘
κ	直径圆盘 C^* 能否完全包含并覆盖定位区域 $\mathcal{R}$ 的布尔判定指标	布尔量 $\{0, 1\}$

4 问题一：定位区域的直径与覆盖判定

4.1 问题分析

问题一要求针对给定的若干检测点空间坐标及实测示向度，建立交会定位模型，设计求解定位多边形几何直径的具体算法，并判定以该直径构成的闭圆盘能否完全覆盖此

定位区域。

求解问题一的核心难点在于：一方面，测向仪存在有界角度误差（ $\pm 1^\circ$ ），需将离散的物理测向数据转化为严格的几何约束；另一方面，交会形成的多边形为连续闭合区域，无法直接通过离散网格采样来精确计算几何直径和判定圆盘覆盖。

为此，本文建立如下求解流程：

$$\{(\mathbf{p}_i, \hat{\theta}_i)\}_{i=1}^n \longrightarrow \{\mathcal{W}_i\}_{i=1}^n \longrightarrow \mathcal{V} \longrightarrow (D, \mathbf{a}^*, \mathbf{b}^*) \longrightarrow \kappa$$

求解过程分为三个阶段。首先，根据测向误差确界将实测示向度转化为半平面约束，并通过求交得到定位多边形的有序极点序列。随后，利用凸分析极值原理证明区域直径必在极点处取得，通过旋转卡壳算法求取最远顶点对和区域几何直径。最后，以该点对连线为直径构造闭圆盘，并结合理论证明、正三角形几何反例和向量点积判据判断其覆盖性。

4.2 模型建立

4.2.1 测向误差角域与半平面模型

为实现上述交会定位与几何解算流程，首要环节是将物理测向数据转化为严密的几何约束。由于测向仪存在角度误差，单个检测点获得的观测不是一条理想射线，而是具有一定张角的扇形角域。因此，多站协同交会定位的本质是求取各测向角域在二维平面上的公共部分。为此，本节利用双向半平面刻画单个检测点的前向观测角域，进而将联合交会定位区域 $\mathcal{R}$ 严格表示为有限闭半平面的交集 [1]，为后续多边形顶点解算与几何直径计算提供标准的不等式输入。

设参与协同交会定位的已知检测点数量为 n 。对于第 i 个检测点，其空间平面位置记为 $\mathbf{p}_i$ ，实测示向度记为 $\hat{\theta}_i$ （ $i = 1, \dots, n$ ）。设真实辐射源的目标位置为 $\mathbf{s}$ ，其相对于检测点 $\mathbf{p}_i$ 的理论真值方位极角记为 θ_i^* 。在测向仪物理误差半宽 $\varepsilon = 1^\circ$ 的有界约束下，实测示向度与真实方位极角满足：

$$\hat{\theta}_i \equiv \theta_i^* + e_i \pmod{2\pi}, \quad |e_i| \leq \varepsilon \quad (1)$$

为了消除传统斜率法在正切奇异角（ $\pi/2$ 与 $3\pi/2$ ）处的数值发散，并统一处理极角 $0/2\pi$ 跨界周期性，定义第 i 次观测在两侧误差边界处的临界单位导向向量：

$$\mathbf{u}_i^- = \begin{bmatrix} \cos(\hat{\theta}_i - \varepsilon) \\ \sin(\hat{\theta}_i - \varepsilon) \end{bmatrix}, \quad \mathbf{u}_i^+ = \begin{bmatrix} \cos(\hat{\theta}_i + \varepsilon) \\ \sin(\hat{\theta}_i + \varepsilon) \end{bmatrix} \quad (2)$$

定义平面向量叉积行列式算子 $\det(\mathbf{a}, \mathbf{b}) = a_x b_y - a_y b_x$ 。当且仅当向量 $\mathbf{b}$ 处于 $\mathbf{a}$ 的逆时针左侧半平面时， $\det(\mathbf{a}, \mathbf{b}) \geq 0$ 。第 i 次测向所确立的前向闭合楔形角域 $\mathcal{W}_i$ 可严格表示为两个闭半平面的交集：

$$\mathcal{W}_i = \{\mathbf{x} \in \mathbb{R}^2 : \det(\mathbf{u}_i^-, \mathbf{x} - \mathbf{p}_i) \geq 0, \quad \det(\mathbf{x} - \mathbf{p}_i, \mathbf{u}_i^+) \geq 0\} \quad (3)$$

全体有效协同观测的公共交会定位多边形 $\mathcal{R}$ 为全体前向角域之交集：

$$\mathcal{R} = \bigcap_{i=1}^n \mathcal{W}_i \quad (4)$$

由于每个闭半平面均为凸集，有限凸集之交仍为凸集。在合理的多站交会基线配置下， $\mathcal{R}$ 为非空、有界的紧致凸多边形，且满足真实目标真值包含性 $\mathbf{s} \in \mathcal{R}$ 。

在模型边界的界定上，本问求解的定位区域 $\mathcal{R}$ 严格由各检测点的测向角域交会形成。发射覆盖半径（1000 ~ 1500 m）、近场 5 米盲区和目标区域 1800 米圆边界共同决定测向信号能否被成功接收。本问已经给出获得有效示向度的检测点，因此后续交会定位只处理这些观测形成的半平面约束，不再引入先验圆弧进行二次裁剪，从而保持凸多边形几何结构的严整性与算法求解的准确性。至此，由全体测向角域共同界定的定位区域被严格表征为包含 $2n$ 条线性不等式的闭凸集系统。

4.3 模型求解

4.3.1 半平面交与定位多边形求解

上述闭半平面系统在代数上完备界定了目标可行域，但隐式线性不等式无法直接提供空间拓扑结构。为支撑后续几何直径度量与覆盖判定，需将其显式转换为逆时针严格排序的极点序列：

$$\mathcal{R} = \{\mathbf{x} \in \mathbb{R}^2 : A\mathbf{x} \leq \mathbf{b}\} = \text{conv}(\mathcal{V}), \quad \mathcal{V} = \{\mathbf{v}_0, \mathbf{v}_1, \dots, \mathbf{v}_{h-1}\} \quad (5)$$

本文采用基于双端队列的增量半平面求交算法。预处理阶段先提取全部 $2n$ 条有向边界的法向角并按极角排序，同时对同向平行边界仅保留约束最严格者。随后，利用双端队列维护当前可行区域的活动边界；每加入一个新半平面，若队头或队尾的相邻交点落入不可行侧，便执行相应的出队剪枝。完成全部半平面的遍历和闭合检验后，联立相邻边界方程得到逆时针严格排列的规范化极点序列 $\mathcal{V}$ 。该算法将连续区域求解转化为有限边界极点计算，时间复杂度为 $O(n \log n)$ ，空间复杂度为 $O(n)$ 。

4.3.2 旋转卡壳法求解区域直径

基于上述解出的外轮廓极点序列 $\mathcal{V}$ ，多边形定位区域 $\mathcal{R}$ 的几何直径 D 严格定义为区域内任意两点间欧氏距离的上确界：

$$D = \max_{\mathbf{x}, \mathbf{y} \in \mathcal{R}} \|\mathbf{x} - \mathbf{y}\|_2 \quad (6)$$

作为连续二维闭集， $\mathcal{R}$ 内包含无穷多个点，直接求解两点间最大间距属于连续极值优化问题。为实现确定性快速求解，关键在于将连续搜索降维至有限极点集合的离散比较。为此，本文给出并证明如下极值分布命题：

命题 1（凸多边形直径端点必为顶点） 紧致凸多边形 $\mathcal{R}$ 的几何直径 D 必在该多边形的某对极点（外轮廓顶点）之间取得，即：

$$D = \max_{\mathbf{v}_j, \mathbf{v}_k \in \mathcal{V}} \|\mathbf{v}_j - \mathbf{v}_k\|_2 \quad (7)$$

证明：任取区域内的任意两点 $\mathbf{x}, \mathbf{y} \in \mathcal{R}$ 。由凸多边形极点表示定理，两点可分别展开为全体极点集的凸组合： $\mathbf{x} = \sum_{j=0}^{h-1} \lambda_j \mathbf{v}_j$ 与 $\mathbf{y} = \sum_{k=0}^{h-1} \mu_k \mathbf{v}_k$ ，其中系数满足 $\lambda_j, \mu_k \geq 0$ 且 $\sum_j \lambda_j = \sum_k \mu_k = 1$ 。利用系数归一性 $\sum_k \mu_k = \sum_j \lambda_j = 1$ 将差向量改写为双重凸组合，并由范数三角不等式放缩：

$$\|\mathbf{x} - \mathbf{y}\|_2 = \left\| \sum_{j=0}^{h-1} \sum_{k=0}^{h-1} \lambda_j \mu_k (\mathbf{v}_j - \mathbf{v}_k) \right\|_2 \leq \sum_{j=0}^{h-1} \sum_{k=0}^{h-1} \lambda_j \mu_k \|\mathbf{v}_j - \mathbf{v}_k\|_2 \leq \max_{0 \leq j, k \leq h-1} \|\mathbf{v}_j - \mathbf{v}_k\|_2 \quad (8)$$

由于双重凸组合系数满足 $\sum_{j=0}^{h-1} \sum_{k=0}^{h-1} \lambda_j \mu_k = 1$ ，连续区域内任意两点间的欧氏间距上界必不超过有限极点对间的最大距离；又由极点包含性 $\mathcal{V} \subset \mathcal{R}$ ，该极大值在极点集上严格可达，证毕。 ■

命题 1 将连续搜索降维至 $\binom{h}{2}$ 对离散顶点比较，但全顶点穷举计算复杂度仍为 $O(h^2)$ 。根据对踵几何特征，两点最大间距必在相互平行的外切支撑线接触点（对踵点对）处取得。为此引入旋转卡壳算法 [2]，通过一对平行卡尺绕多边形旋转步进，仅需在 $O(h)$ 线性时间内遍历全部对踵候选，即可精确提取区域几何直径 D 及最远顶点对 $(\mathbf{a}^*, \mathbf{b}^*)$ 。

4.3.3 直径圆盘覆盖判定

求得几何直径 D 与最远极点对 $(\mathbf{a}^*, \mathbf{b}^*)$ 后，针对以最远点连线为直径构造的闭圆盘（圆心 $\mathbf{c}^* = \frac{\mathbf{a}^* + \mathbf{b}^*}{2}$ ，半径 $R = D/2$ ）：

$$\mathcal{C}^* = \left\{ \mathbf{x} \in \mathbb{R}^2 : \|\mathbf{x} - \mathbf{c}^*\|_2 \leq \frac{D}{2} \right\} \quad (9)$$

直觉常误以为直径既然是区域内最大间距，该圆盘便理应包含多边形全部点。然而基于平面凸多边形的包含几何理论 [3]，严密分析表明：**以最远点间距为直径构造的闭圆盘，并不能恒定保证完全覆盖定位多边形。**

构造理论反例予以证明：设交会定位区域为边长为 D 的正三角形 $\triangle ABC$ 。由正三角形几何性质可知，其任意两点间距离的最大值即为边长，故该凸多边形的几何直径为 D 。若取底边端点 A, B 作为最远点对构造闭圆盘 $\mathcal{C}^*$ ，圆心为其几何中点 $\mathbf{c}^* = (\mathbf{v}_A + \mathbf{v}_B)/2$ ，半径 $R = 0.5D$ 。第三个顶点 C 到圆心 $\mathbf{c}^*$ 的距离为该三角形的高：

$$h_\Delta = \|\mathbf{v}_C - \mathbf{c}^*\|_2 = \frac{\sqrt{3}}{2}D \approx 0.8660D > 0.5D \quad (10)$$

顶点 C 位于该圆盘之外（超出圆盘边界达 $0.366D$ ），否定了恒能完全覆盖的假设。根据几何学荣格定理，平面上直径为 D 的任意有界闭集，其最小覆盖圆半径满足 $R_{\min} \leq \frac{\sqrt{3}}{3}D \approx 0.5774D$ 。而以最远点间距构造的圆盘半径仅为 $0.5D < 0.5774D$ 。因此，当多边形在垂直于直径的方向上横向宽展较大时，其顶点便会超出该圆盘。

为判定具体多边形的覆盖性，利用凸集性质，多边形 $\mathcal{R} = \text{conv}(\mathcal{V})$ 被闭圆盘完全包含充要等价于全体极点落入圆盘内部。借助向量点积恒等式：

$$(\mathbf{x} - \mathbf{a}^*) \cdot (\mathbf{x} - \mathbf{b}^*) = \|\mathbf{x} - \mathbf{c}^*\|_2^2 - \frac{D^2}{4} \quad (11)$$

点 $\mathbf{x} \in \mathcal{C}^* \iff (\mathbf{x} - \mathbf{a}^*) \cdot (\mathbf{x} - \mathbf{b}^*) \leq 0$ 。该式将开方几何距离检验转化为无需开方的线性内积符号判据，避免了浮点舍入误差。据此定义覆盖布尔算子 $\kappa \in \{0, 1\}$ ：

$$\kappa = \begin{cases} 1, & \text{若 } \forall \mathbf{v}_j \in \mathcal{V}, (\mathbf{v}_j - \mathbf{a}^*) \cdot (\mathbf{v}_j - \mathbf{b}^*) \leq 0 \\ 0, & \text{若 } \exists \mathbf{v}_j \in \mathcal{V}, (\mathbf{v}_j - \mathbf{a}^*) \cdot (\mathbf{v}_j - \mathbf{b}^*) > 0 \end{cases} \quad (12)$$

若 $\kappa = 0$ ，记录超出圆盘边界的最大超量 $g_{\text{viol}} = \max_j \max\{(\mathbf{v}_j - \mathbf{a}^*) \cdot (\mathbf{v}_j - \mathbf{b}^*), 0\}$ 及其对应越界顶点。至此，从测向数据到定位多边形、几何直径求解以及圆盘覆盖判定的完整

几何机制如图 2 所示。其中图 2(a) 直观展示了多站测向角域求交围成的定位多边形；图 2(b) 说明了利用旋转卡壳法在对踵平行外切线上提取区域直径与构造直径圆盘的过程；图 2(c) 则清晰刻画了正三角形底边直径圆盘无法覆盖对向顶点的理论反例与越界几何形态。

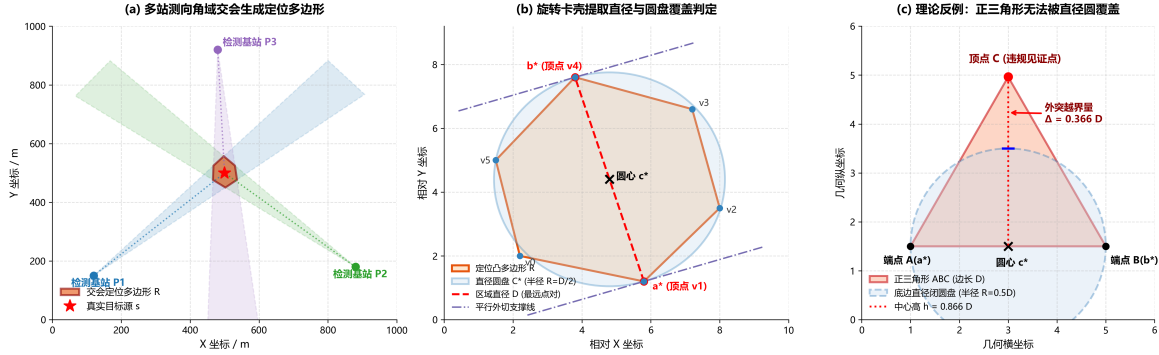


图 2 多站测向角域交会、定位多边形直径提取与圆盘覆盖判定几何机制示意图

4.4 结果分析与模型检验

为检验上述交会定位模型、几何直径算法与覆盖判定准则的正确性与数值稳定性，本节首先通过典型四站对称基准算例与大规模蒙特卡洛统计展示解算过程与覆盖特性规律，随后设计独立的几何枚举对照算法开展多重独立交叉复核与极端工况压力测试，最后分析计算复杂度与异常几何输入的自适应处理机制。

4.4.1 基准算例与覆盖特性分析

为检验算法的基本执行流程与数值精度，首先选取四向对称测向基准算例进行解算。4 个观测站环绕分布于半径 1000 m 的圆周上： $\mathbf{p}_1 = (1000, 0) \text{ m}$ 、 $\mathbf{p}_2 = (0, 1000) \text{ m}$ 、 $\mathbf{p}_3 = (-1000, 0) \text{ m}$ 、 $\mathbf{p}_4 = (0, -1000) \text{ m}$ ，实测示向度分别为指向坐标原点的正四向极角 $180^\circ, 270^\circ, 0^\circ, 90^\circ$ 。在误差半宽 $\varepsilon = 1^\circ$ 约束下，共形成 8 条半平面边界。

算法解算得到严格中心对称的正规凸八边形，覆盖面积为 1197.81 m^2 。旋转卡壳算法求得其最大几何直径为 $D = 48.523 \text{ m}$ ，对应最远顶点对坐标为 $(17.156, -17.156) \text{ m}$ 与 $(-17.156, 17.156) \text{ m}$ 。以该点对连线为直径构造闭圆盘（圆心为坐标原点，半径 24.262 m ），经逐顶点积检验，各顶点的最大点积超量仅为 $9.64 \times 10^{-12} \text{ m}^2$ ，低于预设数值容差，覆盖指标输出 $\kappa = 1$ ，实现了对定位多边形的完全覆盖。基准算例解算指标汇总于表 2。

上述基准算例表明，在空间高度对称的测向环境下，几何直径构造的圆盘确实能够实现了对定位多边形的完全覆盖。然而，在非对称或任意多站协同的普遍场景下，该结论是否仍恒定成立？为从大样本统计层面检验前述理论反例的实证表现，本文基于固定随机种子（20260911）进一步生成 1000 组随机蒙特卡洛测试样本，定义各算例中凸多边形顶点相对直径圆盘边界的最大相对半径超量：

$$\Delta_{\text{rel}} = \frac{\max_j \|\mathbf{v}_j - \mathbf{c}^*\|_2 - D/2}{D/2} \times 100\% \quad (13)$$

当 $\Delta_{\text{rel}} \leq 0$ 时表示多边形完全落入圆盘内（ $\kappa = 1$ ）；当 $\Delta_{\text{rel}} > 0$ 时表示存在顶点超出圆盘边界（ $\kappa = 0$ ）。

表 2 Q1 默认基准算例（四站对称观测）几何解算结果表

几何参数 / 评估指标	符号 / 变量	数值结果	量纲	验证状态与备注
观测基站总数	N_{obs}	4	个	环绕分布于 $R = 1000 \text{ m}$
有效半平面约束数	M	8	条	每站提供 $\pm 1^\circ$ 双向边界
定位区域状态	Status	正常有界	-	严格有界、正常非退化凸多边形
多边形极点总数	h	8	个	严格逆时针拓扑排序
定位多边形覆盖面积	S	1197.81	m^2	对称八边形紧致闭包
定位区域最大几何直径	D	48.523	m	旋转卡壳提取，最远对 $(\mathbf{a}^*, \mathbf{b}^*)$
直径圆盘圆心坐标	$\mathbf{c}^*$	(0.000, 0.000)	m	最远点对连线几何中点
直径圆盘半径	$R = D/2$	24.262	m	待检圆盘半径
逐顶点面积最大违规量	g_{viol}	9.64×10^{-12}	m^2	低于预设容差 10^{-10} ，按零处理
圆盘完全覆盖判定结论	κ	完全覆盖	-	默认基准算例下圆盘实现 100% 覆盖

在 1000 个随机蒙特卡洛测试样本中，共有 347 个算例被直径圆盘完全覆盖（占 34.7%），653 个算例发生外突未被完全覆盖（占 65.3%）。图 3 绘制了将 1000 个算例按相对半径超量由小到大升序排序后的经验分布曲线。

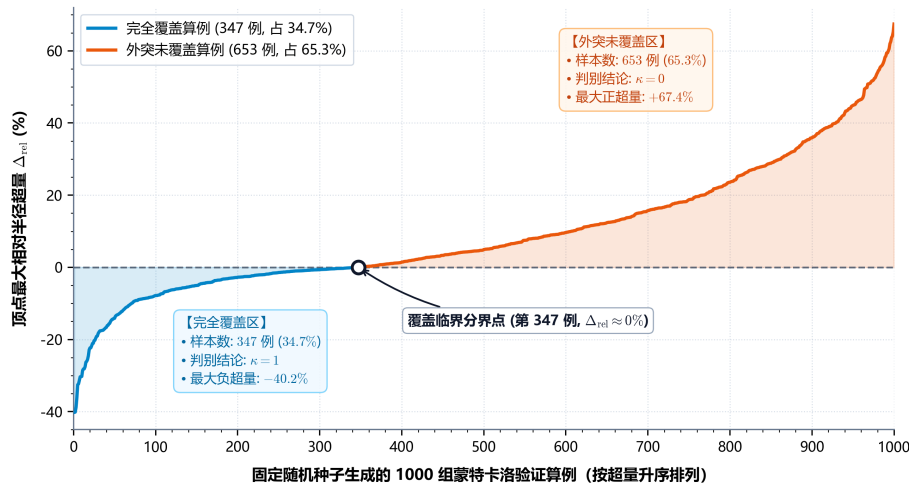


图 3 固定随机种子生成的 1000 个随机蒙特卡洛算例相对半径超量排序经验折线

由图 3 可见，曲线自负超量区跨越零点延伸至正超量区（最大正超量为 +67.4%）。这直观表明多边形能否被其直径圆盘覆盖取决于横向宽展程度：当多边形在垂直于直径方向较窄时容易被圆盘包含；而当形态趋近于正三角形或宽展钝角几何时，顶点便易超出圆盘。该统计实证与理论反例互相印证，证明了算法对覆盖成立与外突失效均具备灵敏准确的双向判别能力。

4.4.2 算法对照与稳健性检验

在明确了基准算例数值与千例统计规律之后，为检验增量半平面交与旋转卡壳主算法在浮点计算与边界剪枝中是否存在潜在偏差，本文专门设计了一套基于穷举原理的独

立对照复核算法，针对三个核心求解环节展开多角度独立交叉校验：

1. **多边形构造对照（全边界两两求交与可行性筛选法）**：不同于主算法依赖极角排序与双端队列动态维护外轮廓，对照算法直接对全体 $2n$ 条半平面边界直线进行全组合联立求解，解出全部 $\binom{2n}{2}$ 个候选交点坐标；随后将每个交点逐一代入全部 $2n$ 个半平面不等式中检验可行性，剔除所有违反任意约束的外侧交点，仅保留全局可行顶点；最后对保留的离散点集执行平面凸包构建，得到无冗余的逆时针多边形顶点集。该方法计算复杂度为 $O(n^3)$ ，但逻辑直观、不涉及增量剪枝，可作为多边形顶点的可靠对照依据。
2. **几何直径求解对照（全顶点对欧氏距离全局穷举法）**：不依赖旋转切线步进与对踵点候选理论，对照算法直接对多边形的全部 h 个顶点展开全局双重循环穷举，计算任意两顶点间的欧氏距离： $D_{\text{ref}} = \max_{0 \leq j < k < h} \|\mathbf{v}_j - \mathbf{v}_k\|_2$ 。该方法复杂度为 $O(h^2)$ ，遍历全部顶点组合，用于直接检验旋转卡壳算法在对踵点搜索时是否存在漏检或步进跳跃。
3. **圆盘覆盖判定对照（顶点外接欧氏半径直接对比法）**：对照算法绕过向量点积代数式，直接计算每个多边形顶点到直径圆心 $\mathbf{c}^* = (\mathbf{a}^* + \mathbf{b}^*)/2$ 的实际欧氏空间距离 $d_j = \|\mathbf{v}_j - \mathbf{c}^*\|_2$ 。当且仅当所有顶点均满足 $d_j \leq D/2 + 10^{-10} \text{ m}$ 时判定为完全覆盖，与主算法的点积准则形成机理独立的对照。

在 1000 组多站协同测向测试算例的全量对照比对中：增量半平面交与全边界两两筛选法求得的多边形极点序列完全一致，面积相对偏差小于 10^{-12} ；旋转卡壳算法与全点对距离穷举法求得的几何直径绝对偏差严格为 0.0 m ；点积判据与欧氏半径判据对圆盘覆盖性的布尔判断结果一致率达 100%。

本文进一步开展五项极端工况下的稳健性压力测试。对观测顺序进行 1000 次随机置换，并人工添加 1000 次同向完全重合半平面后，算法均能自适应处理，输出保持不变。对整个系统施加 37° 空间旋转和大距离平移时，求解区域直径同样严格保持。8443 次增量约束测试均满足定位区域直径单调非增性质；将临界容差调整为 10^{-9} 和 10^{-11} 时，求解状态与覆盖判定也保持高度稳定。双算法对照与稳健性检验结果汇总于表 3。

4.4.3 复杂度与异常情形分析

在算法复杂度与工程可用性方面：常规运行模式下仅执行增量半平面交与旋转卡壳算法，理论时间复杂度为 $O(n \log n)$ ，空间复杂度为 $O(n)$ ；在标准运行环境下，包含全套对照复核的 1000 例测试平均单例耗时仅为 3.24 毫秒，计算效率充分满足工程实时应用需求。同时，算法设计了针对空集、无界可行域、退化线段或单点等异常几何输入的自适应状态判别机制，仅在正常凸多边形状态下输出确定性几何直径与覆盖结论，为后续选点优化与机动搜索策略提供了底层计算支持。

5 问题二：第二检测点选择与候选区域

5.1 问题分析

在问题二中，机器狗在第一检测点 $\mathbf{p}_1$ 测得某干扰源的初始示向度 $\hat{\theta}_1$ 。题设要求确定第二检测点 $\mathbf{p}_2$ 的选择策略与较优候选区域。基于问题一所建立的测向角域半平面交会模型与多边形几何直径解算算法，第二检测点的选择需兼顾以下两点：

表 3 主算法与复核方法对照及大规模检验统计表

计算环节 / 检验项目	正式算法	对照复核方法 / 检验标准	样本规模	检验结论 / 通过率
定位多边形构造	极角排序半平面交 ($O(n \log n)$)	边界两两求交全筛选 ($O(n^3)$)	1000 例	极点集与面积 100% 一致
区域几何直径计算	旋转卡壳算法 ($O(h)$)	全顶点对距离穷举 ($O(h^2)$)	1000 例	直径绝对偏差严格为 0.0 m
直径圆盘覆盖判定	顶点向量点积准则 ($O(h)$)	顶点到圆心欧氏距离准则 ($O(h)$)	1000 例	覆盖布尔值 100% 一致
观测序列乱序不变性	随机置换观测输入顺序	比较置换前后求解直径与覆盖	1000 次	1000 次全部通过 ($\Delta D = 0.0$ m)
冗余约束不变性检验	人工添加同向完全重合半平面	比较添加前后几何输出	1000 次	1000 次全部通过 (完全一致)
刚体空间变换不变性	37° 旋转 + 大距离平移	比较变换前后区域直径	100 次	100 次全部通过 (严格保持)
约束增加单调性检验	逐步递增有效观测半平面	验证几何区域直径单调非增	8443 次	8443 次检验 100% 通过
容差数值敏感性检验	临界容差 10^{-9} 与 10^{-11}	检验求解状态与覆盖分类稳定度	50 次	50 次全部通过 (数值高度稳定)
算法综合执行效率	正式算法 + 双算法对照复核	包含全部复核的千例总耗时	1000 例	总耗时 3.24 s (单例 3.24 ms)

1. **信号接收保证**：干扰源覆盖半径 $R \in [1000, 1500]$ m 未知。为获得不依赖未知实际覆盖半径的确定性接收保证，本文选取如下充分安全条件：第二检测点到该干扰源所有相容位置的距离均不得超过最小覆盖半径 $R_{\min} = 1000$ m。
2. **定位效果最优**：在目标先验分布未知的情况下，以两次交会后最坏情况下的定位多边形几何直径衡量定位效果，采用极小化极大（Min-Max）准则使该最差后验直径达到最小 [4]。

选点以两次交会后的最坏后验定位效果为主目标，并将移动距离与安全余量作为次级优化准则。

5.2 模型建立

5.2.1 初次观测后的目标可行域

设第一检测点坐标为 $\mathbf{p}_1 = (x_1, y_1)^\top$ ，实测示向度为 $\hat{\theta}_1$ 。初次观测后，干扰源真实位置 $\mathbf{s}$ 满足如下几何约束系统：

$$\begin{cases} \mathbf{s} \in \mathcal{W}(\mathbf{p}_1, \hat{\theta}_1, \varepsilon) & (\text{测向角域约束, 误差半宽 } \varepsilon = 1^\circ) \\ \|\mathbf{s} - \mathbf{p}_1\|_2 \leq R_{\max} = 1500 \text{ m} & (\text{信号有效接收覆盖半径约束}) \\ \|\mathbf{s}\|_2 \leq R_{\text{target}} = 1800 \text{ m} & (\text{目标分布圆形区域约束}) \\ \|\mathbf{s} - \mathbf{p}_1\|_2 > r_{\text{near}} = 5 \text{ m} & (\text{近场强信号盲区排除约束}) \end{cases} \quad (14)$$

上述约束的交集即为初测后的目标可行域 $\mathcal{P}_1$ 。为便于计算，采用角域射线与圆边

界切线构建包含 $\mathcal{P}_1$ 的凸多边形外包络 $\mathcal{P}_1^+$ (满足 $\mathcal{P}_1 \subseteq \mathcal{P}_1^+$)。数值实现中, 目标圆区域及最大接收圆均采用 48 边外切正多边形近似, 以保证 $\mathcal{P}_1 \subseteq \mathcal{P}_1^+$ 的包含关系。在基准算例 ($\mathbf{p}_1 = (0, 0) \text{ m}$, $\hat{\theta}_1 = 0^\circ$) 下, $\mathcal{P}_1^+$ 为底边长 52.36 m、高 1500 m 的等腰三角形。

5.2.2 第二检测点的安全候选域

为保证在第二检测点 $\mathbf{p}_2$ 处必定接收到信号, 定义信号接收安全余量:

$$m(\mathbf{p}_2) = R_{\min} - \max_{\mathbf{s} \in \mathcal{P}_1^+} \|\mathbf{p}_2 - \mathbf{s}\|_2 \quad (15)$$

若考虑定位与控制误差, 可设置缓冲距离 $b > 0$ 要求 $m(\mathbf{p}_2) \geq b$; 本文基准模型取 $b = 0$ 。

由于欧氏距离关于点 $\mathbf{s}$ 具有凸性, 点 $\mathbf{p}_2$ 到凸多边形 $\mathcal{P}_1^+$ 内任意点的最大距离必在其有限顶点集 $V(\mathcal{P}_1^+)$ 处取得:

$$\max_{\mathbf{s} \in \mathcal{P}_1^+} \|\mathbf{p}_2 - \mathbf{s}\|_2 = \max_{\mathbf{v} \in V(\mathcal{P}_1^+)} \|\mathbf{p}_2 - \mathbf{v}\|_2 \quad (16)$$

据此, 第二检测点的保守安全候选区域 $\mathcal{U}$ 为:

$$\mathcal{U} = \left(\bigcap_{\mathbf{v} \in V(\mathcal{P}_1^+)} \{\mathbf{p}_2 \in \mathbb{R}^2 : \|\mathbf{p}_2 - \mathbf{v}\|_2 \leq R_{\min}\} \right) \setminus \{\mathbf{p}_1\} \quad (17)$$

即 $\mathcal{U}$ 为以 $\mathcal{P}_1^+$ 各顶点为圆心、 $R_{\min}$ 为半径的闭圆盘交集并排除第一检测点自身。排除 $\mathbf{p}_1$ 是由于题设同地点重复检测的环境误差保持不变, 原地再次测量无法提供独立测向基线。只要 $\mathbf{p}_2 \in \mathcal{U}$, 即可确保对可行域内的任意源位置均处于有效接收范围内。若 $\mathcal{U} = \emptyset$, 表明在以 $R_{\min}$ 为充分安全约束下无可行测点, 算法将给出无解标识并转入回退流程。

5.2.3 第二检测点优化模型

在安全区域 $\mathcal{U}$ 内, 机器狗在 $\mathbf{p}_2$ 处观测可能出现两种情形:

1. 若第二检测点距干扰源真实位置不超过近场阈值 ($\|\mathbf{p}_2 - \mathbf{s}\|_2 \leq r_{\text{near}} = 5 \text{ m}$), 测向接口返回 **near**。机器狗随后可进入光学精确定位与清除流程, 因此在第二测点选择目标中将该反馈视为终止分支, 令其剩余位置不确定性为 0;
2. 若处于近场之外, 测得示向度 $\hat{\theta}_2$ 。在测向误差 $e_2 \in [-\varepsilon, \varepsilon]$ 作用下, 形成第二角域 $\mathcal{W}_2$, 交会后的保守后验可行域为 $\mathcal{P}_2 = \mathcal{P}_1^+ \cap \mathcal{W}_2$ 。

据此, 定义单次观测下的后验定位损失函数为:

$$L(\mathbf{p}_2; \mathbf{s}, e) = \begin{cases} 0, & \|\mathbf{p}_2 - \mathbf{s}\|_2 \leq r_{\text{near}} \\ \text{diam} [\mathcal{P}_1^+ \cap \mathcal{W}(\mathbf{p}_2, \text{bearing}(\mathbf{p}_2, \mathbf{s}) + e, \varepsilon)], & \|\mathbf{p}_2 - \mathbf{s}\|_2 > r_{\text{near}} \end{cases} \quad (18)$$

在连续理论上, 真实相容状态属于 $\mathcal{P}_1$ 。为保守覆盖源位置趋近 5 米盲区边界时的极限情形, 连续评估将状态集合扩展至闭包 $\overline{\mathcal{P}}_1 = \text{cl}(\mathcal{P}_1)$, 数值采样点集相应满足 $S_N \subseteq \overline{\mathcal{P}}_1$ 。交会区域采用外包络 $\mathcal{P}_1^+$ 实施保守估计, 定义保守最坏后验直径为:

$$J^+(\mathbf{p}_2) = \sup_{\mathbf{s} \in \overline{\mathcal{P}}_1, |e| \leq \varepsilon} L(\mathbf{p}_2; \mathbf{s}, e) \quad (19)$$

式中明确区分了真实源状态空间 $\mathbf{s} \in \bar{\mathcal{P}}_1$ 与保守包络几何 $\mathcal{P}_1^+$ 。在数值解算时，采用有限采样集（ N 个代表源状态 $S_N \subset \bar{\mathcal{P}}_1$ 与 M 个误差节点 E_M ）计算离散评估指标：

$$\hat{J}_{N,M}(\mathbf{p}_2) = \max_{\mathbf{s}_i \in S_N, e_j \in E_M} L(\mathbf{p}_2; \mathbf{s}_i, e_j) \quad (20)$$

经全域粗筛与局部寻优后，入围候选点进入精细评估阶段（ $N = 64, M = 7$ ）。设进入精细评估的候选点集合为 $\mathcal{Q}_{\text{final}} \subset \mathcal{U}$ ，当前优化搜索求得的最优评价值与数值推荐点定义为：

$$\tilde{J}_{64,7} = \min_{\mathbf{p} \in \mathcal{Q}_{\text{final}}} \hat{J}_{64,7}(\mathbf{p}), \quad \tilde{\mathbf{p}}_2 \in \arg \min_{\mathbf{p} \in \mathcal{Q}_{\text{final}}} \hat{J}_{64,7}(\mathbf{p}) \quad (21)$$

进入精细评估阶段的候选点按 $(\hat{J}_{64,7}(\mathbf{p}), \|\mathbf{p} - \mathbf{p}_1\|_2, -m(\mathbf{p}))$ 的字典序严格排序并优选。

为进一步量化给出较优候选区域，定义在空间采样网格 $\mathcal{G}_h$ （网格步长 h ）上相对容差 $\tau > 0$ （如 $\tau = 1\%, 3\%, 5\%$ ）的近优候选区域：

$$\hat{\mathcal{C}}_{\tau,h} = \left\{ \mathbf{p} \in \mathcal{G}_h \cap \mathcal{U} : \hat{J}_{N,M}(\mathbf{p}) \leq (1 + \tau) \hat{J}_{N,M,h}^* \right\} \quad (22)$$

式中 $\hat{J}_{N,M,h}^* = \min_{\mathbf{p} \in \mathcal{G}_h \cap \mathcal{U}} \hat{J}_{N,M}(\mathbf{p})$ 为全域网格扫描下的基准最优值。该区域给出了在定位性能损失受控前提下允许选点的几何范围。

5.3 模型求解

5.3.1 分层搜索与精细评估

为兼顾全局搜索视野与计算效率，采用全域粗筛、局部寻优与精细评估三阶段求解算法：

1. **全域粗筛**：以 $\mathcal{P}_1^+$ 的最小包围圆（半径记为 r_{MEC} ）圆心为基准向四周引出 12 个辐射方向，分别在安全射线最大长度的 55% 与 90% 处取样，连同圆心共生成至多 25 个初选候选点，并采用 24 个源点 $\times$ 3 个误差节点 $(-\varepsilon, 0, \varepsilon)$ 进行粗评估；
2. **局部寻优**：选取粗筛前 3 个种子点，以初始步长 $h_0 = \min\{200, \max[20, 0.45(R_{\min} - r_{\text{MEC}})]\}$ 进行 4 轮八邻域步长折半搜索；
3. **精细评估**：将粗筛前 6 点与局部寻优点合并去重构建最终入围集 $\mathcal{Q}_{\text{final}}$ ，采用 64 个源点 $\times$ 7 个误差节点实施最终离散复核，输出字典序排序前三的优质候选点，并以首位点作为基准推荐点 $\tilde{\mathbf{p}}_2$ 。

第二检测点三阶段快速筛选与优化流程如图 4 所示。

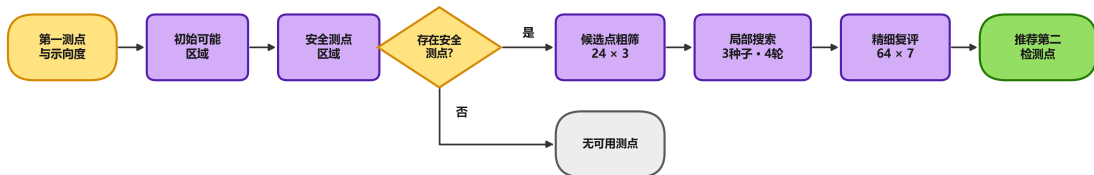


图 4 第二检测点三阶段快速筛选与局部搜索优化流程

5.4 结果分析与模型检验

5.4.1 基准算例与多场景验证

记算法求得的最坏后验直径为 $\tilde{D}_2 := \tilde{J}_{64,7} = \hat{J}_{64,7}(\tilde{\mathbf{p}}_2)$ ，初始外包络的几何直径为 $D_1^+ = \text{diam}(\mathcal{P}_1^+)$ 。据此，定义离散评估下的不确定性压缩比为 $\tilde{\eta} = (D_1^+ - \tilde{D}_2)/D_1^+$ 。对基准算例 ($\mathbf{p}_1 = (0, 0) \text{ m}$, $\hat{\theta}_1 = 0^\circ$) 进行求解，单次计算耗时约 1.2 秒，指标如表 4 所示。

表 4 基准算例下第二检测点优化解算指标

评估指标	符号	数值	量纲	说明
第一检测点位置	$\mathbf{p}_1$	(0.00, 0.00)	m	初始观测位置坐标
初始实测示向度	$\hat{\theta}_1$	0°	度	沿 $+x$ 轴正方向
推荐第二检测点	$\tilde{\mathbf{p}}_2$	(839.64, 538.75)	m	优化所得推荐测点坐标
机器狗移动距离	$\ \tilde{\mathbf{p}}_2 - \mathbf{p}_1\ _2$	997.62	m	两检测点间直线机动位移
保守安全余量	$m(\tilde{\mathbf{p}}_2)$	2.38	m	目标外包络最劣距离余量
初测外包络跨度	D_1^+	1500.23	m	基准算例初始外包络几何直径
离散最坏后验直径	$\tilde{D}_2$	111.90	m	64 源点 $\times$ 7 误差节点采样复核
离散评估压缩比	$\tilde{\eta}$	92.5%	-	$(D_1^+ - \tilde{D}_2)/D_1^+$
单例计算耗时	t_{solve}	~ 1.2	s	算法单次完整求解耗时

在基准算例下，初始观测可行域关于第一示向线呈轴对称分布，优化模型在理论上存在两侧对称的最优解。数值求解在示向线两侧分别获得两组镜像对称的优选测点：

$$\tilde{\mathbf{p}}_2^{(1)} = (839.64, 538.75) \text{ m}, \quad \tilde{\mathbf{p}}_2^{(2)} = (839.64, -538.75) \text{ m} \quad (23)$$

两点移动距离均为 997.62 m，安全余量均为 2.38 m。受离散采样影响，两侧测点的目标函数值存在约 0.14 m 的数值微差（正侧 111.90 m，负侧 112.04 m）。选取正侧测点作为推荐结果，其最坏后验直径由初测的 1500.23 m 降至 111.90 m，不确定性压缩比达到 92.5%。

图 5 展示了保守安全域 $\mathcal{U}$ 的几何形态及推荐测点分布。从几何机理来看，推荐测点分布于第一示向线两侧，其核心优势在于拉大空间交会角，有效避免小角度交会造成的后验多边形径向过度拉伸 [5]，并在机动距离与定位精度间实现综合折中 [6]；更为关键的是，该构型并非人为机械预设固定的正交夹角，而是由最差后验几何直径指标内生驱动得出的最优决策。

为进一步检验算法对不同几何构型的适应性，对圆心、偏置、切向及圆外等 6 个代表性测试场景开展求解，结果如表 5 所示。

在基准测试中，六个代表性场景单次解算耗时均在 1.18 ~ 1.30 s 之间，后验定位直径较初测外包络显著压缩，展现出良好的模型稳健性与实时求解性能。特别地，在圆外场景中无缓冲推荐测点的安全余量仅为 0.27 m，表明在纯定位精度导向下最优测点易贴近安全域边界，实际工程应用中宜结合平台控制精度设置物理安全缓冲。

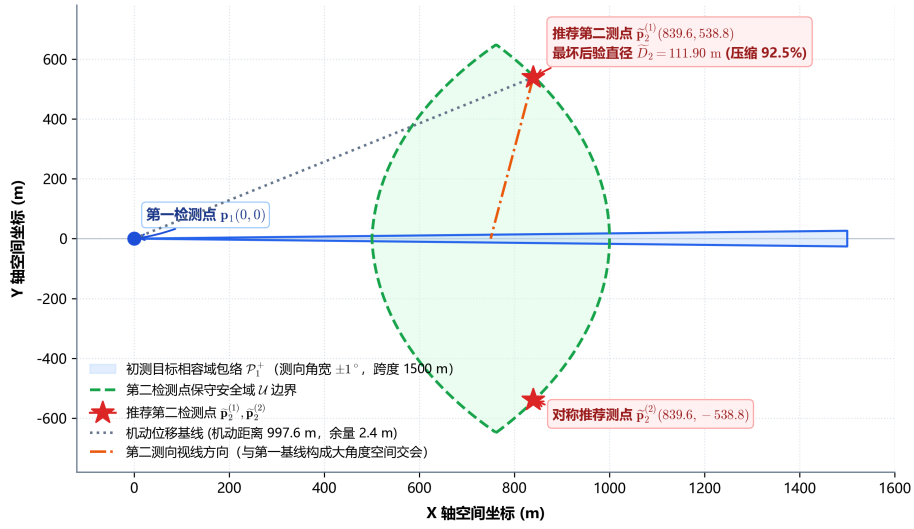


图 5 第二检测点保守安全域 $\mathcal{U}$ 与推荐测点几何分布图

表 5 快速模型在不同测试场景下的实际解算结果对比

测试场景	第一测点与示向	推荐测点 $\tilde{p}_2$	余量	D_1^+	$\tilde{D}_2$	压缩比	耗时与场景说明
中心基准	(0, 0) m, 0°	(839.6, 538.8) m	2.38 m	1500.2 m	111.90 m	92.5%	1.20 s, 对称交会
偏置场景	(300, 500) m, 125°	(-615.9, 898.5) m	1.16 m	1493.9 m	107.21 m	92.8%	1.18 s, 内部非对称
边缘内指	(1600, 0) m, 180°	(760.4, -538.8) m	2.38 m	1500.2 m	111.90 m	92.5%	1.22 s, 边界指心
边缘切向	(1600, 0) m, 90°	(1021.8, 605.0) m	163.20 m	854.7 m	41.41 m	95.2%	1.28 s, 目标圆截断
圆外内指	(2000, 0) m, 180°	(911.1, 454.0) m	0.27 m	1300.3 m	85.87 m	93.4%	1.30 s, 圆外测点余量极敏感
跨界回绕	(0, 0) m, 359.5°	(844.6, 531.3) m	2.20 m	1500.5 m	111.87 m	92.5%	1.24 s, 角度平滑跨越

5.4.2 近优候选区域

为给出具体的较优候选区域，在基准算例下以网格步长 $h = 25 \text{ m}$ 对保守安全域 $\mathcal{U}$ 实施全域网格扫描。扫描范围由各安全圆盘的公共包围盒确定，即 $x \in [500, 1000] \text{ m}$ 、 $y \in [-950, 950] \text{ m}$ ，其中共有 702 个有效安全网格点。对这些网格点，采用与全域粗筛阶段一致的 24 个代表源点和 3 个误差节点计算离散指标 $\hat{J}_{24,3}$ 。全域网格最优值为 111.893 m，对应最优网格点 (850, 525) m。各近优候选区域计算结果汇总于表 6，其中区域面积按入选网格点数乘以单个网格单元面积 ($h^2 = 625 \text{ m}^2$) 进行估算。

表 6 基准算例下不同容差近优候选域 $\hat{C}_{\tau,h}$ 的 25 m 网格统计

相对容差 τ	直径阈值	网格点数	网格面积估计	8-连通分量	总包围盒 (x, y)
1%	113.012 m	4	0.0025 km ²	4	$[800, 850] \times [-600, 600] \text{ m}$
3%	115.250 m	14	0.00875 km ²	2	$[775, 875] \times [-625, 625] \text{ m}$
5%	117.488 m	28	0.0175 km ²	2	$[750, 900] \times [-625, 625] \text{ m}$

计算结果表明：当相对容差 τ 取 3% 与 5% 时，近优候选区域在初始示向线两侧各形成一个连通分支，呈现严格的对称分布；当容差收紧至 1% 时，候选区域高度集中于最优测点邻域，在 25 m 网格步长下包含 4 个相邻网格点。

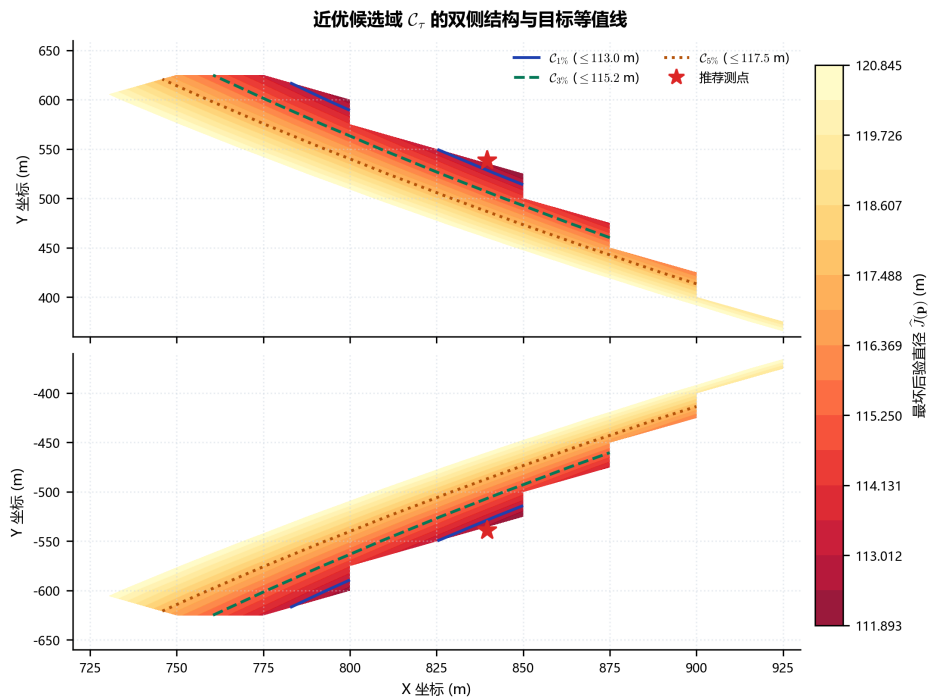


图 6 基准算例下近优候选域 C_τ 的双侧结构与最坏后验直径等值线分布图

5.4.3 运行稳定性检验

为排除单次计算的偶然波动，对表 5 中的 6 个代表性场景分别独立重复执行 20 次完整解算（共计 120 次）。表 7 给出了各场景耗时的统计均值、标准差、中位数、95 分位数（P95）及最大值。

实测表明，各场景平均耗时稳定在 1.216 ~ 1.286 s，P95 均不超过 1.313 s，全样本最大耗时仅 1.326 s，且变异系数均低于 2%。统计结果证实该算法计算效率高度稳定，完全能够支持机器狗在动态任务中进行实时在线决策。

表 7 六大测试场景各 20 次独立完整解算耗时统计表（单位：s）

测试场景	耗时均值	标准差	中位数	P95 分位数	最大耗时
中心基准	1.224	0.019	1.223	1.253	1.270
偏置场景	1.216	0.012	1.214	1.232	1.250
边缘内指	1.238	0.016	1.234	1.273	1.275
边缘切向	1.286	0.017	1.284	1.313	1.326
圆外内指	1.264	0.009	1.264	1.282	1.286
跨界回绕	1.268	0.010	1.265	1.286	1.292

5.4.4 安全缓冲敏感性分析

将信号接收安全要求由 $m(\mathbf{p}_2) \geq 0$ 收紧为带有物理安全缓冲 $b > 0$ 的约束 $m(\mathbf{p}_2) \geq b$ 。在算法实现中，仅将选点安全约束半径收紧为 $R_{\min} - b$ ，而发射源的初始后验状态集 $\mathcal{P}_1^+$ 仍基于真实通信距离 $R_{\min} = 1000$ m 构建，避免因人为引入安全缓冲而错误缩小目标可行域。中心基准场景下的敏感性解算结果汇总于表 8。

表 8 中心基准场景下不同安全缓冲距离 b 的选点敏感性分析表

安全缓冲 b	推荐测点坐标 (x, y)	实际安全余量 m	最坏后验直径 $\tilde{D}_2$	相对 $b = 0$ 性能损失
0 m	(839.6, 538.8) m	2.38 m	111.90 m	基准 (0%)
5 m	(837.9, 532.6) m	7.20 m	112.85 m	+0.85%
10 m	(836.1, 526.4) m	12.03 m	113.47 m	+1.40%
20 m	(832.5, 513.8) m	21.71 m	115.37 m	+3.11%

由表 8 可知，当预留 10 m 的安全缓冲时，推荐测点向安全域内部适当收缩，实际安全余量提升至 12.03 m，而后验直径仅增加 1.57 m（相对性能损失仅 1.40%），表明系统在牺牲极小定位精度的前提下可显著提升行进安全性。

进一步地，设 r_{MEC} 为外包络 $\mathcal{P}_1^+$ 的最小外接圆半径，则安全圆盘交集非空的理论充要条件为：

$$b \leq b_{\max} = R_{\min} - r_{\text{MEC}} \quad (24)$$

对于圆外内指场景，外包络最小外接圆半径为 $r_{\text{MEC}} = 650.268$ m，该场景理论最大容许缓冲为 $b_{\max} = 349.732$ m。数值扫描验证表明：在 $b = 349$ m 时算法仍能解出可行测点（ $\tilde{D}_2 = 650.27$ m），而在 $b = 350$ m 时安全域为空集，算法返回无解标识，与理论极限吻合。在 $b = 0$ 时，由于模型以极小化后验直径为主目标，优选测点贴近安全域边界，安全余量仅为 0.27 m。设置物理安全缓冲 $b > 0$ 后，候选区域向最小外接圆圆心收缩，算法也通过自适应后撤推荐测点，以满足新的缓冲要求。与此同时，最坏后验直径平缓增大，反映了信号接收安全性与定位精度之间的权衡。本节建立的选点优化模型与安全域分析方法，为后续多阶段搜索与机动决策提供了计算基础。

6 问题三：全向干扰源的搜索定位与清除

6.1 问题分析

问题三要求机器狗在半径 $R_{\text{target}} = 1800 \text{ m}$ 的圆形区域内，自主搜索、定位并清除全部未知的全向干扰源，在确保无一遗漏的前提下极小化任务总时间。相比前两问，该任务面临三重更为严苛的物理约束与现实挑战：

1. **先验信息完全未知：** 真实目标数量 $N \in [10, 16]$ 、所属频段 $\mathcal{C} = \{1, \dots, 20\}$ 、空间物理坐标 $\mathbf{s}_c$ 及各频道的有效接收半径 $R_c \in [1000, 1500] \text{ m}$ 事先均未知；
2. **空间观测存在区域异构性：** 机载测向传感器存在确界误差 $[-1^\circ, 1^\circ]$ ；当机器狗与目标间距 $\leq 5 \text{ m}$ 时，由于信号饱和无法测向，但可直接实施有效清除； $5 \sim 20 \text{ m}$ 区间内兼具测向与清除条件； $20 \text{ m} \sim R_c$ 仅具备测向条件；超出 R_c 则无任何信号响应；
3. **无外部终止提示：** 外部环境不主动提供剩余存活目标数或任务完成信号，系统必须完全依赖机载记录与状态空间演化，自主确证全场目标已彻底清除并安全决断终止。

为确保全场所有客观存在的干扰源被彻底清除，首要目标为清除率达到 100%，在此硬性前提下极小化全任务时间，建立分层优化目标：

$$\text{lex min } \{1 - \eta_{\text{clear}}, T_{\text{total}}\} \quad (25)$$

其中 $\eta_{\text{clear}} = K/N = 1$ 。本文同时考虑完全清除的理论保证和在线调度效率。理论层面依次建立全域覆盖 [7]、真值包含、安全清除和自主退出条件，从数学上杜绝目标遗漏；执行层面则采用极小极大几何收缩准则和动态优先级调度，使多个频道能够交错完成搜索、追踪与就地清除 [8]，从而大幅压缩行进和测向时间。

6.2 模型建立

6.2.1 机器狗状态与时间模型

机器狗在决策步 k 的物理状态记为 $\mathbf{x}_k = (\mathbf{p}_k, h_k, T_k)$ ，分别表示当前位置、天线调谐频道与累计虚拟时间。按照模拟器的坐标指令模型，机器狗以速率 $v = 5 \text{ m/s}$ 在相邻指令坐标之间直线机动，并将动作目标坐标 $\mathbf{q}$ 作为下一步的位置状态。动作集与状态转移方程为：

$$a_k \in \{\text{measure}(\mathbf{q}, c), \text{clear}(\mathbf{q}, c), \text{recover}, \text{exit}\} \quad (26)$$

$$\mathbf{p}_{k+1} = \mathbf{q}, \quad h_{k+1} = \begin{cases} c, & a_k = \text{measure}(\mathbf{q}, c) \\ h_k, & \text{其它} \end{cases} \quad (27)$$

虚拟时间增量统一由位移时间、调谐时间与动作执行时间构成：

$$\Delta T_k = \frac{\|\mathbf{q} - \mathbf{p}_k\|_2}{v} + \mathbb{I}(h_k \neq c \wedge a_k = \text{measure}) \cdot 1 \text{ s} + t_{\text{act}}(a_k) \quad (28)$$

其中动作执行时间为：测向动作 $t_{\text{act}}(\text{measure}) = 5 \text{ s}$ ；清除动作 $t_{\text{act}}(\text{clear}) = 5 \text{ s}$ （3s 光学定位 + 2s 激光发射，失败则结算 3s）。

6.2.2 观测反馈与可行域更新

执行动作后，获得离散观测反馈 y_k 。目标位置 $\mathbf{s}_c$ 与实际测点 $\mathbf{p}_{\text{actual}}$ 满足如下几何关系：

$$y_k = \begin{cases} \text{near}, & \|\mathbf{s}_c - \mathbf{p}_{\text{actual}}\|_2 \leq 5 \text{ m} \implies \mathbf{s}_c \in \mathcal{B}(\mathbf{p}_{\text{actual}}, 5) \\ \text{direction}(\hat{\theta}), & 5 \text{ m} < \|\mathbf{s}_c - \mathbf{p}_{\text{actual}}\|_2 \leq R_c \implies \mathbf{s}_c \in \mathcal{W}(\mathbf{p}_{\text{actual}}, \hat{\theta}) \\ \text{no_signal}, & \|\mathbf{s}_c - \mathbf{p}_{\text{actual}}\|_2 > R_c \geq 1000 \text{ m} \implies \mathbf{s}_c \notin \mathcal{B}(\mathbf{p}_{\text{actual}}, 1000) \\ \text{clear_success}, & \|\mathbf{s}_c - \mathbf{p}_{\text{actual}}\|_2 \leq 20 \text{ m} \implies \text{清除成功} \\ \text{clear_fail}, & \|\mathbf{s}_c - \mathbf{p}_{\text{actual}}\|_2 > 20 \text{ m} \implies \mathbf{s}_c \notin \mathcal{B}(\mathbf{p}_{\text{actual}}, 20) \end{cases} \quad (29)$$

其中 $\mathcal{W}(\mathbf{p}, \hat{\theta})$ 为实测方位角误差半宽 $\varepsilon = 1^\circ$ 界定的双向半平面角楔。

对于每个频道 $c \in \mathcal{C}$ ，定义四种追踪状态：未探测 $\mathcal{S}_{\text{unk}}$ 、探测存活 $\mathcal{S}_{\text{alive}}$ 、已清除 $\mathcal{S}_{\text{clr}}$ 与确证空置 $\mathcal{S}_{\text{abs}}$ 。系统维护目标理论可行域 $X_{c,k}$ 的紧致凸外包多边形 $K_{c,k}^+$ 。频道的离散状态按照观测事件确定性转移：首次捕获角楔或近场信号后进入 $\mathcal{S}_{\text{alive}}$ ，七个基准站均无信号时进入 $\mathcal{S}_{\text{abs}}$ ，成功清除后进入吸收态 $\mathcal{S}_{\text{clr}}$ 。在目标尚未清除期间，系统继续利用新获得的角楔收缩其凸外包：

$$K_{c,k+1}^+ = K_{c,k}^+ \cap \mathcal{W}(\mathbf{p}_{\text{actual}}, \hat{\theta}) \quad (30)$$

系统实时保证 $X_{c,k} \subseteq K_{c,k}^+$ ，只要 $K_{c,k}^+$ 满足可清除几何条件，理论可行域亦必满足，从而在保障计算凸性与数值效率的同时杜绝漏检。

6.3 完全清除条件

在未知目标分布、有限感知半径、测量误差以及缺乏外部终止提示的严苛条件下，单纯依赖启发式搜索无法杜绝目标遗漏风险。为此，针对全域完全清除任务建立如下数理推导与判定准则体系：第一阶段证明七点蜂窝基准测站对目标全域的完全覆盖，确保任意位置的目标均能被探测；第二阶段证明多次测向交会得到的凸外包多边形恒包含真实目标，确保目标物理位置不被错误剔除；第三阶段建立基于最小外接圆与离散网格的清除判据，保证激光清除成功率并在有限步内收敛；第四阶段给出系统自主安全退出的判定准则，确保全场无残留目标后及时退出。

6.3.1 七点搜索网的全域覆盖

在开展主动定位与精准清除前，首要任务是建立全域基准搜索网，确保无论未知干扰源分布在何处、有效接收半径取何值（在物理下限 $R_c \geq 1000 \text{ m}$ 下），都能在有限个基准测站内被机器狗可靠探测到，从而为后续锁定与清除提供可发现性前提。

构造包含 7 个基准测站的集合 $\mathcal{Q}_7 = \{\mathbf{q}_0\} \cup \{\mathbf{q}_m\}_{m=1}^6$ ，其中中心站 $\mathbf{q}_0 = (0, 0)^\top$ ，外围正六边形对称站分布半径设为 $R_{\text{ring}} = 1140 \text{ m}$ ：

$$\mathbf{q}_m = \left(R_{\text{ring}} \cos \frac{(m-1)\pi}{3}, R_{\text{ring}} \sin \frac{(m-1)\pi}{3} \right)^\top, \quad m = 1, \dots, 6 \quad (31)$$

命题 2（七点蜂窝测站实现全域无盲区覆盖） 以保守感知下限 $R_{\text{min}} = 1000 \text{ m}$ 为半径的 7 个闭圆盘之并集，严格包含半径 1800 m 的目标总区域 $\mathcal{D}_0$ ，即 $\mathcal{D}_0 \subset \bigcup_{m=0}^6 \mathcal{B}(\mathbf{q}_m, R_{\text{min}})$ 。

证明：中心核区 $\|\mathbf{x}\|_2 \leq 1000 \text{ m}$ 直接落入中心基准站圆域 $\mathcal{B}(\mathbf{q}_0, 1000)$ 内。对于外环区间 $r \in [1000, 1800] \text{ m}$ ，考查对称主扇区 $\theta \in [0, \pi/3]$ 内点到测站 $\mathbf{q}_1$ 与 $\mathbf{q}_2$ 的欧氏距离平方 $d_1^2(r, \theta) = r^2 + R_{\text{ring}}^2 - 2rR_{\text{ring}} \cos \theta$ 。由对称性与单调性分析，函数在角平分线 $\theta = \pi/6$ 处关于 r 的偏导数为 $\frac{\partial(d_1^2)}{\partial r} = 2r - \sqrt{3}R_{\text{ring}} > 0$ （当 $r \geq 1000 > \frac{\sqrt{3}}{2} \times 1140 \approx 987.3 \text{ m}$ 时恒正），全域距离测站最远的点位于最外周边缘处，坐标为 $\mathbf{x}_{\text{max}} = (900\sqrt{3}, 900)^\top$ 。计算其与 $\mathbf{q}_1$ 的最大欧氏距离：

$$d_{\text{max}} = \|\mathbf{x}_{\text{max}} - \mathbf{q}_1\|_2 = \sqrt{(900\sqrt{3} - 1140)^2 + 900^2} \approx 992.69 \text{ m} < 1000 \text{ m} = R_{\text{min}} \quad (32)$$

因全域最大距离严格小于保守感知半径，全场不存在探测盲区，目标区域全部点均被完全覆盖，命题得证。 ■

推论：任意客观存在的干扰源在遍历 $\mathcal{Q}_7$ 过程中必被至少一个测站探测捕获；若某一频道在 $\mathcal{Q}_7$ 全部 7 站均无信号，可严格确证该频道无目标，标记为空置 $\mathcal{S}_{\text{abs}}$ 。七点蜂窝全域覆盖构型及机器狗执行巡航定位与激光清除的典型机动轨迹如图 7 所示。

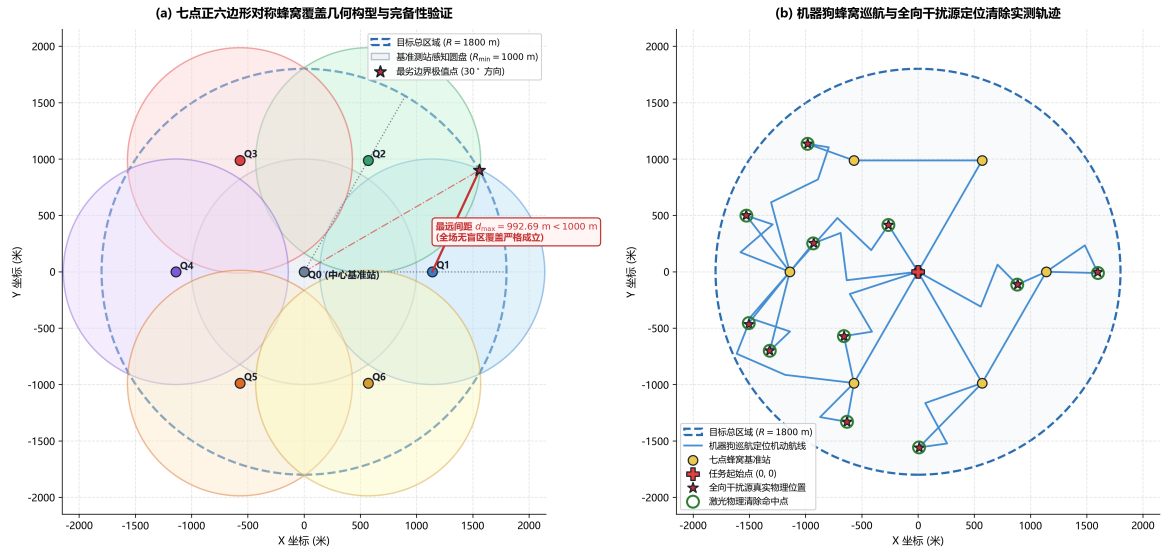


图 7 七点正六边形对称蜂窝全域覆盖几何构型 (a) 与机器狗巡航清除典型轨迹 (b)

6.3.2 真值包含与清除判据

命题 2 确保了全场目标均可被可靠探测。然而，一旦某一频道首次捕获测向信号，由于机载测向仪存在确界误差 $[-1^\circ, 1^\circ]$ ，多次观测交汇形成的可行域必须在数学上严格保持对真实物理位置的可靠包络，避免因几何求交裁剪而将真实目标错误剔除。为此，首先确立交会凸多边形包含真实目标的数学命题。

命题 3（交会凸多边形恒包含目标真实位置） 在任意步 $k \geq 0$ ，算法维护的凸外包多边形恒满足 $\mathbf{s}_c \in X_{c,k} \subseteq K_{c,k}^+$ 。

证明：初始外切多边形满足 $\mathbf{s}_c \in \mathcal{P}_0$ 。假设第 k 步包含性成立；当系统接收到新方位角 $\hat{\theta}$ 时，由测向误差界 $|\hat{\theta} - \theta^*| \leq 1^\circ$ 可知，目标真值必位于对应角楔 $\mathcal{W}$ 内。因此，更新后的外包 $K_{c,k+1}^+ = K_{c,k}^+ \cap \mathcal{W}$ 仍包含 $\mathbf{s}_c$ 。若定位阶段返回 no_signal，算法不从当前凸外包中删除区域，而是停止主动测向并进入末端直接清除判定或网格兜底，故负观测不会破坏真值包含性。由数学归纳法可知命题得证。 ■

在确立凸外包多边形 $K_{c,k}^+$ 恒包真值后，机器狗需根据外包尺度选择直接清除或网格兜底。激光清除的有效半径为 20 m；为降低边界浮点误差对常规定位流程的影响，本文将 19.5 m 设为提前清除阈值，并保留 20 m 作为主动测向预算用尽后的严格末端判定。若定位区域仍未收敛，则通过有限网格遍历完成兜底清除。记 $K_{c,k}^+$ 的最小外接圆 (MEB) 半径为 $r(K_{c,k}^+)$ ，圆心为 $\mathbf{c}(K_{c,k}^+)$ 。

判据 1（最小外接圆直接清除与有限网格兜底） 针对频道 c 的凸外包 $K_{c,k}^+$ ，采用以下分层判定：

1. 常规定位提前清除：若 $r(K_{c,k}^+) \leq 19.5 \text{ m}$ ，则移动至圆心 $\mathbf{c}(K_{c,k}^+)$ 实施清除。由于 $\mathbf{s}_c \in K_{c,k}^+$ ，有

$$\|\mathbf{s}_c - \mathbf{c}(K_{c,k}^+)\|_2 \leq r(K_{c,k}^+) \leq 19.5 \text{ m} < 20 \text{ m}. \quad (33)$$

其中 0.5 m 是相对于法定清除半径预留的数值稳健裕量，而非额外的物理到位误差。

2. 测向预算结束后的直接清除：主动测向预算用尽或无法继续生成安全候选点时，重新计算最小外接圆。若 $r(K_{c,k}^+) \leq 20 \text{ m}$ ，则移动至其圆心实施清除。
3. 有限网格兜底清除：若仍有 $r(K_{c,k}^+) > 20 \text{ m}$ ，则将外接水平矩形剖分为边长 $\Delta g = 26 \text{ m}$ 的正方形网格，并遍历所有与 $K_{c,k}^+$ 相交的单元中心。每个单元中心到单元内任意点的最大距离满足

$$d_{\max} \leq \frac{\sqrt{2}}{2} \Delta g \approx 18.385 \text{ m} < 20 \text{ m}. \quad (34)$$

因此，至多经过 $M_c = \lceil W_c / \Delta g \rceil \lceil H_c / \Delta g \rceil$ 个有限网格单元即可完成兜底清除。

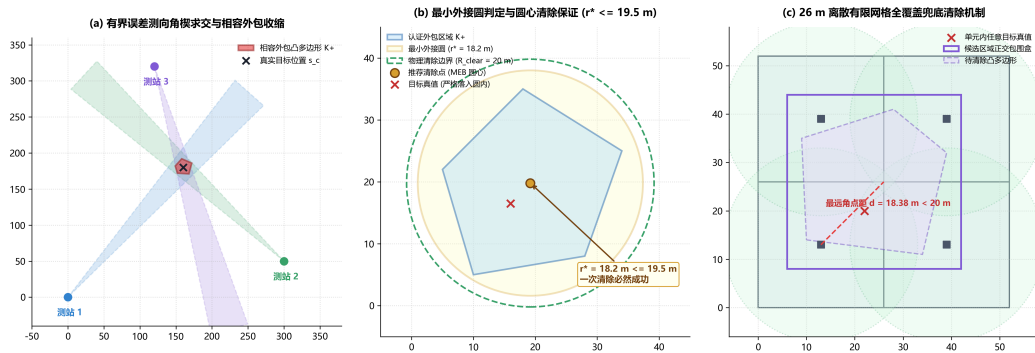


图 8 角楔交会、MEB 判据与网格兜底机制示意图

6.3.3 全局终止判定

在完成各个被探测目标的逐一清除后，系统面临的关键决策在于：在外部环境不提供剩余存活目标总数的情况下，机器狗何时可以确信全场已绝对无任何存活目标，从而自主决断终止任务？若盲目提前退出将造成漏检，若无止境游弋则会导致时间超时。为此，依据各频道追踪状态单向转移且不回退的规律，建立全局任务自主退出的充分判定条件。记已清除频道集为 $\mathcal{C}_k^{\text{cl}}$ ，已证实空置频道集为 $\mathcal{C}_k^{\text{abs}}$ 。

准则 2（全场无残留自主退出条件） 若系统无未决通信事务，且满足下列双重判定之一：

$$|\mathcal{C}_k^{\text{cl}}| = 16 \quad \text{或} \quad \mathcal{C}_k^{\text{cl}} \cup \mathcal{C}_k^{\text{abs}} = \mathcal{C} \quad (35)$$

则执行退出动作 **exit** 终止任务。此时全场所有客观存在的真实干扰源已全部被清除，无一遗漏 ($\eta_{\text{clear}} = 1$)。

证明： 真实目标总数不超过 16，若清除 16 个目标则全场无残留；若 $\mathcal{C}_k^{\text{cl}} \cup \mathcal{C}_k^{\text{abs}} = \mathcal{C}$ ，因全体 20 频道均处于清除或已证空置的吸收态，全场无任何遗漏存活目标，清除率严格为 100%。 ■

6.4 定位与调度策略

为缩短全任务时间，系统在全局层面滚动选择“继续访问未完成的七点基准站”或“处理已发现的存活频道”。一旦选中存活频道，频道内部流程再依次执行提前清除判定、主动定位、末端直接清除判定与网格兜底。该两层结构与实际程序的任务组织方式一致。

6.4.1 安全检测点选择

针对存活且尚未满足提前清除条件的频道，算法沿用问题二的保守候选生成与有限状态评价方法。候选点 $\mathbf{b}$ 必须满足

$$\max_{\mathbf{x} \in K_{c,k}^+} \|\mathbf{b} - \mathbf{x}\|_2 \leq 995 \text{ m}, \quad (36)$$

从而在最小接收半径为 1000 m 时仍保留 5 m 的数值安全余量。记 $\hat{D}_c(\mathbf{b})$ 为 30 个代表目标位置与 3 个误差节点下得到的离散最坏后验直径， $\mathbf{c}_c$ 为当前外包的最小外接圆圆心，并定义

$$L_c(\mathbf{b}) = \|\mathbf{p}_k - \mathbf{b}\|_2 + \|\mathbf{b} - \mathbf{c}_c\|_2, \quad E_c(\mathbf{b}) = \max\{0, \hat{D}_c(\mathbf{b}) - 39 \text{ m}\}. \quad (37)$$

算法按照下列四元组作字典序最小化：

$$\left(L_c(\mathbf{b}) + 0.85E_c(\mathbf{b}), \hat{D}_c(\mathbf{b}), L_c(\mathbf{b}), -m_c(\mathbf{b}) \right), \quad (38)$$

其中 $m_c(\mathbf{b}) = 995 \text{ m} - \max_{\mathbf{x} \in K_{c,k}^+} \|\mathbf{b} - \mathbf{x}\|_2$ 为保守接收余量。离散粗筛后，算法选取排序最优的 3 个候选作为种子，分别进行 3 轮八方向局部精修。该评价同时考虑测量后前往当前目标中心的机动距离和超出 39 m 目标直径的后验不确定性。

6.4.2 滚动任务调度

在全局层面，记未访问基准站任务与存活频道任务的并集为 $\mathcal{T}_k = \mathcal{T}_k^{\text{search}} \cup \mathcal{T}_k^{\text{source}}$ 。对任务 τ 定义目标位置 $\mathbf{q}_\tau$ 、类别优先级 π_τ 与稳定索引 i_τ ，其中存活频道取 $\pi_\tau = 0$ ，基准站取 $\pi_\tau = 1$ 。系统采用

$$\tau_k = \arg \min_{\tau \in \mathcal{T}_k} (\|\mathbf{p}_k - \mathbf{q}_\tau\|_2, \pi_\tau, i_\tau) \quad (39)$$

作字典序选择。因此，系统通常优先执行距离当前位置最近的任务；当距离相同时，先处理已发现目标，再访问基准站。清除、主动定位和网格兜底均在选中的频道内部按判据 1 顺序执行，不另设与实现不一致的固定权重评分。

6.5 算法实现与分析

6.5.1 在线控制流程

综合状态演化、完全清除条件检验与滚动调度，多全向干扰源在线搜索、定位与清除闭环状态机流程如图 9 所示，整体在线自主控制主算法如算法 1 所示。

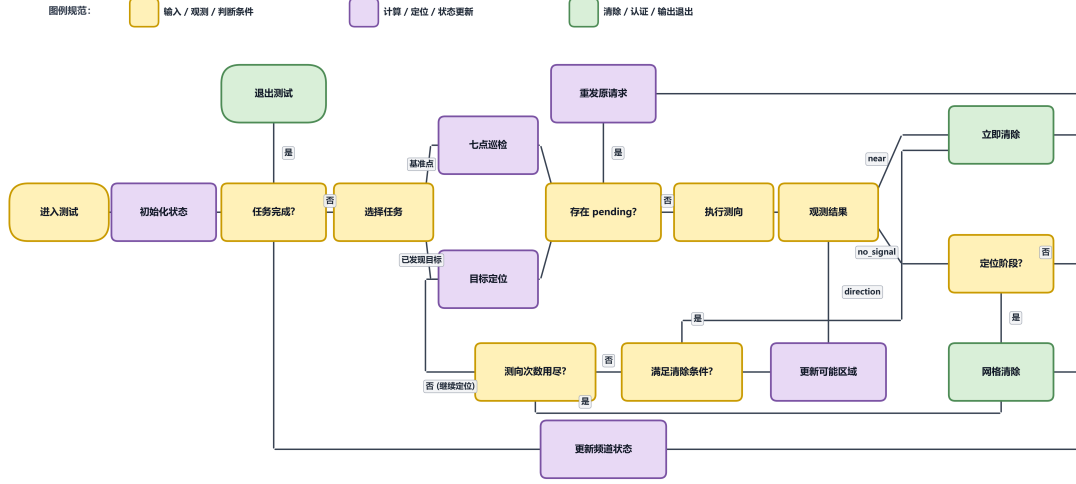


图 9 多全向干扰源在线搜索、定位与清除闭环状态机流程

系统为每个动作请求附加唯一标识编号。若遇到未决状态（pending），则暂停派发新任务，并使用相同载荷和相同请求编号重发原动作。程序最多重试 $N_{\text{retry}} = 2$ 次，即包含首次请求时单个动作最多发送 3 次；若仍无法恢复，则停止继续决策且不签发完成证书，避免把响应未知的动作误记为已完成。

表 9 算法 1：未知多干扰源在线协同搜索、定位与清除主算法

输入：区域半径 $R_{\text{target}} = 1800 \text{ m}$ ，感知下限 $R_{\text{min}} = 1000 \text{ m}$ ，误差界 $\varepsilon = 1^\circ$ ，频道集 $\mathcal{C} = \{1, \dots, 20\}$

输出：物理动作执行序列与全场完备清除报告

- 1: 初始化：状态 $\mathbf{p}_0 = (0, 0)^\top$ ， $h_0 = 1$ ， $T_0 = 0$ ， $\forall c \in \mathcal{C}$ 设 $L_{c,0} = \mathcal{S}_{\text{unk}}$ ， $K_{c,0}^+ = \mathcal{P}_0$ ，构建 $\mathcal{Q}_7$
- 2: **while** 终止条件不满足 **do**
- 3: 若存在未决事务，重新发送动作指令并跳至第 8 步；
- 4: 检验准则 2 终止条件：若 $|\mathcal{C}_k^{\text{cl}}| = 16$ 或 $\mathcal{C}_k^{\text{cl}} \cup \mathcal{C}_k^{\text{abs}} = \mathcal{C}$ ，派发 $a_k = \text{exit}$ 终止主循环；
- 5: 在未访问基准站与存活频道之间，按“机动距离、任务类别、稳定索引”的字典序选择下一任务；
- 6: 若选中存活频道，先检验 $r \leq 19.5 \text{ m}$ ；否则至多进行 4 次主动测向，随后按 $r \leq 20 \text{ m}$ 直接清除或转入 26 米网格兜底；
- 7: 若选中基准站，则移动至该站并依次扫描尚未结案的频道；对返回的 `direction`、`near` 与 `no_signal` 分别更新外包、就地清除或记录缺席证据；
- 8: 将已确认的动作反馈追加至历史序列，更新物理状态、频道状态和完成证书所需证据；
- 9: **end while**
- 10: **return** 任务完备达成，输出执行动作序列与统计结果

6.5.2 有限步终止性分析

为验证算法 1 在实际调度执行中不存在死循环与死锁，下面结合七点覆盖、真值包含与退出准则，证明该搜索清除过程必在有限个离散动作步内终止：

命题 4（搜索清除过程有限步终止） 在目标静止、测向误差满足给定确界、七点搜索网完整执行且每个已发出的通信事务均能在有限次重试内获得确定响应的条件下，算法 1 可在有限离散动作步数内完成全部实际目标的清除并安全退出。

证明：基准搜索至多包含 $7 \times 20 = 140$ 个互不重复的检测动作，每个存活目标的主动测向次数受预算 $N_{loc_max} = 4$ 限制。若目标在该预算内仍未满足直接清除条件，算法转入有限网格兜底；由于未访问单元数随清除尝试单调减少，该过程必在有限步内结束。通信事务按相同请求编号进行至多 2 次重试，并由确定响应驱动状态转移；各频道状态只会由未探明转为已发现、确证空置或已清除。因此，系统最终在有限步内满足准则 2 并退出。 ■

计算复杂度与执行时间：设当前凸外包的极点数为 h ，候选点数量为 K ，代表目标位置数量为 S ，误差节点数量为 E 。单个候选点的离散后验评价复杂度为 $O(SEh)$ ，一轮候选评价的总复杂度为 $O(KSEh)$ ；半平面裁剪和最小外接圆计算分别随当前极点数线性或期望线性增长。三次正式测试的程序运行时间为 $3.180 \sim 4.862$ s，平均为 4.215 s，远低于 1200 s 的程序运行时限。

6.6 测试结果与分析

为检验当前完整策略在重复随机场景中的执行稳定性，本文使用本地模拟器对固定种子 0 至 99 的 100 个场景逐一运行。该批结果取自当前默认参数下的结构化批处理输出，种子区间与各项统计字段均保留在支撑材料中。该实验属于本地重复测试，与后文三次官方模拟器正式测试分开报告。

6.6.1 本地重复测试

100 个场景均完成全部目标清除，且未出现失败清除。任务时间与程序运行时间的统计结果见表 10。

表 10 固定种子 0 至 99 的 100 组本地重复测试结果

统计指标	结果
完成全部目标清除的场景数	100/100
全场景目标清除率	100%
虚拟任务总时间均值	3373.913 s
虚拟任务总时间标准差（样本标准差）	315.839 s
虚拟任务总时间最大值	4340.883 s
单源平均定位清除时间的场景均值	269.162 s/源
程序运行时间均值	3.091 s
程序运行时间最大值	5.239 s
失败清除总次数	0

该批 100 场结果表明，当前策略在所选本地场景中保持了完整清除和零失败清除。虚拟任务时间仍存在约 315.839 s 的样本标准差，说明目标数量与空间分布会显著影响机动代价；因此，本批结果用于描述当前测试分布下的有限样本表现，不外推为任意分

布下的总体性能。

6.6.2 正式测试结果

在确定关键算法参数（单目标主动测向次数上限 $N_{\text{loc_max}} = 4$ ，提前清除阈值 $r^* = 19.5 \text{ m}$ ，末端直接清除阈值 20 m ，网格边长 $\Delta g = 26 \text{ m}$ ）后，本文在官方模拟器中完成三次正式测试。按照题目规定的字段，测试结果汇总于表 11。

表 11 基于赛题规范模拟环境的三次测试结果汇总表

正式测试案例编码	清除数量	平均定位清除时间/s	程序运行时间/s
2T4W-AQ2B-BFJ9-DETS	11 个	249.178	3.180
VC2X-UWRN-NBUP-2X6V	14 个	241.330	4.603
M9X2-6D5X-E84D-5AEH	14 个	240.549	4.862

三次测试的虚拟任务总时间分别为 2740.959 s 、 3378.626 s 与 3367.691 s ，算术平均为 3162.425 s 。三次测试共清除 39 个目标，按目标数量汇总后的加权平均定位清除时间为 243.263 s/源 ，程序运行时间均值为 4.215 s 。测试结果与理论机制分析如下：

1. **完全清除与清除分支：**三次测试清除率均达到 100%。39 个目标中，36 个通过外接圆心清除，3 个在返回 **near** 后就地清除；全部 39 次清除请求均首次执行成功，未发生失败清除；
2. **退出条件触发规律一致：**三次测试均通过蜂窝拓扑扫描与动态状态账本，在全域 20 个频道的信号状态均确立为已清除 (**cleared**) 或已认证空置 (**absent_certified**) 时触发准则安全退出，与理论推导吻合；
3. **时间合规性：**最大虚拟任务时间为 3378.626 s ，低于模拟器规定的 360000 s 虚拟时间上限；程序运行时间为 $3.180 \sim 4.862 \text{ s}$ ，均远低于 1200 s 的程序运行上限。

7 问题四：混合干扰源的搜索定位与清除

7.1 问题分析

问题四在问题三的基础上，进一步引入了半平面定向辐射干扰源。各干扰源独占单一频道的物理规律保持不变，目标总数仍处于 $10 \sim 16$ 个之间。但定向干扰源的具体数目及其辐射主瓣朝向均完全未知。其余运动速度、调谐与测向耗时、测向角度误差 $[-1^\circ, 1^\circ]$ 、以及 20 m 激光清除等条件与前述问题完全一致。

相比全向辐射，定向辐射带来了三个新的难点：

1. **半平面定向辐射导致无信号成因复杂：**定向源仅在其辐射朝向所确定的 180° 闭半平面内发射信号。测站未收到信号可能由于频道空置、目标已清除、空间超距、或测站处于发射盲侧。因此，单次无信号不能直接排除对应几何区域。
2. **单测站无法保证发现，需多测站协同包围：**全向目标只要距离小于 1000 m 即可被单测站探测；而定向目标若背对测站，即便距离很近也无法感知。要发现未知朝向的目标，必须依靠多个测站从不同方位对其形成几何包围。
3. **追加测向耗时与网格遍历成本的平衡：**初次发现目标后，若盲目追加测向可能再次落入盲侧而徒劳失联；若完全依赖网格遍历，时间成本又过高。因此需要在基准搜索之外，建立轻量自适应决策逻辑。

本文首先构建由 25 个测站组成的双环网络 [9]，利用凸包包围性质确保任意未知朝向的目标均能被可靠发现。在此基础上，进一步引入安全距离选点、对称镜像探测和自适应尾部探测，在确保完全清除的前提下尽量减少机动与网格遍历时间。

7.2 模型建立

7.2.1 目标物理状态

系统共有 20 个离散频道，记为 $\mathcal{C} = \{1, 2, \dots, 20\}$ 。对客观存在目标的频道 ($e_c = 1$)，其物理状态由五元组表征：

$$\mathbf{X}_c = (e_c, \mathbf{x}_c, R_c, \tau_c, \mathbf{u}_c), \quad c \in \mathcal{C}, e_c = 1 \quad (40)$$

式中各参数含义如下：

1. $e_c \in \{0, 1\}$ 表示频道 c 是否存在物理目标，目标总数满足 $N = \sum_{c=1}^{20} e_c \in [10, 16]$ ；
2. $\mathbf{x}_c = (x_{c,1}, x_{c,2})^\top \in \Omega$ 为目标二维平面坐标；
3. $R_c \in [1000, 1500]$ m 为该目标的信号覆盖半径；
4. $\tau_c \in \{\text{O}, \text{D}\}$ 表示辐射类型，O 为全向辐射，D 为半平面定向辐射；
5. $\mathbf{u}_c = (\cos \phi_c, \sin \phi_c)^\top$ 为定向天线的法向辐射单位向量（全向目标不受该向量约束）。若频道无目标 ($e_c = 0$)，则后序分量不予定义。

为刻画作业过程中目标的清除过程，引入清除变量 $h_c^k \in \{0, 1\}$ （初始为 0，清除后置为 1），定义时步 k 处的动态活动变量：

$$a_c^k = e_c(1 - h_c^k) \in \{0, 1\} \quad (41)$$

当机器狗在目标 20 m 范围内成功清除目标后， h_c^k 置为 1， a_c^k 变为 0。

7.2.2 定向接收与观测模型

对于存活目标 ($a_c^k = 1$)，定义其在检测位置 $\mathbf{q} = (q_1, q_2)^\top$ 处的几何可接收条件：

$$\mathcal{V}_c(\mathbf{q}) = [\|\mathbf{q} - \mathbf{x}_c\| \leq R_c] \wedge [\tau_c = \text{O} \vee \mathbf{u}_c^\top(\mathbf{q} - \mathbf{x}_c) \geq 0] \quad (42)$$

包含法向分界线 $\mathbf{u}_c^\top(\mathbf{q} - \mathbf{x}_c) = 0$ 在内的平角区域均可正常通信；若测站与目标重合 ($\mathbf{q} = \mathbf{x}_c$)，约定处于有效视场。

为清晰厘清无信号反馈的成因，将其分解为四类互斥物理事件：

1. $E_0 = \{e_c = 0\}$ ：该频道本就未布设目标；
2. $E_1 = \{e_c = 1, h_c^k = 1\}$ ：该频道目标此前已被成功清除；
3. $E_2 = \{a_c^k = 1, \|\mathbf{q} - \mathbf{x}_c\| > R_c\}$ ：目标存活但距离超出接收范围；
4. $E_3 = \{a_c^k = 1, \|\mathbf{q} - \mathbf{x}_c\| \leq R_c, \tau_c = \text{D}, \mathbf{u}_c^\top(\mathbf{q} - \mathbf{x}_c) < 0\}$ ：目标存活且在距离范围内，但测站处于定向天线的发射盲侧。

四类事件两两互斥，其并集构成无信号事件 $E_{\text{none}} = E_0 \cup E_1 \cup E_2 \cup E_3$ 。

测站在 $\mathbf{q}$ 处对频道 c 的实测反馈 $y_c^k(\mathbf{q})$ 严格划分为三个互斥分支：

$$y_c^k(\mathbf{q}) = \begin{cases} \text{no_signal}, & E_0 \vee E_1 \vee E_2 \vee E_3, \\ \text{near}, & a_c^k = 1 \wedge \mathcal{V}_c(\mathbf{q}) \wedge \|\mathbf{q} - \mathbf{x}_c\| \leq 5 \text{ m}, \\ \text{direction}(\hat{\theta}_c), & a_c^k = 1 \wedge \mathcal{V}_c(\mathbf{q}) \wedge \|\mathbf{q} - \mathbf{x}_c\| > 5 \text{ m} \end{cases} \quad (43)$$

当返回示向角 $\hat{\theta}_c$ 时，真实方位角 $\theta_c^* = \text{atan2}(x_{c,2} - q_2, x_{c,1} - q_1)$ 与实测示向角满足圆周

角距离误差约束：

$$d_{S^1}(\theta_c^*, \hat{\theta}_c) = |\text{wrap}_{[-180^\circ, 180^\circ]}(\theta_c^* - \hat{\theta}_c)| \leq \varepsilon = 1^\circ \quad (44)$$

为防止浮点截断误差在半平面求交时误删边界解，交会计算时采用容差 $\varepsilon_{\text{tol}} = 1.005^\circ$ 。作业全流程中各测量分支实行统一规则：若检测返回 **near**（表明目标已处于 $5 \text{ m} \leq 20 \text{ m}$ 清除区内），机器狗直接在当前停靠位置实施激光清除，并置该频道状态为已清除。

7.2.3 频道状态与时间模型

机器狗针对每个频道 $c \in \mathcal{C}$ 独立维护五元追踪状态：

$$\mathcal{I}_c^k = (s_c^k, P_c^k, \mathcal{M}_c^k, \mathcal{V}_c^{\text{hist}}, \mathcal{B}_c^{\text{hist}}) \quad (45)$$

包括离散状态 $s_c^k \in \{\text{未探明}, \text{已发现}, \text{定位中}, \text{网格遍历}, \text{已清除}, \text{确证空置}\}$ ；紧致凸多边形位置外包 $P_c^k \subset \mathbb{R}^2$ （初始为目标圆外切正四十八边形）；25 个基准测站的已测集合 $\mathcal{M}_c^k \subseteq \{1, \dots, 25\}$ ；以及历史正观测测站集 $\mathcal{V}_c^{\text{hist}}$ 与盲侧测站集 $\mathcal{B}_c^{\text{hist}}$ 。

虚拟物理总时间由位移、天线调谐、停靠检测与光学清除耗时累加确定：

$$T_{\text{virtual}} = \frac{L}{v} + 5N_{\text{meas}} + 1N_{\text{switch}} + 3N_{\text{clear}}^{\text{fail}} + 5N_{\text{clear}}^{\text{succ}} \quad (46)$$

式中速度 $v = 5 \text{ m/s}$ ； N_{meas} 为检测总次数（每次 5 s）； N_{switch} 为切频次数（每次 1 s）；清除成功耗时 5 s，未命中耗时 3 s。程序算法决策总耗时满足赛题规程约束 $T_{\text{wall}} \leq 20 \text{ min}$ 。

7.3 完全清除条件

要实现对未知混合干扰源的完全清除，核心障碍在于半平面定向辐射导致的背向无信号盲区。若仅凭经验搜索，定向源极易因持续处于盲侧而被漏检，或在初次发现后因盲目机动而再次失联。为此，本节按照全域发现、交会定位、局部清除与全局退出的递进逻辑，建立完备的几何判定准则，为后续在线协同算法提供严密的理论支撑。

7.3.1 多视角覆盖与 25 点双环搜索网

针对半平面定向天线单侧发射的物理约束，首先建立测站分布能否探测到未知朝向目标的几何条件：

命题 5（测站凸包对半平面定向目标的探测条件） 设测站集合为 $\mathcal{S}$ ，信号最大接收半径为 r 。对于目标位置 $\mathbf{x}$ ，记其覆盖范围内的测站子集为：

$$\mathcal{S}_r(\mathbf{x}) = \{\mathbf{q} \in \mathcal{S} \mid \|\mathbf{q} - \mathbf{x}\| \leq r\} \quad (47)$$

在 180° 闭半平面辐射模型下，目标在任意未知朝向 $\mathbf{u}$ 下均至少能被一个测站探测到的充要条件为：

$$\mathbf{x} \in \text{conv}(\mathcal{S}_r(\mathbf{x})) \quad (48)$$

即目标位置落入局部有效测站集的闭凸包内部。

证明：(1) **充分条件证明：**若 $\mathbf{x} \in \text{conv}(\mathcal{S}_r(\mathbf{x}))$ ，存在一组权值 $\lambda_i \geq 0$ 且 $\sum_{i=1}^m \lambda_i = 1$ ，使得：

$$\mathbf{x} = \sum_{i=1}^m \lambda_i \mathbf{q}_i, \quad \mathbf{q}_i \in \mathcal{S}_r(\mathbf{x}) \quad (49)$$

对任意单位辐射方向向量 $\mathbf{u}$ ，两端作内积可得：

$$\sum_{i=1}^m \lambda_i \mathbf{u}^\top (\mathbf{q}_i - \mathbf{x}) = \mathbf{u}^\top \mathbf{0} = 0 \quad (50)$$

因为 $\lambda_i \geq 0$ 且和为 1，各项内积不可能同时严格小于 0。因此必存在某个测站 $\mathbf{q}_k \in \mathcal{S}_r(\mathbf{x})$ 满足：

$$\mathbf{u}^\top(\mathbf{q}_k - \mathbf{x}) \geq 0 \quad \text{且} \quad \|\mathbf{q}_k - \mathbf{x}\| \leq r \quad (51)$$

这说明测站 $\mathbf{q}_k$ 处于目标发射半平面内且在通信距离内，目标必能被有效探测。

(2) 必要条件证明：若 $\mathbf{x} \notin C = \text{conv}(\mathcal{S}_r(\mathbf{x}))$ ，因 $\mathcal{S}_r(\mathbf{x})$ 为有限点集， C 为平面有界闭凸集。由严格分离超平面定理，存在非零向量 $\mathbf{v}$ 使得：

$$\mathbf{v}^\top(\mathbf{q} - \mathbf{x}) < 0, \quad \forall \mathbf{q} \in C \quad (52)$$

取目标辐射朝向 $\mathbf{u} = \mathbf{v}/\|\mathbf{v}\|$ ，则所有局部测站均满足：

$$\mathbf{u}^\top(\mathbf{q}_i - \mathbf{x}) < 0, \quad \forall \mathbf{q}_i \in \mathcal{S}_r(\mathbf{x}) \quad (53)$$

此时距离在 r 以内的测站全部落入目标发射盲侧，超出 r 的测站又因超距无法接收，故不存在可探测测站。证毕。 ■

推论 1 (保底接收半径下的全向发现充分条件) 针对覆盖半径确界 $R_c \in [1000, 1500]$ m，若在保底半径 $R_{\min} = 1000$ m 下满足 $\mathbf{x} \in \text{conv}(\mathcal{S}_{1000}(\mathbf{x}))$ ，则是目标在全工况、全方向下均能被发现的统一充分条件。

根据命题 5 与推论 1，在目标区域 Ω 内设计由 25 个测站构成的双环三角形测站网：坐标原点 (0,0) 设置 1 个中心测站；内环在半径 $R_1 = 950$ m 圆周上布设 12 个测站，极角为 $i \times 30^\circ$ ；外环在半径 $R_2 = 1870$ m 圆周上布设 12 个测站，交错旋转 15° （极角为 $j \times 30^\circ + 15^\circ$ ）。几何构型如图 10 所示，参数验证汇总于表 12。

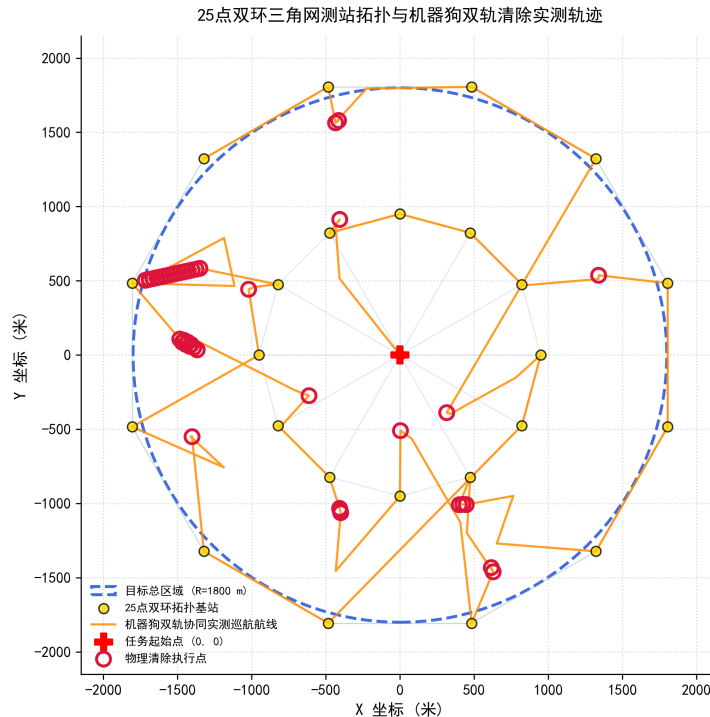


图 10 25 点双环拓扑分布构型与机器狗双轨清除实测轨迹

该网络共划分出 36 个三角形单元，单元边长涵盖四类连线：

1. 中心至内环边长： $d_1 = R_1 = 950$ m；
2. 内环相邻边长： $d_2 = 2R_1 \sin 15^\circ \approx 491.76$ m；

3. 外环相邻边长: $d_3 = 2R_2 \sin 15^\circ \approx 968.02 \text{ m}$;

4. 内外环跨接边长: $d_4 = \sqrt{R_1^2 + R_2^2 - 2R_1R_2 \cos 15^\circ} \approx 983.60 \text{ m}$ 。

所有单元的最长边满足 $d_{\max} = 983.60 \text{ m} < 1000 \text{ m}$ 。因此, 区域内任意点所在的三角形三顶点, 到该点的距离均小于 1000 m , 且构成包含该点的闭凸包, 满足推论 1 的可探测条件。此外, 外环正十二边形内切圆半径为 $R_2 \cos 15^\circ \approx 1806.28 \text{ m} > 1800 \text{ m}$, 实现了对目标区域的完全覆盖。

表 12 25 点双环拓扑几何参数与方向完备验证表

几何拓扑参数项	解析表达式 / 理论值	数值计算结果	全域探测保证判定结论
外环正十二边形内切圆半径	$R_2 \cos(15^\circ) = 1870 \times 0.9659$	1806.28 m	大于目标圆半径 1800 m, 实现全域几何包络
剖分三角形最大边长 $d_{\max}$	$\sqrt{R_1^2 + R_2^2 - 2R_1R_2 \cos(15^\circ)}$	983.60 m	小于保底半径 1000 m, 具备 16.40 m 探测裕度
三角形单元几何顶点测距	任意内点至三顶点距离均 $\leq d_{\max}$	$\leq 983.60 \text{ m}$	每个单元内部点均能被其所
未知朝向多视角凸包判定	目标恒处于所在三角形三顶点闭凸包内	满足命题 5 条件	在三角形 3 顶点有效覆盖 任意未知朝向至少落入一个 顶点的有效视场内

7.3.2 真值包含与安全选点

25 点双环拓扑在宏观上保证了全域任意未知朝向的目标至少能被一个基准测站捕获。而在实际作业中, 一旦机器狗在某一测站 $\mathbf{q}_1$ 处首次捕获频道 c 的有效示向信号, 系统便需要由全域巡航转入对该频道的交会定位。为确保后续多测站协同交会不丢失目标, 首先必须保证交会多边形始终包含真实物理位置, 且后续选点能够排除距离超距的混淆。

当在测站 $\mathbf{q}_1$ 处首次捕获频道 c 的有效示向角 $\hat{\theta}_1$ 时, 构建初始凸多边形位置外包:

$$P_c^1 = P_\Omega^{\text{out}} \cap \mathcal{B}^{\text{out}}(\mathbf{q}_1, 1500) \cap \mathcal{W}(\mathbf{q}_1, \hat{\theta}_1, \varepsilon_{\text{tol}}) \quad (54)$$

式中 P_Ω^{out} 为目标区域的外切正多边形, $\mathcal{B}^{\text{out}}$ 为测站最大覆盖圆的外切多边形, $\mathcal{W}$ 为示向角楔区域。在确定性测向误差界 $\varepsilon = 1^\circ$ 条件下, 利用示向角楔持续求交可以不断压缩目标可行域, 该多边形恒包含目标真实位置, 具体由如下引理保证:

引理 1 (示向角交会多边形恒包含目标真实位置) 设初始外包满足 $\mathbf{x}_c \in P_c^1$ 。后续若仅采用有效示向角楔 $\mathcal{W}_k$ 进行半平面求交更新:

$$P_c^{k+1} = P_c^k \cap \mathcal{W}_k \quad (55)$$

则对任意步数 $k \geq 1$, 均恒有 $\mathbf{x}_c \in P_c^k$ 。

引理 1 保证了只要测站获得有效示向角, 交会多边形便能稳健收缩并锁定真实目标。然而, 仅凭单次测向获得的角楔面积依然巨大, 必须机动至第二测站进行双向交会。在定向天线半平面发射背景下, 若第二测站与目标的距离超出覆盖半径 R_c , 不仅无法接收信号, 且系统无法区分未收到信号究竟是因为距离超距还是落入目标盲区。因此, 第二检测点必须满足严格的距离安全准则, 使无信号反馈不再受到距离超限因素的混淆。对于候选测站 $\mathbf{q}$, 该准则要求其到多边形 P_c^k 内任意可能目标位置的距离均不超过安全阈值。由于欧氏距离关于目标位置 $\mathbf{x}$ 为凸函数, 上述最大距离可在顶点集 $\text{Vert}(P_c^k)$ 上取得:

$$d_{\max}(\mathbf{q}, P_c^k) = \max_{\mathbf{x} \in P_c^k} \|\mathbf{q} - \mathbf{x}\| = \max_{\mathbf{v} \in \text{Vert}(P_c^k)} \|\mathbf{q} - \mathbf{v}\| \quad (56)$$

算法筛选满足 $d_{\max}(\mathbf{q}, P_c^k) \leq 995 \text{ m}$ 的候选测站。由于 $995 \text{ m} < 1000 \text{ m} \leq R_c$ ，这保证了测站对多边形内任意位置均在感知范围之内；此时若未收到信号，必为目标背向盲侧，而非距离超距。

7.3.3 直接清除与网格遍历

经过多测站交会收缩后，目标真实位置已被限定在紧致凸多边形 P_c^k 内部。此时系统需要驱动机器狗实施物理清除，机载激光的有效清除半径为 20 m 。若定位多边形已充分收缩，机器狗可停靠其最小外接圆圆心并直接清除目标；反之，若多边形仍然较大或主动测向预算已经耗尽，系统便转入无遗漏的网格遍历。为此，建立外接圆直接清除与正方形网格遍历相结合的双轨判定准则：

判据 2（外接圆直接清除与正方形网格遍历条件） 针对频道 c 的多边形可行域 P_c^k ，采用 Welzl 算法计算其最小外接圆半径 $\rho(P_c^k)$ 及圆心 $\mathbf{c}_c$ ：

1. **常规定位提前清除：**若 $\rho(P_c^k) \leq 19.5 \text{ m}$ ，机动至 $\mathbf{c}_c$ 实施激光清除。其中 20.0 m 为题目规定的有效清除半径，采用 19.5 m 作为提前判定阈值，预留 0.5 m 数值稳健裕量；
2. **尾部探测后清除：**在完成最后一次尾部追加探测后，若后验外接圆半径满足充分清除界 $\rho(P_c^{k+1}) \leq 20.0 \text{ m}$ ，直接机动至新圆心实施清除；
3. **网格遍历清除：**若未达上述条件，对 P_c^k 的外接水平矩形作边长 $\Delta g = 26 \text{ m}$ 正方形网格划分，完整保留所有与 P_c^k 相交的单元，沿蛇形次序依次遍历各相交单元中心实施清除。

为证明网格遍历策略在几何上绝不会发生遗漏，给出如下单元覆盖保证：

引理 2（相交正方形网格遍历对目标的覆盖保证） 边长为 $\Delta g = 26 \text{ m}$ 的正方形网格单元中，中心点至单元内任意点的最大距离为：

$$r_{\text{cell}} = \frac{\sqrt{2}}{2} \Delta g \approx 18.385 \text{ m} < 20.0 \text{ m} \quad (57)$$

因该距离严格小于激光清除半径，机器狗只需停靠各相交网格中心执行清除，即可完全覆盖对应单元。只要目标位于相交网格并集内，有限步遍历必能完成清除。

7.3.4 全局终止判定

前述命题与引理保证了任意单个目标从被发现、定位到物理清除的完整过程。然而，在全自主作业背景下，全场客观存在的目标总数未知（在 10 至 16 之间随机），且机器狗并不知道具体哪些频道存在目标。为了避免盲目过早停工造成漏检，同时防止全场目标清除完毕后陷入无效空转，系统必须建立全局可判定的自主退出准则：

准则 3（混合干扰源全域搜索终止与退出条件） 当满足以下任一条件时，系统终止搜索任务并安全退出：

1. **目标数量达上限：**累计清除的目标频道数达到理论上限 $|\mathcal{C}^{\text{clear}}| = 16$ ；
2. **全频道检测完成：**所有已发现的目标均已完成清除，且其余未发现目标的频道在 25 个基准测站的检测中均明确返回 `no_signal`，即满足 $\forall c \in \mathcal{C}, c \in \mathcal{C}^{\text{clear}} \vee [\mathcal{M}_c = \{1, \dots, 25\} \wedge \forall i \in \{1, \dots, 25\}, y_{c,i} = \text{no_signal}]$ 。若通信超时或响应未知，不得计入无信号。

7.4 在线策略与算法

第 6.3 节建立的 25 点基准拓扑、距离安全交会与网格遍历构成了确保 100% 清除的理论保证主链。然而，若系统单纯依赖基准测站巡航与大范围网格遍历，一旦在第二测站遇到定向盲侧便直接退化为网格扫描，将消耗过多的机动与清除时间。因此，本文在理论保证主链之外增加两条轻量决策支路 [10]：盲侧失联时尝试对称镜像探测，定位多边形仍较大时评估尾部追加探测的预期收益。两项机制在不改变完全清除保证主链的前提下，用于显著压缩任务总时间。

7.4.1 镜像探测与自适应尾部探测

若测站对当前外包多边形满足安全距离条件 ($d_{\max} \leq 995 \text{ m}$)，但实测明确返回无信号，则可判定测站处于定向目标的辐射盲侧。针对该情况，设计两项优化机制：

1. **条件对称镜像探测**：若主动测向预算尚未耗尽，算法沿首次示向中轴线将失联测站对称反射至对侧几何位置：

$$\mathbf{q}_{\text{mirror}} = \mathbf{q}_1 + 2[(\mathbf{q}_2 - \mathbf{q}_1)^\top \mathbf{u}(\hat{\theta}_1)] \mathbf{u}(\hat{\theta}_1) - (\mathbf{q}_2 - \mathbf{q}_1) \quad (58)$$

重新校验顶点距离安全后前往探测，增加落入有效发射半平面的概率。

2. **自适应尾部追加探测**：在当前多边形内离散生成 96 个位置与 36 个法向朝向样本，筛选相容于历史观测的样本子集 $\mathcal{H}_k$ 。定义候选测站的相容状态比例：

$$p_v(\mathbf{q}) = \frac{\#\{(\mathbf{x}_i, \mathbf{u}_j) \in \mathcal{H}_k \mid \mathbf{q} \text{ 对该状态满足可接收条件}\}}{\#\mathcal{H}_k} \in [0, 1] \quad (59)$$

若 $\mathcal{H}_k = \emptyset$ ，则不触发追加探测。

综合上述收益与代价量化关系，自适应尾部追加探测的触发判据确立如下：

判据 3（尾部探测收益代价判定条件） 记网格遍历预估耗时为 T_{grid} ，候选点探测耗时为 $T_{\text{probe}} = d/v + 5 \text{ s} + \mathbb{I}_{\text{switch}} \times 1 \text{ s}$ ($\mathbb{I}_{\text{switch}}$ 为切频指示)。定义非空相交示向集以及探测后的后验最大可能直径：

$$\Theta_k^+(\mathbf{q}) = \{\theta \in \Theta_{\text{sample}} \mid P_c^k \cap \mathcal{W}(\mathbf{q}, \theta, \varepsilon_{\text{tol}}) \neq \emptyset\} \quad (60)$$

$$D_{\text{post}} = \max_{\theta \in \Theta_k^+(\mathbf{q})} \text{diam}(P_c^k \cap \mathcal{W}(\mathbf{q}, \theta, \varepsilon_{\text{tol}})) \quad (61)$$

若 $\Theta_k^+(\mathbf{q}) = \emptyset$ 则不触发探测。当相容比例满足 $p_v \geq 0.30$ ，且预估时间收益高于机动探测代价时，执行追加探测：

$$T_{\text{probe}} < T_{\text{grid}} \cdot p_v \left[1 - \left(\frac{D_{\text{post}}}{D} \right)^2 \right] \quad (62)$$

探测后的分支处理规则如下：

1. 若返回 **near**，直接在当前位置执行激光清除；
2. 若获取有效示向，更新位置外包。若后验外接圆半径满足 $\rho(P_c^{k+1}) \leq 20.0 \text{ m}$ 则直接清除，否则转入 26 米网格遍历；
3. 若再次返回 **no_signal** 或判据不满足，直接转入 26 米网格遍历清除。

7.4.2 系统架构与算法流程

系统协同架构如图 12 所示，全向与定向混合干扰源搜索、补充探测与清除流程如图 11 所示，算法流程汇总于表 13。

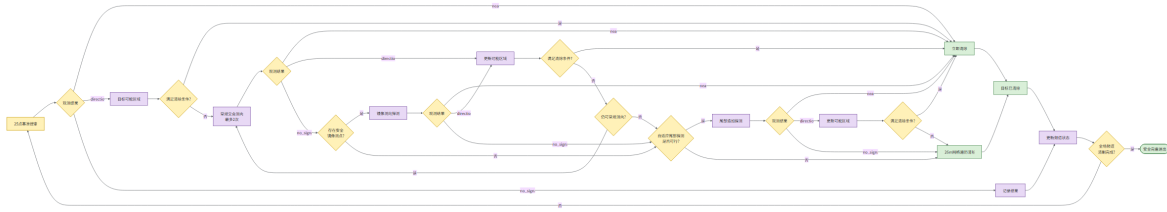


图 11 全向与定向混合干扰源搜索、补充探测与清除流程

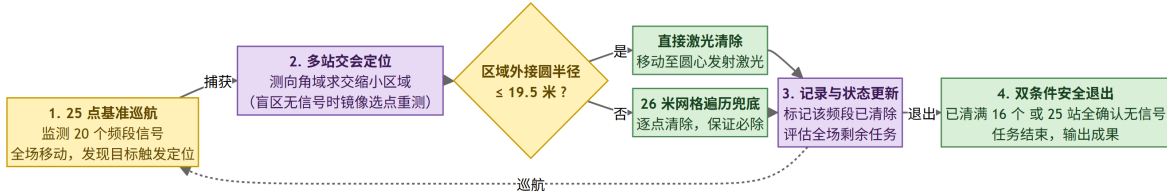


图 12 全向一定向混合干扰源自主协同搜索与清除系统架构图

表 13 混合干扰源多视角搜索与清除算法流程表

算法 2：基于多视角拓扑包围与自适应尾部探测的混合干扰源清除算法	
输入：	目标区域 Ω ，25 点双环拓扑 S_{25} ，物理动作时间模型，主动测向预算 $N_{loc_max} = 2$
输出：	机器狗动作序列，目标清除记录表，任务总虚拟时间 $T_{virtual}$
步骤 1	初始化位姿 $\mathbf{p}_0 = (0, 0)$ ，频道信息状态 $\mathcal{I}_c^0$ ，规划 25 点基准搜索测站巡航次序；
步骤 2	检查准则 3 退出条件：若满足则结束任务并输出报告，否则进入主调度循环；
步骤 3	优先检查是否存在满足直接清除条件频道 ($\rho \leq 19.5 \text{ m}$)：若存在则机动至圆心实施清除；
步骤 4	从主动测向次数少于 2 次的定位中频道中选择目标，并在满足 $d_{max} \leq 995 \text{ m}$ 的候选测站中确定下一测点。完成测向后，若返回 near ，则在当前位置直接清除；若获得有效示向，则求交裁剪并更新外包；若返回 no_signal ，则评估对称镜像测站；
步骤 5	对仍未收敛的频道评估判据 3。若满足尾部探测条件，则追加一次测向：返回 near 时直接清除；获得有效示向且 $\rho \leq 20.0 \text{ m}$ 时直接清除；其余情况转入网格遍历。若判据不满足，则直接遍历与当前外包相交的网格；
步骤 6	若无待处理定位频道，沿基准拓扑机动至下一基准测站，扫描未结案频道；
步骤 7	接收物理动作反馈，单向更新频道状态、凸多边形外包及 25 点检测记录，返回步骤 2；

7.4.3 完全清除与终止性分析

算法 2 将全域基准拓扑巡航、距离安全交会、自适应效率支路与网格遍历有机融为一体。为验证该系统在理论上绝不会发生死循环、漏检或无限震荡，下面结合引理 1、引理 2 与准则 3，证明该清除过程必在有限个离散动作步内终止：

命题 6（混合干扰源搜索清除过程有限步终止） 在目标静止、总数不超过 16 个且测向误差在确界内的物理假设下，算法 2 必在有限个离散动作步数内完成全部实际存在目标的清除并安全退出。

证明：在搜索清除任务全过程中，系统可生成的离散决策动作总量存在有限确定上界：

- 1. **基准搜索任务有限：**25 个基准测站对 20 个频道的巡航扫描总次数至多为 $25 \times 20 = 500$ 次；
- 2. **主动探测任务有限：**实际目标至多 16 个，每个目标常规主动测向至多 2 次、尾部探测至多 1 次；
- 3. **网格清除任务有限：**根据引理 1，目标真实位置恒包含于凸多边形外包内；由于外包多边形有界，其相交的 26 米正方形网格单元数必然有限，且每个单元中心至多访问清除 1 次。

算法按照固定优先级次序逐一推进待定位与待清除频道，每一步动作均单调消耗待处理任务集，不存在死循环与状态回退，因此算法必在有限离散步数内完成全部实际存在目标的清除并满足准则 3 退出条件。证毕。 ■

7.5 测试结果与分析

7.5.1 本地配对实验

为检验自适应尾部追加探测对缩减任务耗时的具体作用，在 100 组随机生成的同分布场景（随机种子 0 至 99）下开展同场景配对对比实验，对比未启用尾部探测的方案（V2）与本文完整方案（V3）。统计数据汇总于表 14。

表 14 V2 与 V3 算法在 100 组配对场景下的对比实验表

评估指标项	未启用尾部探测方案 (V2)	本文完整方案 (V3)	配对测试增益分析
全场景目标清除率	100% (100 / 100 场)	100% (100 / 100 场)	保持 100% 完备清除底线
全任务虚拟总时间均值	7323.1 s	7213.3 s	配对平均缩减 109.8 s
虚拟总时间 95% 置信区间	[7148.2, 7498.0] s	[7042.8, 7383.8] s	均值差配对区间 [-158.7, -60.9] s
样本最劣场景任务总时间	10094.6 s (种子 43)	9448.1 s (种子 43)	样本最劣工况耗时减少 646.5 s
单场平均无效清除尝试次数	46.78 次	27.99 次	无效网格遍历减少 40.2%
逐案例耗时胜平负统计分布	基准方案低 6 场	本文方案低 35 场	59 场耗时相同

实验数据表明，在 100 组配对样本中，两种方案目标清除率均达到 100%；V3 相对 V2 耗时降低、相同和升高的场数分别为 35、59 和 6；全场平均无效清除尝试由 46.78 次降至 27.99 次，样本集中最长耗时减少 646.5 s。

7.5.2 演练与正式测试结果

在接入比赛官方评测环境的连续批次中，算法进行了 17 轮全流程实测演练，检验在不同目标数目（涵盖 10 ~ 16 个各档位）下的运行指标，实测数据汇总于表 15。

实测表明，在累计 221 个未知全向与定向混合目标中，系统清除率达到 100%。按照赛题规范要求，针对未知混合干扰源开展三次正式测试，实测结果汇总于表 16。三次测

表 15 17 轮真实接口规范演练测试结果汇总表

演练测试轮次区间	测试场次	累计清除源数	全场清除率	加权平均单源耗时	单源耗时中位数
少目标工况（10 ~ 11 源）	4 场	42 个	100%	612.8 s/源	598.2 s/源
中目标工况（12 ~ 14 源）	8 场	104 个	100%	523.4 s/源	496.5 s/源
多目标工况（15 ~ 16 源）	5 场	75 个	100%	493.6 s/源	481.0 s/源
17 轮全演练总体汇总	17 场	221 个	100%	530.41 s/源	502.43 s/源

试共清除 38 个目标（清除率 100%），单源平均定位清除时间分别为 559.214 s、586.146 s 与 466.809 s，加权均值达 533.675 s/源。

表 16 赛题规范三次测试结果记录表

正式测试案例编码	清除目标数量	平均定位清除时间 / s	虚拟任务总时间 / s	程序算法运行时间 / s
4Z33-SF8B-2CGX-7UYM	12 个	559.214	6710.566	5.546
MAF3-M9C7-BEAZ-D6V9	12 个	586.146	7033.752	6.100
KKFA-B32Q-ZKTZ-ETUB	14 个	466.809	6535.328	6.046
汇总 / 加权均值	38 个	533.675	6759.882	5.897

8 模型评价、改进与推广

8.1 模型优点

1. 几何覆盖保障与数理推导：不依赖目标先验分布假设，基于平面几何凸包包含定理与极值分离性质，构建完备覆盖与真值包含关系；
2. 保证主链与效率支路协同分工：以基准多视角覆盖和有限网格遍历承担完备清除保证，以距离安全探测和尾部探测优化动作次序；
3. 计算效率与在线实测表现：Q3 三次正式测试的程序运行时间均值为 4.215 s，Q4 为 5.897 s，均远低于 1200 s 的程序运行上限；100 组本地重复测试、100 组 Q4 配对实验与正式测试共同记录了当前实现的运行表现。

8.2 模型局限与改进

1. 极端多径退化对多边形求交的数值扰动：当极端测量噪声导致测向角域畸变时，半平面求交可能出现微小退化多边形，需进一步增强近退化边界的自适应拓扑容差修复机制。
2. 通信异常恢复状态机仍需增强：当前模型假设通信偶发中断单次重试成功，若出现长时间信道离线且服务端事务未决，需进一步补充本地持久化恢复协议。

8.3 模型推广

本文建立的多视角凸包覆盖与误差集交会决策方法，可推广应用于：

1. 复杂电磁环境下的无人地面平台自主搜寻与干扰源定位；
2. 具有半平面辐射特性与角度测向约束的地面无线电信标协同搜索；
3. 有限视场角约束下的多移动机器人协同搜索与环境信标定位。

AI 工具使用声明

本参赛队在竞赛过程中使用了人工智能（AI）工具，主要用于论文排版语法检查、可视化绘图代码调试与部分文字措辞润色，详细使用情况见支撑材料《AI 工具使用详

情.pdf》。本论文的核心数学建模、公式推导、算法求解及数值分析均由本参赛队三位队员独立主导完成，并对全篇内容进行了完整的人工核验。

参考文献

- [1] GHOLAMI M R, GEZICI S, STRÖM E G. Worst-case positioning error and sensor placement for range-difference based localization[J]. IEEE Transactions on Signal Processing, 2013, 61(14): 3532-3546.
- [2] TOUSSAINT G T. Solving geometric problems with the rotating calipers[C]//Proceedings of IEEE MELECON '83. Athens: IEEE, 1983: 1-8.
- [3] WELZL E. Smallest enclosing disks (balls and ellipsoids)[M]//New Results and New Trends in Computer Science. Berlin: Springer, 1991: 359-370.
- [4] TOKEKAR P, ISLER V. Sensor placement and selection for bearing sensors with bounded uncertainty[C]//IEEE International Conference on Robotics and Automation (ICRA). Karlsruhe: IEEE, 2013: 2515-2520.
- [5] DOĞANÇAY K, HMAM H. Optimal angular sensor separation for AOA localization[J]. Signal Processing, 2008, 88(3): 648-660.
- [6] VANDER HOOK P, TOKEKAR P, ISLER V. Cautious greedy strategy for active localization with bounded-error sensors[C]//IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). Chicago: IEEE, 2014: 4320-4325.
- [7] CHOSET H. Coverage for robotics: A survey of recent results[J]. Annals of Mathematics and Artificial Intelligence, 2001, 31(1): 113-126.
- [8] SONG D, KIM C Y, YI J. Simultaneous localization of multiple unknown and transient radio sources using a mobile robot[J]. IEEE Transactions on Robotics, 2012, 28(3): 668-680.
- [9] GUVENSAN M A, YAVUZ A G. On coverage issues in directional sensor networks: A survey[J]. Computer Networks, 2011, 55(15): 3358-3378.
- [10] BERCEA I O, ISLER V, KHULLER S. Minimizing uncertainty through sensor placement with angle constraints[C]//Proceedings of the 28th Canadian Conference on Computational Geometry (CCCG). Vancouver: Simon Fraser University, 2016: 28-34.

程序运行环境：Python

A 问题一程序代码

Listing 1 q1_geometry.py

```
1 from __future__ import annotations
2
3 from collections import deque
4 from dataclasses import dataclass, field
5 from enum import Enum
6 import math
7 from typing import Any, Iterable, Sequence
8
9
10 Point = tuple[float, float]
11
12
13 class RegionStatus(str, Enum):
14     VALID = "VALID"
15     EMPTY = "EMPTY"
16     UNBOUNDED = "UNBOUNDED"
17     DEGENERATE = "DEGENERATE"
18     NUMERICALLY_UNCERTAIN = "NUMERICALLY_UNCERTAIN"
19
20
21 @dataclass(frozen=True)
22 class HalfPlane:
23
24     q: Point
25     d: Point
26     source: str = ""
27
28
29 @dataclass
30 class Q1Result:
31     status: RegionStatus
32     vertices: tuple[Point, ...] = ()
33     diameter: float | None = None
34     diameter_pair: tuple[Point, Point] | None = None
35     circle_center: Point | None = None
36     covered: bool | None = None
37     max_violation: float | None = None
38     violating_vertex: Point | None = None
39     diagnostics: dict[str, Any] = field(default_factory=dict)
40
41     def to_dict(self) -> dict[str, Any]:
42         return {
43             "status": self.status.value,
44             "vertices": [list(point) for point in self.vertices],
45             "diameter": self.diameter,
46             "diameter_pair": (
```

```

47         [list(self.diameter_pair[0]), list(self.diameter_pair[1])]
48         if self.diameter_pair is not None
49         else None
50     ),
51     "circle_center": list(self.circle_center) if self.circle_center else None,
52     "covered": self.covered,
53     "max_violation": self.max_violation,
54     "violating_vertex": (
55         list(self.violating_vertex) if self.violating_vertex else None
56     ),
57     "diagnostics": self.diagnostics,
58 }
59
60
61 def _add(a: Point, b: Point) -> Point:
62     return (a[0] + b[0], a[1] + b[1])
63
64
65 def _sub(a: Point, b: Point) -> Point:
66     return (a[0] - b[0], a[1] - b[1])
67
68
69 def _mul(a: Point, value: float) -> Point:
70     return (a[0] * value, a[1] * value)
71
72
73 def _dot(a: Point, b: Point) -> float:
74     return a[0] * b[0] + a[1] * b[1]
75
76
77 def _cross(a: Point, b: Point) -> float:
78     return a[0] * b[1] - a[1] * b[0]
79
80
81 def _norm_sq(a: Point) -> float:
82     return _dot(a, a)
83
84
85 def _distance_sq(a: Point, b: Point) -> float:
86     return _norm_sq(_sub(a, b))
87
88
89 def make_halfplane(q: Point, d: Point, source: str = "") -> HalfPlane:
90     values = (*q, *d)
91     if not all(math.isfinite(value) for value in values):
92         raise ValueError("半平面的坐标与方向必须是有限数")
93     length = math.hypot(d[0], d[1])
94     if length == 0.0:
95         raise ValueError("半平面方向不能为零向量")
96     return HalfPlane((float(q[0]), float(q[1])), (d[0] / length, d[1] / length), source)
97
98
99 def bearing_halfplanes(

```

```

100     observations: Sequence[tuple[float, float, float]],
101     error_deg: float = 1.0,
102 ) -> list[HalfPlane]:
103     """将 `(x, y, bearing_degrees)` 观测转换为 2n 个约束。"""
104
105     if not observations:
106         raise ValueError("至少需要一条示向观测")
107     if not math.isfinite(error_deg) or not 0.0 < error_deg < 90.0:
108         raise ValueError("error_deg 必须是位于 (0, 90) 内的有限数")
109
110     epsilon = math.radians(error_deg)
111     result: list[HalfPlane] = []
112     for index, (x, y, bearing_deg) in enumerate(observations):
113         if not all(math.isfinite(value) for value in (x, y, bearing_deg)):
114             raise ValueError("观测值必须是有限数")
115         theta = math.radians(bearing_deg % 360.0)
116         lower = (math.cos(theta - epsilon), math.sin(theta - epsilon))
117         upper = (math.cos(theta + epsilon), math.sin(theta + epsilon))
118         q = (float(x), float(y))
119         result.append(make_halfplane(q, lower, f"observation_{index}:lower"))
120         result.append(make_halfplane(q, (-upper[0], -upper[1]), f"observation_{index}:upper"))
121     return result
122
123
124 def side(line: HalfPlane, point: Point) -> float:
125     return _cross(line.d, _sub(point, line.q))
126
127
128 def contains(line: HalfPlane, point: Point, tolerance: float = 1e-10) -> bool:
129     return side(line, point) >= -tolerance
130
131
132 def _line_intersection(
133     first: HalfPlane,
134     second: HalfPlane,
135     parallel_tolerance: float,
136 ) -> Point | None:
137     denominator = _cross(first.d, second.d)
138     if abs(denominator) <= parallel_tolerance:
139         return None
140     parameter = _cross(_sub(second.q, first.q), second.d) / denominator
141     return _add(first.q, _mul(first.d, parameter))
142
143
144 def _constraint(line: HalfPlane) -> tuple[float, float, float]:
145     a = line.d[1]
146     b = -line.d[0]
147     c = a * line.q[0] + b * line.q[1]
148     return (a, b, c)
149
150
151 def _find_feasible_point(

```

```

152     lines: Sequence[HalfPlane], tolerance: float
153 ) -> Point | None:
154     """采用精确结构的增量二维线性可行检查。"""
155
156     constraints = [_constraint(line) for line in lines]
157     point = (0.0, 0.0)
158     for index, (a, b, c) in enumerate(constraints):
159         if a * point[0] + b * point[1] <= c + tolerance:
160             continue
161
162         normal_sq = a * a + b * b
163         base = (a * c / normal_sq, b * c / normal_sq)
164         direction = (-b / math.sqrt(normal_sq), a / math.sqrt(normal_sq))
165         lower = -math.inf
166         upper = math.inf
167         for previous_a, previous_b, previous_c in constraints[:index]:
168             coefficient = previous_a * direction[0] + previous_b * direction[1]
169             allowance = previous_c - previous_a * base[0] - previous_b * base[1]
170             if abs(coefficient) <= tolerance:
171                 if allowance < -tolerance:
172                     return None
173                 continue
174             bound = allowance / coefficient
175             if coefficient > 0.0:
176                 upper = min(upper, bound)
177             else:
178                 lower = max(lower, bound)
179             if lower > upper + tolerance:
180                 return None
181
182         if lower <= 0.0 <= upper:
183             parameter = 0.0
184         elif upper < 0.0:
185             parameter = upper
186         else:
187             parameter = lower
188         point = _add(base, _mul(direction, parameter))
189
190     if all(contains(line, point, 10.0 * tolerance) for line in lines):
191         return point
192     return None
193
194
195 def _has_recession_direction(lines: Sequence[HalfPlane], tolerance: float) -> bool:
196
197     if not lines:
198         return True
199     angles = sorted(
200         math.atan2(-line.d[0], line.d[1]) % (2.0 * math.pi) for line in lines
201     )
202     gaps = [angles[index + 1] - angles[index] for index in range(len(angles) - 1)]
203     gaps.append(angles[0] + 2.0 * math.pi - angles[-1])
204     return max(gaps) >= math.pi - 10.0 * tolerance

```

```

205
206
207 def _same_direction(first: HalfPlane, second: HalfPlane, tolerance: float) -> bool:
208     return abs(_cross(first.d, second.d)) <= tolerance and _dot(first.d, second.d) > 0.0
209
210
211 def _more_restrictive(first: HalfPlane, second: HalfPlane) -> HalfPlane:
212     return second if side(first, second.q) > 0.0 else first
213
214
215 def _sort_and_merge_same_direction(
216     lines: Sequence[HalfPlane], tolerance: float
217 ) -> list[HalfPlane]:
218     ordered = sorted(lines, key=lambda line: math.atan2(line.d[1], line.d[0]) % (2.0 * math
219         .pi))
220     merged: list[HalfPlane] = []
221     for line in ordered:
222         if merged and _same_direction(merged[-1], line, tolerance):
223             merged[-1] = _more_restrictive(merged[-1], line)
224         else:
225             merged.append(line)
226
227     if len(merged) > 1 and _same_direction(merged[0], merged[-1], tolerance):
228         merged[0] = _more_restrictive(merged[0], merged[-1])
229         merged.pop()
230     return merged
231
232 def _convex_hull(points: Iterable[Point], tolerance: float) -> list[Point]:
233     ordered = sorted(points)
234     unique: list[Point] = []
235     for point in ordered:
236         if not unique or _distance_sq(point, unique[-1]) > tolerance * tolerance:
237             unique.append(point)
238     if len(unique) <= 1:
239         return unique
240
241     def turn(o: Point, a: Point, b: Point) -> float:
242         return _cross(_sub(a, o), _sub(b, o))
243
244     lower: list[Point] = []
245     for point in unique:
246         while len(lower) >= 2 and turn(lower[-2], lower[-1], point) <= tolerance:
247             lower.pop()
248         lower.append(point)
249     upper: list[Point] = []
250     for point in reversed(unique):
251         while len(upper) >= 2 and turn(upper[-2], upper[-1], point) <= tolerance:
252             upper.pop()
253         upper.append(point)
254     return lower[:-1] + upper[::-1]
255
256

```

```

257 def polygon_area(vertices: Sequence[Point]) -> float:
258     if len(vertices) < 3:
259         return 0.0
260     doubled = sum(
261         _cross(vertices[index], vertices[(index + 1) % len(vertices)])
262         for index in range(len(vertices))
263     )
264     return 0.5 * abs(doubled)
265
266
267 def halfplane_intersection_main(
268     lines: Sequence[HalfPlane], tolerance: float = 1e-10
269 ) -> list[Point]:
270     """半平面交"""
271
272     parallel_tolerance = max(tolerance * 0.1, 1e-14)
273     ordered = _sort_and_merge_same_direction(lines, parallel_tolerance)
274     active: deque[HalfPlane] = deque()
275
276     for line in ordered:
277         while len(active) >= 2:
278             point = _line_intersection(active[-2], active[-1], parallel_tolerance)
279             if point is None:
280                 break
281             if contains(line, point, tolerance):
282                 break
283             active.pop()
284         while len(active) >= 2:
285             point = _line_intersection(active[0], active[1], parallel_tolerance)
286             if point is None:
287                 break
288             if contains(line, point, tolerance):
289                 break
290             active.popleft()
291         active.append(line)
292
293     while len(active) >= 3:
294         point = _line_intersection(active[-2], active[-1], parallel_tolerance)
295         if point is not None and contains(active[0], point, tolerance):
296             break
297         active.pop()
298     while len(active) >= 3:
299         point = _line_intersection(active[0], active[1], parallel_tolerance)
300         if point is not None and contains(active[-1], point, tolerance):
301             break
302         active.popleft()
303
304     if len(active) < 3:
305         return []
306     active_list = list(active)
307     vertices: list[Point] = []
308     for index, line in enumerate(active_list):
309         next_line = active_list[(index + 1) % len(active_list)]

```

```

310     point = _line_intersection(line, next_line, parallel_tolerance)
311     if point is None or not all(
312         contains(constraint, point, 20.0 * tolerance) for constraint in lines
313     ):
314         return []
315     vertices.append(point)
316     return _convex_hull(vertices, 20.0 * tolerance)
317
318
319 def halfplane_intersection_reference(
320     lines: Sequence[HalfPlane], tolerance: float = 1e-10
321 ) -> list[Point]:
322     """枚举两两边界交点。"""
323
324     parallel_tolerance = max(tolerance * 0.1, 1e-14)
325     candidates: list[Point] = []
326     for first_index, first in enumerate(lines):
327         for second in lines[first_index + 1 :]:
328             point = _line_intersection(first, second, parallel_tolerance)
329             if point is None:
330                 continue
331             if all(contains(line, point, 20.0 * tolerance) for line in lines):
332                 candidates.append(point)
333     return _convex_hull(candidates, 20.0 * tolerance)
334
335
336 def diameter_bruteforce(vertices: Sequence[Point]) -> tuple[float, tuple[Point, Point]]:
337     if not vertices:
338         raise ValueError("计算直径至少需要一个顶点")
339     best_pair = (vertices[0], vertices[0])
340     best_sq = 0.0
341     for index, first in enumerate(vertices):
342         for second in vertices[index + 1 :]:
343             distance_sq = _distance_sq(first, second)
344             if distance_sq > best_sq:
345                 best_sq = distance_sq
346                 best_pair = (first, second)
347     return math.sqrt(best_sq), best_pair
348
349
350 def diameter_rotating_calipers(
351     vertices: Sequence[Point], tolerance: float = 1e-10
352 ) -> tuple[float, tuple[Point, Point]]:
353     """旋转卡壳"""
354
355     count = len(vertices)
356     if count == 0:
357         raise ValueError("计算直径至少需要一个顶点")
358     if count <= 2:
359         return diameter_bruteforce(vertices)
360
361     best_sq = 0.0
362     best_pair = (vertices[0], vertices[0])

```

```

363
364 def update(first_index: int, second_index: int) -> None:
365     nonlocal best_sq, best_pair
366     distance_sq = _distance_sq(vertices[first_index], vertices[second_index])
367     if distance_sq > best_sq:
368         best_sq = distance_sq
369         best_pair = (vertices[first_index], vertices[second_index])
370
371 opposite = 1
372 for index in range(count):
373     next_index = (index + 1) % count
374     edge = _sub(vertices[next_index], vertices[index])
375
376     def support(candidate: int) -> float:
377         return abs(_cross(edge, _sub(vertices[candidate], vertices[index])))
378
379     while support((opposite + 1) % count) > support(opposite) + tolerance:
380         opposite = (opposite + 1) % count
381         update(index, opposite)
382         update(next_index, opposite)
383         next_opposite = (opposite + 1) % count
384         if abs(support(next_opposite) - support(opposite)) <= tolerance:
385             update(index, next_opposite)
386             update(next_index, next_opposite)
387
388     return math.sqrt(best_sq), best_pair
389
390
391 def coverage_by_dot_product(
392     vertices: Sequence[Point],
393     diameter_pair: tuple[Point, Point],
394     tolerance: float = 1e-10,
395 ) -> tuple[bool, float, Point | None]:
396     first, second = diameter_pair
397     diameter_sq = _distance_sq(first, second)
398     threshold = tolerance * max(1.0, diameter_sq)
399     values = [(_dot(_sub(vertex, first), _sub(vertex, second)), vertex) for vertex in
400               vertices]
401     maximum, witness = max(values, default=(0.0, None), key=lambda item: item[0])
402     return maximum <= threshold, max(0.0, maximum), witness if maximum > threshold else
403     None
404
405 def coverage_by_center_distance(
406     vertices: Sequence[Point],
407     diameter_pair: tuple[Point, Point],
408     tolerance: float = 1e-10,
409 ) -> bool:
410     first, second = diameter_pair
411     center = _mul(_add(first, second), 0.5)
412     radius_sq = 0.25 * _distance_sq(first, second)
413     threshold = tolerance * max(1.0, radius_sq)
414     return all(_distance_sq(vertex, center) <= radius_sq + threshold for vertex in vertices

```

```

    )
414
415
416 def _vertex_sets_agree(first: Sequence[Point], second: Sequence[Point], tolerance: float)
    -> bool:
417     if len(first) != len(second):
418         return False
419     limit_sq = (100.0 * tolerance) ** 2
420     return all(min(_distance_sq(point, other) for other in second) <= limit_sq for point in
        first)
421
422
423 def _normalization(lines: Sequence[HalfPlane]) -> tuple[Point, float]:
424     if not lines:
425         return (0.0, 0.0), 1.0
426     center = (
427         sum(line.q[0] for line in lines) / len(lines),
428         sum(line.q[1] for line in lines) / len(lines),
429     )
430     scale = max((math.hypot(*_sub(line.q, center)) for line in lines), default=1.0)
431     return center, max(scale, 1.0)
432
433
434 def solve_halfplanes(
435     halfplanes: Sequence[HalfPlane],
436     *,
437     tolerance: float = 1e-10,
438     verify: bool = True,
439 ) -> Q1Result:
440     if not math.isfinite(tolerance) or tolerance <= 0.0:
441         raise ValueError("tolerance 必须是正有限数")
442     lines = [make_halfplane(line.q, line.d, line.source) for line in halfplanes]
443     if not lines:
444         return Q1Result(RegionStatus.UNBOUNDED, diagnostics={"constraint_count": 0})
445
446     center, scale = _normalization(lines)
447     normalized = [
448         make_halfplane(_mul(_sub(line.q, center), 1.0 / scale), line.d, line.source)
449         for line in lines
450     ]
451     diagnostics: dict[str, Any] = {
452         "constraint_count": len(lines),
453         "normalization_center": list(center),
454         "normalization_scale": scale,
455         "tolerance": tolerance,
456         "formal_vertex_algorithm": "angle-sorted deque half-plane intersection",
457         "reference_vertex_algorithm": "pairwise boundary intersections with full screening"
458     }
459     main_vertices = halfplane_intersection_main(normalized, tolerance)
460     diagnostics["main_vertex_count"] = len(main_vertices)
461     diagnostics["normalized_area"] = polygon_area(main_vertices)
462     main_is_2d = len(main_vertices) >= 3 and polygon_area(main_vertices) > 100.0 *

```

```

tolerance
463 has_recession = _has_recession_direction(normalized, tolerance)
464
465 if not main_is_2d or has_recession:
466     feasible_point = _find_feasible_point(normalized, tolerance)
467     if feasible_point is None:
468         return Q1Result(RegionStatus.EMPTY, diagnostics=diagnostics)
469     if has_recession:
470         return Q1Result(RegionStatus.UNBOUNDED, diagnostics=diagnostics)
471     reference_vertices = halfplane_intersection_reference(normalized, tolerance)
472     diagnostics["reference_vertex_count"] = len(reference_vertices)
473     diagnostics["reference_normalized_area"] = polygon_area(reference_vertices)
474     if len(reference_vertices) < 3 or polygon_area(reference_vertices) <= 100.0 *
tolerance:
475         return Q1Result(RegionStatus.DEGENERATE, diagnostics=diagnostics)
476     return Q1Result(RegionStatus.NUMERICALLY_UNCERTAIN, diagnostics=diagnostics)
477
478 reference_vertices: list[Point] | None = None
479 if verify:
480     reference_vertices = halfplane_intersection_reference(normalized, tolerance)
481     vertex_agreement = _vertex_sets_agree(main_vertices, reference_vertices, tolerance)
482     diagnostics["main_reference_vertex_agreement"] = vertex_agreement
483     diagnostics["reference_vertex_count"] = len(reference_vertices)
484     diagnostics["reference_normalized_area"] = polygon_area(reference_vertices)
485     if not vertex_agreement:
486         return Q1Result(RegionStatus.NUMERICALLY_UNCERTAIN, diagnostics=diagnostics)
487 else:
488     diagnostics["reference_checks_run"] = False
489
490 diameter, pair = diameter_rotating_calipers(main_vertices, tolerance)
491 if verify:
492     assert reference_vertices is not None
493     reference_diameter, _ = diameter_bruteforce(reference_vertices)
494     diameter_agreement = math.isclose(
495         diameter,
496         reference_diameter,
497         rel_tol=100.0 * tolerance,
498         abs_tol=100.0 * tolerance,
499     )
500     diagnostics["calipers_bruteforce_diameter_agreement"] = diameter_agreement
501     diagnostics["normalized_reference_diameter"] = reference_diameter
502     if not diameter_agreement:
503         return Q1Result(RegionStatus.NUMERICALLY_UNCERTAIN, diagnostics=diagnostics)
504
505 covered, violation, violating_vertex = coverage_by_dot_product(
506     main_vertices, pair, tolerance
507 )
508 if verify:
509     reference_covered = coverage_by_center_distance(main_vertices, pair, tolerance)
510     diagnostics["coverage_formula_agreement"] = covered == reference_covered
511     if covered != reference_covered:
512         return Q1Result(RegionStatus.NUMERICALLY_UNCERTAIN, diagnostics=diagnostics)
513

```

```

514 def restore(point: Point) -> Point:
515     return _add(center, _mul(point, scale))
516
517 restored_vertices = tuple(restore(point) for point in main_vertices)
518 restored_pair = (restore(pair[0]), restore(pair[1]))
519 restored_center = _mul(_add(restored_pair[0], restored_pair[1]), 0.5)
520 restored_witness = restore(violating_vertex) if violating_vertex is not None else None
521 return Q1Result(
522     status=RegionStatus.VALID,
523     vertices=restored_vertices,
524     diameter=diameter * scale,
525     diameter_pair=restored_pair,
526     circle_center=restored_center,
527     covered=covered,
528     max_violation=violation * scale * scale,
529     violating_vertex=restored_witness,
530     diagnostics=diagnostics,
531 )
532
533
534 def solve_observations(
535     observations: Sequence[tuple[float, float, float]],
536     *,
537     error_deg: float = 1.0,
538     tolerance: float = 1e-10,
539     verify: bool = True,
540 ) -> Q1Result:
541     return solve_halfplanes(
542         bearing_halfplanes(observations, error_deg),
543         tolerance=tolerance,
544         verify=verify,
545     )
546
547
548 __all__ = [
549     "HalfPlane",
550     "Point",
551     "Q1Result",
552     "RegionStatus",
553     "bearing_halfplanes",
554     "contains",
555     "coverage_by_center_distance",
556     "coverage_by_dot_product",
557     "diameter_bruteforce",
558     "diameter_rotating_calipers",
559     "halfplane_intersection_main",
560     "halfplane_intersection_reference",
561     "make_halfplane",
562     "polygon_area",
563     "side",
564     "solve_halfplanes",
565     "solve_observations",
566 ]

```

Listing 2 run_q1.py

```
1 from __future__ import annotations
2
3 import argparse
4 import json
5 from pathlib import Path
6 from typing import Any
7
8 from q1_geometry import solve_observations
9
10
11 EXAMPLE_OBSERVATIONS = [
12     {"x": 1000.0, "y": 0.0, "bearing_deg": 180.0},
13     {"x": 0.0, "y": 1000.0, "bearing_deg": 270.0},
14     {"x": -1000.0, "y": 0.0, "bearing_deg": 0.0},
15     {"x": 0.0, "y": -1000.0, "bearing_deg": 90.0},
16 ]
17
18
19 def _parse_observations(payload: Any) -> list[tuple[float, float, float]]:
20     rows = payload.get("observations") if isinstance(payload, dict) else payload
21     if not isinstance(rows, list):
22         raise ValueError("输入必须是列表, 或是包含 'observations' 字段的对象")
23     observations = []
24     for row in rows:
25         if isinstance(row, dict):
26             observations.append((float(row["x"]), float(row["y"]), float(row["bearing_deg"])))
27         elif isinstance(row, list) and len(row) == 3:
28             observations.append((float(row[0]), float(row[1]), float(row[2])))
29         else:
30             raise ValueError("每条观测必须采用 {x,y,bearing_deg} 或 [x,y,bearing_deg] 格式")
31     return observations
32
33
34 def main() -> int:
35     parser = argparse.ArgumentParser(description="求解 MCM 2026 B 题 Q1 示向几何问题")
36     parser.add_argument("--input", type=Path, help="UTF-8 编码的 JSON 观测文件")
37     parser.add_argument("--output", type=Path, help="可选的 UTF-8 编码 JSON 结果文件")
38     parser.add_argument("--error-deg", type=float, default=1.0)
39     parser.add_argument("--tolerance", type=float, default=1e-10)
40     args = parser.parse_args()
41
42     if args.input:
43         payload = json.loads(args.input.read_text(encoding="utf-8"))
44     else:
45         payload = {"observations": EXAMPLE_OBSERVATIONS}
46     observations = _parse_observations(payload)
47     result = solve_observations(
48         observations,
```

```

49     error_deg=args.error_deg,
50     tolerance=args.tolerance,
51     verify=True,
52 )
53 text = json.dumps(result.to_dict(), ensure_ascii=False, indent=2)
54 if args.output:
55     args.output.parent.mkdir(parents=True, exist_ok=True)
56     args.output.write_text(text + "\n", encoding="utf-8")
57 print(text)
58 return 0 if result.status.value == "VALID" else 2
59
60
61 if __name__ == "__main__":
62     raise SystemExit(main())

```

Listing 3 verify_q1.py

```

1  from __future__ import annotations
2
3  import argparse
4  import json
5  import math
6  from pathlib import Path
7  import platform
8  import random
9  import sys
10 import time
11
12 from q1_geometry import RegionStatus, bearing_halfplanes, contains, solve_observations
13
14
15 def random_ring_case(seed: int) -> tuple[list[tuple[float, float, float]], tuple[float,
16     float]]:
17     rng = random.Random(seed)
18     source = (rng.uniform(-500.0, 500.0), rng.uniform(-500.0, 500.0))
19     count = rng.randint(5, 14)
20     phase = rng.uniform(0.0, 2.0 * math.pi)
21     observations = []
22     for index in range(count):
23         angle = phase + 2.0 * math.pi * index / count + rng.uniform(-0.12, 0.12)
24         radius = rng.uniform(500.0, 1500.0)
25         detector = (
26             source[0] + radius * math.cos(angle),
27             source[1] + radius * math.sin(angle),
28         )
29         true_bearing = math.degrees(
30             math.atan2(source[1] - detector[1], source[0] - detector[0])
31         )
32         observed_bearing = true_bearing + rng.uniform(-0.95, 0.95)
33         observations.append((detector[0], detector[1], observed_bearing))
34     return observations, source
35

```

```

36 def _assert_close(first: float, second: float, label: str) -> None:
37     if not math.isclose(first, second, rel_tol=1e-8, abs_tol=1e-7):
38         raise AssertionError(f"{label}: {first!r} != {second!r}")
39
40
41 def run_verification(random_cases: int, base_seed: int) -> dict:
42     started = time.perf_counter()
43     counts = {
44         "valid_cases": 0,
45         "covered_cases": 0,
46         "not_covered_cases": 0,
47         "order_checks": 0,
48         "duplicate_checks": 0,
49         "rigid_transform_checks": 0,
50         "monotonicity_checks": 0,
51         "tolerance_sensitivity_checks": 0,
52     }
53     maximum_diameter_difference = 0.0
54
55     for case_index in range(random_cases):
56         case_seed = base_seed + case_index
57         observations, source = random_ring_case(case_seed)
58         result = solve_observations(observations)
59         if result.status is not RegionStatus.VALID:
60             raise AssertionError(f"种子 {case_seed}: 预期状态为 VALID, 实际为 {result.status}
61             .value}")
62         counts["valid_cases"] += 1
63         counts["covered_cases" if result.covered else "not_covered_cases"] += 1
64
65         if not all(contains(line, source, 1e-8) for line in bearing_halfplanes(observations
66         )):
67             raise AssertionError(f"种子 {case_seed}: 生成的源点落在示向区域之外")
68         required_diagnostics = (
69             "main_reference_vertex_agreement",
70             "calipers_bruteforce_diameter_agreement",
71             "coverage_formula_agreement",
72         )
73         if not all(result.diagnostics.get(name) for name in required_diagnostics):
74             raise AssertionError(f"种子 {case_seed}: 内部差分检查失败")
75
76         shuffled = observations[:]
77         random.Random(case_seed ^ 0x5A5A5A5A).shuffle(shuffled)
78         reordered = solve_observations(shuffled)
79         if reordered.status is not RegionStatus.VALID or reordered.covered != result.
80         covered:
81             raise AssertionError(f"种子 {case_seed}: 观测顺序不变性检查失败")
82             difference = abs((reordered.diameter or 0.0) - (result.diameter or 0.0))
83             maximum_diameter_difference = max(maximum_diameter_difference, difference)
84             _assert_close(reordered.diameter or 0.0, result.diameter or 0.0, "顺序变换后的直径")
85             counts["order_checks"] += 1
86
87         duplicated = solve_observations(observations + [observations[0]])
88         if duplicated.status is not RegionStatus.VALID or duplicated.covered != result.

```

```

covered:
86         raise AssertionError(f"种子 {case_seed}: 重复约束不变性检查失败")
87     _assert_close(duplicated.diameter or 0.0, result.diameter or 0.0, "重复约束后的直径"
88 )
89
90     counts["duplicate_checks"] += 1
91
92     previous_diameter = None
93     for prefix_size in range(2, len(observations) + 1):
94         prefix = solve_observations(observations[:prefix_size])
95         if prefix.status is RegionStatus.VALID:
96             if previous_diameter is not None and (prefix.diameter or 0.0) >
97                 previous_diameter + 1e-7:
98                 raise AssertionError(f"种子 {case_seed}: 增加约束后直径反而增大")
99                 previous_diameter = prefix.diameter
100                 counts["monotonicity_checks"] += 1
101
102     if case_index % 10 == 0:
103         angle_deg = 37.0
104         angle = math.radians(angle_deg)
105         cosine, sine = math.cos(angle), math.sin(angle)
106         transformed = [
107             (
108                 cosine * x - sine * y + 1234.5,
109                 sine * x + cosine * y - 987.25,
110                 bearing + angle_deg,
111             )
112             for x, y, bearing in observations
113         ]
114         moved = solve_observations(transformed)
115         if moved.status is not RegionStatus.VALID or moved.covered != result.covered:
116             raise AssertionError(f"种子 {case_seed}: 刚体变换不变性检查失败")
117             _assert_close(moved.diameter or 0.0, result.diameter or 0.0, "刚体变换后的直径")
118             counts["rigid_transform_checks"] += 1
119
120     if case_index % 20 == 0:
121         loose = solve_observations(observations, tolerance=1e-9)
122         tight = solve_observations(observations, tolerance=1e-11)
123         if loose.status is not RegionStatus.VALID or tight.status is not RegionStatus.
124             VALID:
125             raise AssertionError(f"种子 {case_seed}: 容差改变了正常案例的状态")
126             if loose.covered != tight.covered:
127                 raise AssertionError(f"种子 {case_seed}: 容差改变了覆盖判定")
128                 _assert_close(loose.diameter or 0.0, tight.diameter or 0.0, "不同容差下的直径")
129                 counts["tolerance_sensitivity_checks"] += 1
130
131     elapsed = time.perf_counter() - started
132     return {
133         "verification_status": "PASS",
134         "model_scope": "angle-only bearing intersection; no distance or artificial box
135         clipping",
136         "base_seed": base_seed,
137         "random_case_count": random_cases,
138         "counts": counts,

```

```

134     "maximum_order_diameter_difference_m": maximum_diameter_difference,
135     "elapsed_seconds": elapsed,
136     "mean_seconds_per_random_case": elapsed / random_cases if random_cases else 0.0,
137     "python_version": sys.version.split()[0],
138     "platform": platform.platform(),
139 }
140
141
142 def main() -> int:
143     parser = argparse.ArgumentParser(description="验证 Q1 几何算法实现")
144     parser.add_argument("--random-cases", type=int, default=1000)
145     parser.add_argument("--seed", type=int, default=20260911)
146     parser.add_argument(
147         "--output",
148         type=Path,
149         default=Path(__file__).resolve().parents[1] / "results" / "q1_verification.json",
150     )
151     args = parser.parse_args()
152     if args.random_cases <= 0:
153         parser.error("--random-cases 必须为正数")
154
155     summary = run_verification(args.random_cases, args.seed)
156     args.output.parent.mkdir(parents=True, exist_ok=True)
157     args.output.write_text(json.dumps(summary, ensure_ascii=False, indent=2) + "\n",
158                             encoding="utf-8")
159     print(json.dumps(summary, ensure_ascii=False, indent=2))
160     return 0
161
162 if __name__ == "__main__":
163     raise SystemExit(main())

```

Listing 4 plot_q1_validation.py

```

1 from __future__ import annotations
2 import argparse
3 import csv
4 import json
5 import math
6 from pathlib import Path
7 import matplotlib as mpl
8 from matplotlib import font_manager
9 import numpy as np
10 import ultraplot as uplt
11 EXPORT_DPI = 1000
12
13 def _resolve_chinese_font() -> str:
14     system_font = Path(r"C:\Windows\Fonts\msyh.ttc")
15     if system_font.exists():
16         font_manager.fontManager.addfont(system_font)
17         return font_manager.FontProperties(fname=system_font).get_name()
18     for family in ("Microsoft YaHei", "Microsoft YaHei UI"):
19         try:

```

```

20         path = font_manager.findfont(family, fallback_to_default=False)
21     except ValueError:
22         continue
23     font_manager.fontManager.addfont(path)
24     return family
25     raise RuntimeError("中文图注需要安装 Microsoft YaHei 字体")
26
27 def main() -> int:
28     q1_dir = Path(__file__).resolve().parents[1]
29     parser = argparse.ArgumentParser(description="绘制 Q1 验证图")
30     parser.add_argument(
31         "--example",
32         type=Path,
33         default=q1_dir / "results" / "q1_example_result.json",
34     )
35     parser.add_argument(
36         "--cases",
37         type=Path,
38         default=q1_dir / "results" / "q1_validation_cases.csv",
39     )
40     parser.add_argument(
41         "--summary",
42         type=Path,
43         default=q1_dir / "results" / "q1_verification.json",
44     )
45     parser.add_argument(
46         "--output",
47         type=Path,
48         default=q1_dir / "output" / "figures" / "q1_validation.png",
49     )
50     args = parser.parse_args()
51
52     example = json.loads(args.example.read_text(encoding="utf-8"))
53     summary = json.loads(args.summary.read_text(encoding="utf-8"))
54     if example["status"] != "VALID":
55         raise ValueError("示例结果的状态必须为 VALID")
56     vertices = np.asarray(example["vertices"], dtype=float)
57     diameter_pair = np.asarray(example["diameter_pair"], dtype=float)
58     center = np.asarray(example["circle_center"], dtype=float)
59     diameter = float(example["diameter"])
60     if vertices.ndim != 2 or vertices.shape[1] != 2 or not np.isfinite(vertices).all():
61         raise ValueError("示例顶点必须是元素有限的 n×2 数组")
62
63     with args.cases.open("r", encoding="utf-8", newline="") as stream:
64         rows = list(csv.DictReader(stream))
65         if len(rows) != int(summary["random_case_count"]):
66             raise ValueError("案例数据数量与验证摘要不一致")
67         ranks = np.asarray([int(row["rank"]) for row in rows])
68         covered = np.asarray([bool(int(row["covered"])) for row in rows])
69         excess_percent = 100.0 * np.asarray(
70             [float(row["relative_radius_excess"]) for row in rows]
71         )
72         if not np.isfinite(excess_percent).all():

```

```

73     raise ValueError("覆盖诊断值必须是有限数")
74
75     with uplt.rc.context({}):
76         family = _resolve_chinese_font()
77         mpl.rcParams["font.family"] = [family, "DejaVu Sans"]
78         fig, ax = uplt.subplots(
79             ncols=1,
80             journal="nat2",
81             share=False,
82             tight=True,
83         )
84
85         covered_count = int(covered.sum())
86         not_covered_count = int((~covered).sum())
87         ax.scatter(
88             ranks[covered],
89             excess_percent[covered],
90             s=8,
91             color="tab:blue",
92             label=f"覆盖 ({covered_count})",
93         )
94         ax.scatter(
95             ranks[~covered],
96             excess_percent[~covered],
97             s=8,
98             color="tab:orange",
99             label=f"不覆盖 ({not_covered_count})",
100        )
101        ax.axhline(0.0, color="black", linewidth=0.9)
102        ax.legend(loc="ul", ncols=1, frame=False)
103        ax.format(
104            xlabel="按覆盖余量排序的验证实例",
105            ylabel="最大相对半径超量 (%)",
106            xlim=(0, len(rows) + 1),
107            grid=True,
108        )
109        args.output.parent.mkdir(parents=True, exist_ok=True)
110        fig.save(args.output, dpi=EXPORT_DPI)
111    return 0
112
113 if __name__ == "__main__":
114     raise SystemExit(main())

```

B 问题二程序代码

Listing 5 q2_fast.py

```

1 from __future__ import annotations
2
3 from dataclasses import dataclass, replace
4 import math
5 from time import perf_counter

```

```

6 from typing import Sequence
7
8 from q2_geometry import Point, bearing_deg, distance, normalize_angle_deg
9 from q2_state import FirstObservation, Q2Config, StateDomain, build_state_domain
10 from q1_geometry import (
11     HalfPlane,
12     RegionStatus,
13     bearing_halfplanes,
14     diameter_rotating_calipers,
15     make_halfplane,
16     solve_halfplanes,
17 )
18
19
20 @dataclass(frozen=True)
21 class FastOptions:
22     circle_sides: int = 48
23     state_angle_cells: int = 33
24     state_radial_cells: int = 24
25     coarse_source_count: int = 24
26     final_source_count: int = 64
27     coarse_error_nodes: int = 3
28     final_error_nodes: int = 7
29     seed_count: int = 3
30     local_rounds: int = 4
31     local_initial_step_m: float | None = None
32     recommendation_count: int = 3
33     tolerance: float = 1e-9
34
35
36 @dataclass(frozen=True)
37 class EnclosingCircle:
38     center: Point
39     radius: float
40
41
42 @dataclass(frozen=True)
43 class CandidateScore:
44     point: Point
45     stage: str
46     movement_m: float
47     conservative_safety_margin_m: float
48     worst_posterior_diameter_m: float
49     worst_source: Point | None
50     worst_error_deg: float | None
51     worst_observed_bearing_deg: float | None
52     worst_posterior_vertex_count: int
53     evaluated_direction_cases: int
54     evaluated_near_cases: int
55
56     def to_dict(self) -> dict[str, object]:
57         return {
58             "point_m": list(self.point),

```

```

59         "stage": self.stage,
60         "movement_m": self.movement_m,
61         "conservative_safety_margin_m": self.conservative_safety_margin_m,
62         "worst_posterior_diameter_m": self.worst_posterior_diameter_m,
63         "worst_source_m": list(self.worst_source) if self.worst_source else None,
64         "worst_error_deg": self.worst_error_deg,
65         "worst_observed_bearing_deg": self.worst_observed_bearing_deg,
66         "worst_posterior_vertex_count": self.worst_posterior_vertex_count,
67         "evaluated_direction_cases": self.evaluated_direction_cases,
68         "evaluated_near_cases": self.evaluated_near_cases,
69     }
70
71
72 def _disk_outer_halfplanes(
73     center: Point, radius: float, sides: int, source: str
74 ) -> list[HalfPlane]:
75     if sides < 8:
76         raise ValueError("circle_sides 必须至少为 8")
77     result: list[HalfPlane] = []
78     for index in range(sides):
79         angle = 2.0 * math.pi * index / sides
80         ux, uy = math.cos(angle), math.sin(angle)
81         q = (center[0] + radius * ux, center[1] + radius * uy)
82         result.append(make_halfplane(q, (-uy, ux), f"{source}_{index}"))
83     return result
84
85
86 def build_position_outer_polygon(
87     first: FirstObservation, config: Q2Config, circle_sides: int = 48
88 ) -> list[Point]:
89     """构造与第一次观测相容的全部源位置的凸外包。"""
90     constraints = bearing_halfplanes(
91         [(first.point[0], first.point[1], first.bearing_deg)],
92         error_deg=config.error_deg,
93     )
94     constraints.extend(
95         _disk_outer_halfplanes(config.center, config.target_radius, circle_sides, "target")
96     )
97     constraints.extend(
98         _disk_outer_halfplanes(first.point, config.receive_max, circle_sides, "range")
99     )
100     answer = solve_halfplanes(constraints, tolerance=1e-10, verify=False)
101     if answer.status is not RegionStatus.VALID or len(answer.vertices) < 3:
102         raise ValueError("第一次观测没有对应的正常可行源区域")
103     return [(float(x), float(y)) for x, y in answer.vertices]
104
105
106 def conservative_safety_margin(
107     point: Point, outer_polygon: Sequence[Point], receive_min: float
108 ) -> float:
109     return receive_min - max(distance(point, vertex) for vertex in outer_polygon)
110
111

```

```

112 def _pair_circle(first: Point, second: Point) -> EnclosingCircle:
113     center = ((first[0] + second[0]) / 2.0, (first[1] + second[1]) / 2.0)
114     return EnclosingCircle(center, distance(first, second) / 2.0)
115
116
117 def _triple_circle(a: Point, b: Point, c: Point) -> EnclosingCircle | None:
118     denominator = 2.0 * (
119         a[0] * (b[1] - c[1])
120         + b[0] * (c[1] - a[1])
121         + c[0] * (a[1] - b[1])
122     )
123     if abs(denominator) <= 1e-12:
124         return None
125     aa = a[0] * a[0] + a[1] * a[1]
126     bb = b[0] * b[0] + b[1] * b[1]
127     cc = c[0] * c[0] + c[1] * c[1]
128     center = (
129         (aa * (b[1] - c[1]) + bb * (c[1] - a[1]) + cc * (a[1] - b[1]))
130         / denominator,
131         (aa * (c[0] - b[0]) + bb * (a[0] - c[0]) + cc * (b[0] - a[0]))
132         / denominator,
133     )
134     return EnclosingCircle(center, distance(center, a))
135
136
137 def minimum_enclosing_circle(points: Sequence[Point]) -> EnclosingCircle:
138     if not points:
139         raise ValueError("计算最小包围圆至少需要一个点")
140     candidates = [EnclosingCircle(point, 0.0) for point in points]
141     for i, first in enumerate(points):
142         for second in points[i + 1 :]:
143             candidates.append(_pair_circle(first, second))
144     for i, first in enumerate(points):
145         for j in range(i + 1, len(points)):
146             for k in range(j + 1, len(points)):
147                 circle = _triple_circle(first, points[j], points[k])
148                 if circle is not None:
149                     candidates.append(circle)
150     feasible = [
151         circle
152         for circle in candidates
153         if all(distance(circle.center, point) <= circle.radius + 1e-7 for point in points)
154     ]
155     if not feasible:
156         raise RuntimeError("最小包围圆构造失败")
157     return min(feasible, key=lambda item: item.radius)
158
159
160 def _ray_safe_extent(
161     origin: Point, direction: Point, polygon: Sequence[Point], radius: float
162 ) -> float:
163     upper = math.inf
164     for vertex in polygon:

```

```

165     wx, wy = origin[0] - vertex[0], origin[1] - vertex[1]
166     projection = wx * direction[0] + wy * direction[1]
167     discriminant = projection * projection - (wx * wx + wy * wy - radius * radius)
168     if discriminant < -1e-7:
169         return 0.0
170     upper = min(upper, -projection + math.sqrt(max(0.0, discriminant)))
171     return max(0.0, upper)
172
173
174 def _deduplicate(points: Sequence[Point], tolerance: float = 1e-6) -> list[Point]:
175     result: list[Point] = []
176     for point in points:
177         if all(distance(point, other) > tolerance for other in result):
178             result.append(point)
179     return result
180
181
182 def generate_sparse_candidates(
183     first: FirstObservation,
184     config: Q2Config,
185     polygon: Sequence[Point],
186     *,
187     tolerance: float = 1e-9,
188 ) -> tuple[list[Point], EnclosingCircle]:
189     circle = minimum_enclosing_circle(polygon)
190     if circle.radius > config.receive_min + tolerance:
191         return [], circle
192     axis = math.radians(normalize_angle_deg(first.bearing_deg))
193     directions = [
194         (math.cos(axis + index * math.pi / 6.0), math.sin(axis + index * math.pi / 6.0))
195         for index in range(12)
196     ]
197     raw = [circle.center]
198     for direction in directions:
199         extent = _ray_safe_extent(circle.center, direction, polygon, config.receive_min)
200         for fraction in (0.55, 0.9):
201             raw.append(
202                 (
203                     circle.center[0] + fraction * extent * direction[0],
204                     circle.center[1] + fraction * extent * direction[1],
205                 )
206             )
207     candidates = [
208         point
209         for point in _deduplicate(raw)
210         if distance(point, first.point) > tolerance
211         and conservative_safety_margin(point, polygon, config.receive_min) >= -1e-7
212     ]
213     return candidates, circle
214
215
216 def _representative_sources(domain: StateDomain, count: int) -> list[Point]:
217     pool = _deduplicate(

```

```

218         list(domain.support_points) + [cell.point for cell in domain.samples], 1e-8
219     )
220     if len(pool) <= count:
221         return pool
222     center = (
223         sum(point[0] for point in pool) / len(pool),
224         sum(point[1] for point in pool) / len(pool),
225     )
226     selected = [max(pool, key=lambda point: distance(point, center))]
227     while len(selected) < count:
228         selected.append(
229             max(pool, key=lambda point: min(distance(point, chosen) for chosen in selected)
230         )
231     )
232     return _deduplicate(selected, 1e-8)[:count]
233
234 def _error_grid(error_deg: float, nodes: int) -> list[float]:
235     if nodes <= 1 or error_deg == 0.0:
236         return [0.0]
237     return [-error_deg + 2.0 * error_deg * index / (nodes - 1) for index in range(nodes)]
238
239
240 def _side(line: HalfPlane, point: Point) -> float:
241     return line.d[0] * (point[1] - line.q[1]) - line.d[1] * (point[0] - line.q[0])
242
243
244 def _clip_halfplane(
245     polygon: Sequence[Point], line: HalfPlane, tolerance: float
246 ) -> list[Point]:
247     if not polygon:
248         return []
249     result: list[Point] = []
250     previous = polygon[-1]
251     previous_side = _side(line, previous)
252     for current in polygon:
253         current_side = _side(line, current)
254         previous_in = previous_side >= -tolerance
255         current_in = current_side >= -tolerance
256         if previous_in != current_in:
257             denominator = previous_side - current_side
258             fraction = 0.5 if abs(denominator) <= tolerance else previous_side /
denominator
259             result.append(
260                 (
261                     previous[0] + fraction * (current[0] - previous[0]),
262                     previous[1] + fraction * (current[1] - previous[1]),
263                 )
264             )
265         if current_in:
266             result.append(current)
267         previous, previous_side = current, current_side
268     return _deduplicate(result, 1e-8)

```

```

269
270
271 def posterior_polygon(
272     prior_polygon: Sequence[Point],
273     candidate: Point,
274     observed_bearing_deg: float,
275     error_deg: float,
276     tolerance: float = 1e-9,
277 ) -> list[Point]:
278     result = list(prior_polygon)
279     lines = bearing_halfplanes(
280         [(candidate[0], candidate[1], observed_bearing_deg)], error_deg=error_deg
281     )
282     for line in lines:
283         result = _clip_halfplane(result, line, tolerance)
284     return result
285
286
287 def evaluate_candidate(
288     candidate: Point,
289     stage: str,
290     first: FirstObservation,
291     config: Q2Config,
292     polygon: Sequence[Point],
293     sources: Sequence[Point],
294     errors: Sequence[float],
295 ) -> CandidateScore:
296     margin = conservative_safety_margin(candidate, polygon, config.receive_min)
297     worst = -1.0
298     witness_source: Point | None = None
299     witness_error: float | None = None
300     witness_bearing: float | None = None
301     witness_vertices = 0
302     direction_cases = 0
303     near_cases = 0
304     for source in sources:
305         if distance(candidate, source) <= config.near_radius:
306             near_cases += 1
307             worst = max(worst, 0.0)
308             continue
309         true_bearing = bearing_deg(candidate, source)
310         for error in errors:
311             direction_cases += 1
312             observed = normalize_angle_deg(true_bearing + error)
313             posterior = posterior_polygon(
314                 polygon, candidate, observed, config.error_deg
315             )
316             if len(posterior) <= 1:
317                 diameter = 0.0
318             else:
319                 diameter = diameter_rotating_calipers(posterior)[0]
320             if diameter > worst:
321                 worst = diameter

```

```

322         witness_source = source
323         witness_error = error
324         witness_bearing = observed
325         witness_vertices = len(posterior)
326     if worst < 0.0:
327         worst = math.inf
328     return CandidateScore(
329         candidate,
330         stage,
331         distance(first.point, candidate),
332         margin,
333         worst,
334         witness_source,
335         witness_error,
336         witness_bearing,
337         witness_vertices,
338         direction_cases,
339         near_cases,
340     )
341
342
343 def _score_key(score: CandidateScore) -> tuple[float, float, float]:
344     return (
345         score.worst_posterior_diameter_m,
346         score.movement_m,
347         -score.conservative_safety_margin_m,
348     )
349
350
351 def _local_refine(
352     seed: CandidateScore,
353     first: FirstObservation,
354     config: Q2Config,
355     polygon: Sequence[Point],
356     sources: Sequence[Point],
357     errors: Sequence[float],
358     step: float,
359     rounds: int,
360 ) -> tuple[CandidateScore, int]:
361     best = seed
362     evaluations = 0
363     directions = [
364         (math.cos(index * math.pi / 4.0), math.sin(index * math.pi / 4.0))
365         for index in range(8)
366     ]
367     for _ in range(rounds):
368         neighbors = [
369             (best.point[0] + step * dx, best.point[1] + step * dy)
370             for dx, dy in directions
371         ]
372         for point in neighbors:
373             if conservative_safety_margin(point, polygon, config.receive_min) < -1e-7:
374                 continue

```

```

375         score = evaluate_candidate(
376             point, "local_refinement", first, config, polygon, sources, errors
377         )
378         evaluations += 1
379         if _score_key(score) < _score_key(best):
380             best = score
381         step *= 0.5
382     return best, evaluations
383
384
385 def solve_fast_q2(
386     first: FirstObservation,
387     config: Q2Config | None = None,
388     options: FastOptions | None = None,
389     safety_buffer_m: float = 0.0,
390 ) -> dict[str, object]:
391     """安全候选和快速极小极大推荐结果。"""
392     started = perf_counter()
393     config = config or Q2Config()
394     options = options or FastOptions()
395     first.validate()
396     config.validate()
397     if not math.isfinite(safety_buffer_m) or safety_buffer_m < 0.0:
398         raise ValueError("safety_buffer_m 必须是非负有限数")
399     effective_receive_radius = config.receive_min - safety_buffer_m
400     if effective_receive_radius <= config.near_radius:
401         raise ValueError("设置安全缓冲后已没有可用的接收半径")
402     safety_config = replace(config, receive_min=effective_receive_radius)
403     safety_config.validate()
404     polygon = build_position_outer_polygon(first, config, options.circle_sides)
405     domain = build_state_domain(
406         first,
407         config,
408         angle_cells=options.state_angle_cells,
409         radial_cells=options.state_radial_cells,
410         tolerance=options.tolerance,
411     )
412     candidates, circle = generate_sparse_candidates(
413         first, safety_config, polygon, tolerance=options.tolerance
414     )
415     base: dict[str, object] = {
416         "algorithm": "CONSERVATIVE_SPARSE_MINIMAX",
417         "evidence_level": {
418             "reception_safety": "conservative outer-polygon certificate",
419             "localization_objective": "representative-state numerical minimax",
420             "global_optimality": "not claimed",
421         },
422         "first_observation": {
423             "point_m": list(first.point),
424             "bearing_deg": normalize_angle_deg(first.bearing_deg),
425         },
426         "safe_region": {
427             "outer_polygon_vertices_m": [list(point) for point in polygon],

```

```

428         "outer_polygon_vertex_count": len(polygon),
429         "minimum_enclosing_circle_center_m": list(circle.center),
430         "minimum_enclosing_circle_radius_m": circle.radius,
431         "minimum_receive_radius_m": config.receive_min,
432         "safety_buffer_m": safety_buffer_m,
433         "effective_safe_radius_m": effective_receive_radius,
434         "nonempty_under_outer_approximation": bool(candidates),
435     },
436     "state_domain": {
437         "status": domain.status.value,
438         "source_sample_count": len(domain.samples),
439         "support_point_count": len(domain.support_points),
440         "approximate_area_m2": domain.approximate_area_m2,
441     },
442 }
443 if not candidates:
444     base.update(
445         {
446             "result_status": "NO_CONSERVATIVELY_SAFE_CANDIDATE",
447             "recommendations": [],
448             "elapsed_seconds": perf_counter() - started,
449         }
450     )
451     return base
452
453 coarse_sources = _representative_sources(domain, options.coarse_source_count)
454 coarse_errors = _error_grid(config.error_deg, options.coarse_error_nodes)
455 coarse_scores = [
456     evaluate_candidate(
457         point, "sparse_search", first, safety_config, polygon, coarse_sources,
458         coarse_errors
459     )
460     for point in candidates
461 ]
462 coarse_scores.sort(key=_score_key)
463 slack = max(0.0, effective_receive_radius - circle.radius)
464 initial_step = options.local_initial_step_m or min(200.0, max(20.0, 0.45 * slack))
465 refined: list[CandidateScore] = []
466 local_evaluations = 0
467 for seed in coarse_scores[: options.seed_count]:
468     result, used = _local_refine(
469         seed,
470         first,
471         safety_config,
472         polygon,
473         coarse_sources,
474         coarse_errors,
475         initial_step,
476         options.local_rounds,
477     )
478     refined.append(result)
479     local_evaluations += used

```

```

480     finalist_points = _deduplicate(
481         [score.point for score in coarse_scores[: max(6, options.seed_count)]]
482         + [score.point for score in refined]
483     )
484     final_sources = _representative_sources(domain, options.final_source_count)
485     final_errors = _error_grid(config.error_deg, options.final_error_nodes)
486     final_scores = [
487         evaluate_candidate(
488             point, "final_recheck", first, safety_config, polygon, final_sources,
489             final_errors
490         )
491         for point in finalist_points
492     ]
493     final_scores.sort(key=_score_key)
494     recommendations = final_scores[: options.recommendation_count]
495     base.update(
496         {
497             "result_status": "FAST_NUMERICAL_SOLUTION",
498             "search": {
499                 "sparse_candidate_count": len(candidates),
500                 "coarse_source_count": len(coarse_sources),
501                 "coarse_error_nodes": len(coarse_errors),
502                 "local_seed_count": min(options.seed_count, len(coarse_scores)),
503                 "local_rounds": options.local_rounds,
504                 "local_initial_step_m": initial_step,
505                 "local_candidate_evaluations": local_evaluations,
506                 "finalist_count": len(finalist_points),
507                 "final_source_count": len(final_sources),
508                 "final_error_nodes": len(final_errors),
509             },
510             "best_sparse_candidate": coarse_scores[0].to_dict(),
511             "recommendations": [score.to_dict() for score in recommendations],
512             "elapsed_seconds": perf_counter() - started,
513         }
514     )
515     return base
516
517 __all__ = [
518     "CandidateScore",
519     "EnclosingCircle",
520     "FastOptions",
521     "build_position_outer_polygon",
522     "conservative_safety_margin",
523     "evaluate_candidate",
524     "generate_sparse_candidates",
525     "minimum_enclosing_circle",
526     "posterior_polygon",
527     "solve_fast_q2",
528 ]

```

Listing 6 q2_geometry.py

```

1  """Q2 的几何基础函数与 Q1 示向交会适配器。
2
3  高频路径以 ``verify=False`` 调用已经验证的 Q1 求解器；
4  部分测试可启用 Q1 的独立复核。
5  """
6
7  from __future__ import annotations
8
9  from collections import Counter
10 from dataclasses import dataclass
11 from enum import Enum
12 import math
13 from pathlib import Path
14 import sys
15 from typing import Sequence
16
17
18 Point = tuple[float, float]
19
20 _Q1_CODE = Path(__file__).resolve().parents[2] / "q1" / "code"
21 if str(_Q1_CODE) not in sys.path:
22     sys.path.insert(0, str(_Q1_CODE))
23
24 from q1_geometry import RegionStatus, solve_observations # noqa: E402
25
26
27 class Branch(str, Enum):
28     NEAR = "near"
29     DIRECTION = "direction"
30     NO_SIGNAL = "no_signal"
31
32
33 def _require_finite(values: Sequence[float], name: str) -> None:
34     if not all(math.isfinite(value) for value in values):
35         raise ValueError(f"{name} 只能包含有限数")
36
37
38 def normalize_angle_deg(angle: float) -> float:
39     if not math.isfinite(angle):
40         raise ValueError("角度必须是有限数")
41     return angle % 360.0
42
43
44 def circular_distance_deg(first: float, second: float) -> float:
45     delta = (normalize_angle_deg(first) - normalize_angle_deg(second) + 180.0) % 360.0
46     return abs(delta - 180.0)
47
48
49 def distance(first: Point, second: Point) -> float:
50     _require_finite((*first, *second), "points")
51     return math.hypot(first[0] - second[0], first[1] - second[1])
52

```

```

53
54 def bearing_deg(origin: Point, target: Point) -> float:
55     _require_finite((*origin, *target), "points")
56     dx = target[0] - origin[0]
57     dy = target[1] - origin[1]
58     if dx == 0.0 and dy == 0.0:
59         raise ValueError("两点重合时示向角没有定义")
60     return normalize_angle_deg(math.degrees(math.atan2(dy, dx)))
61
62
63 def ray_disk_interval(
64     origin: Point,
65     direction_rad: float,
66     center: Point,
67     radius: float,
68     *,
69     tolerance: float = 1e-12,
70 ) -> tuple[float, float] | None:
71
72     _require_finite((*origin, direction_rad, *center, radius, tolerance), "ray/disk data")
73     if radius <= 0.0:
74         raise ValueError("半径必须为正数")
75     if tolerance <= 0.0:
76         raise ValueError("容差必须为正数")
77     ux = math.cos(direction_rad)
78     uy = math.sin(direction_rad)
79     vx = origin[0] - center[0]
80     vy = origin[1] - center[1]
81     projection = vx * ux + vy * uy
82     discriminant = projection * projection - (vx * vx + vy * vy - radius * radius)
83     if discriminant < -tolerance:
84         return None
85     root = math.sqrt(max(0.0, discriminant))
86     lower_raw = -projection - root
87     upper_raw = -projection + root
88     if upper_raw < -tolerance:
89         return None
90     lower = max(0.0, lower_raw)
91     upper = max(0.0, upper_raw)
92     if lower > upper + tolerance:
93         return None
94     return (lower, upper)
95
96
97 def classify_branch(
98     source: Point,
99     candidate: Point,
100     receive_radius: float,
101     near_radius: float,
102     *,
103     tolerance: float = 1e-9,
104 ) -> Branch:
105     _require_finite(

```

```

106         (*source, *candidate, receive_radius, near_radius, tolerance),
107         "branch data",
108     )
109     if near_radius < 0.0 or receive_radius <= near_radius:
110         raise ValueError("参数必须满足 0 <= near_radius < receive_radius")
111     separation = distance(source, candidate)
112     if separation <= near_radius + tolerance:
113         return Branch.NEAR
114     if separation <= receive_radius + tolerance:
115         return Branch.DIRECTION
116     return Branch.NO_SIGNAL
117
118
119 def pure_angle_diameter(
120     first_point: Point,
121     first_bearing_deg: float,
122     second_point: Point,
123     second_bearing_deg: float,
124     *,
125     error_deg: float = 1.0,
126     tolerance: float = 1e-10,
127     verify: bool = False,
128 ) -> tuple[float, str]:
129     """返回扩展直径及底层 Q1 状态。"""
130
131     result = solve_observations(
132         [
133             (first_point[0], first_point[1], first_bearing_deg),
134             (second_point[0], second_point[1], second_bearing_deg),
135         ],
136         error_deg=error_deg,
137         tolerance=tolerance,
138         verify=verify,
139     )
140     if result.status is RegionStatus.VALID and result.diameter is not None:
141         return (result.diameter, result.status.value)
142     return (math.inf, result.status.value)
143
144
145 @dataclass(frozen=True)
146 class AngleQualityTable:
147
148     step_deg: float
149     diameters: tuple[float, ...]
150     statuses: tuple[str, ...]
151
152     @classmethod
153     def build(
154         cls,
155         first_point: Point,
156         first_bearing_deg: float,
157         second_point: Point,
158         *,

```

```

159     error_deg: float,
160     requested_step_deg: float,
161     tolerance: float = 1e-10,
162     verify: bool = False,
163 ) -> "AngleQualityTable":
164     if not math.isfinite(requested_step_deg) or requested_step_deg <= 0.0:
165         raise ValueError("requested_step_deg 必须是正有限数")
166     effective_step = (
167         min(requested_step_deg, 2.0 * error_deg)
168         if error_deg > 0.0
169         else requested_step_deg
170     )
171     count = max(8, math.ceil(360.0 / effective_step))
172     step = 360.0 / count
173     diameters: list[float] = []
174     statuses: list[str] = []
175     for index in range(count):
176         value, status = pure_angle_diameter(
177             first_point,
178             first_bearing_deg,
179             second_point,
180             index * step,
181             error_deg=error_deg,
182             tolerance=tolerance,
183             verify=verify,
184         )
185         diameters.append(value)
186         statuses.append(status)
187     return cls(step, tuple(diameters), tuple(statuses))
188
189 def worst_for_true_bearing(self, true_bearing_deg: float, error_deg: float) -> float:
190     if not math.isfinite(error_deg) or error_deg < 0.0:
191         raise ValueError("error_deg 必须是非负有限数")
192     count = len(self.diameters)
193     center = int(round(normalize_angle_deg(true_bearing_deg) / self.step_deg)) % count
194     span = math.ceil(error_deg / self.step_deg) + 1
195     selected: list[float] = []
196     for offset in range(-span, span + 1):
197         index = (center + offset) % count
198         angle = index * self.step_deg
199         if circular_distance_deg(angle, true_bearing_deg) <= error_deg + 1e-12:
200             selected.append(self.diameters[index])
201     if not selected:
202         selected.append(self.diameters[center])
203     return max(selected)
204
205 def status_counts(self) -> dict[str, int]:
206     return dict(sorted(Counter(self.statuses).items()))
207
208
209 __all__ = [
210     "AngleQualityTable",
211     "Branch",

```

```

212     "Point",
213     "bearing_deg",
214     "circular_distance_deg",
215     "classify_branch",
216     "distance",
217     "normalize_angle_deg",
218     "pure_angle_diameter",
219     "ray_disk_interval",
220 ]

```

Listing 7 q2_state.py

```

1  from __future__ import annotations
2
3  from dataclasses import dataclass
4  from enum import Enum
5  import math
6
7  from q2_geometry import Point, distance, normalize_angle_deg, ray_disk_interval
8
9
10 class DomainStatus(str, Enum):
11     EMPTY = "EMPTY"
12     DEGENERATE = "DEGENERATE"
13     REGULAR = "REGULAR"
14
15
16 @dataclass(frozen=True)
17 class Q2Config:
18     center: Point = (0.0, 0.0)
19     target_radius: float = 1800.0
20     error_deg: float = 1.0
21     near_radius: float = 5.0
22     receive_min: float = 1000.0
23     receive_max: float = 1500.0
24
25     def validate(self) -> None:
26         values = (*self.center, self.target_radius, self.error_deg, self.near_radius,
27                  self.receive_min, self.receive_max)
28         if not all(math.isfinite(value) for value in values):
29             raise ValueError("配置值必须是有限数")
30         if self.target_radius <= 0.0 or self.error_deg < 0.0:
31             raise ValueError("target_radius 必须为正数, error_deg 必须非负")
32         if not 0.0 <= self.near_radius < self.receive_min < self.receive_max:
33             raise ValueError("参数必须满足 0 <= near_radius < receive_min < receive_max")
34
35
36 @dataclass(frozen=True)
37 class FirstObservation:
38     point: Point
39     bearing_deg: float
40
41     def validate(self) -> None:

```

```

42         if not all(math.isfinite(value) for value in (*self.point, self.bearing_deg)):
43             raise ValueError("第一次观测值必须是有限数")
44
45
46 @dataclass(frozen=True)
47 class SourceCell:
48     point: Point
49     distance_from_first: float
50     weight_m2: float
51
52
53 @dataclass(frozen=True)
54 class StateDomain:
55     status: DomainStatus
56     samples: tuple[SourceCell, ...]
57     support_points: tuple[Point, ...]
58     possible_radius_range: tuple[float, float] | None
59     area_radius_range: tuple[float, float] | None
60     approximate_area_m2: float
61     angle_cells: int
62     radial_cells: int
63
64     @property
65     def has_positive_area(self) -> bool:
66         return self.status is DomainStatus.REGULAR and bool(self.samples)
67
68
69 def _clip_interval(
70     interval: tuple[float, float] | None,
71     lower: float,
72     upper: float,
73     *,
74     tolerance: float,
75 ) -> tuple[float, float] | None:
76     if interval is None:
77         return None
78     lo = max(interval[0], lower)
79     hi = min(interval[1], upper)
80     if hi <= lower + tolerance or hi < lo - tolerance:
81         return None
82     return (lo, hi)
83
84
85 def _point_on_ray(origin: Point, angle_rad: float, radius: float) -> Point:
86     return (
87         origin[0] + radius * math.cos(angle_rad),
88         origin[1] + radius * math.sin(angle_rad),
89     )
90
91
92 def build_state_domain(
93     first: FirstObservation,
94     config: Q2Config,

```

```

95     *,
96     angle_cells: int = 25,
97     radial_cells: int = 20,
98     tolerance: float = 1e-9,
99 ) -> StateDomain:
100
101     first.validate()
102     config.validate()
103     if angle_cells <= 0 or radial_cells <= 0:
104         raise ValueError("angle_cells 和 radial_cells 必须为正数")
105
106     center_rad = math.radians(normalize_angle_deg(first.bearing_deg))
107     half_width = math.radians(config.error_deg)
108     angle_width = 2.0 * half_width / angle_cells if angle_cells else 0.0
109     sample_cells: list[SourceCell] = []
110     support_points: list[Point] = []
111     logical_radai: list[float] = []
112
113     boundary_angles = [center_rad - half_width, center_rad + half_width]
114     if config.error_deg == 0.0:
115         boundary_angles = [center_rad]
116     for angle in boundary_angles:
117         interval = _clip_interval(
118             ray_disk_interval(first.point, angle, config.center, config.target_radius),
119             config.near_radius,
120             config.receive_max,
121             tolerance=tolerance,
122         )
123         if interval is None:
124             continue
125         logical_radai.extend(interval)
126         support_points.extend(
127             [_point_on_ray(first.point, angle, interval[0]),
128              _point_on_ray(first.point, angle, interval[1])]
129         )
130
131     if config.error_deg == 0.0:
132         mid_angles = [center_rad]
133         cell_widths = [0.0]
134     else:
135         mid_angles = [center_rad - half_width + (index + 0.5) * angle_width
136                       for index in range(angle_cells)]
137         cell_widths = [angle_width] * angle_cells
138
139     for angle, cell_width in zip(mid_angles, cell_widths):
140         interval = _clip_interval(
141             ray_disk_interval(first.point, angle, config.center, config.target_radius),
142             config.near_radius,
143             config.receive_max,
144             tolerance=tolerance,
145         )
146         if interval is None:
147             continue

```

```

148     lo, hi = interval
149     logical_radial.extend((lo, hi))
150     support_points.extend(
151         [_point_on_ray(first.point, angle, lo), _point_on_ray(first.point, angle, hi)]
152     )
153     if cell_width <= 0.0 or hi <= lo + tolerance:
154         continue
155     lo2 = lo * lo
156     hi2 = hi * hi
157     for radial_index in range(radial_cells):
158         left2 = lo2 + (hi2 - lo2) * radial_index / radial_cells
159         right2 = lo2 + (hi2 - lo2) * (radial_index + 1) / radial_cells
160         radius = math.sqrt(0.5 * (left2 + right2))
161         weight = 0.5 * (right2 - left2) * cell_width
162         sample_cells.append(
163             SourceCell(_point_on_ray(first.point, angle, radius), radius, weight)
164         )
165
166     if not logical_radial:
167         return StateDomain(
168             DomainStatus.EMPTY, (), (), None, None, 0.0, angle_cells, radial_cells
169         )
170
171     possible_low = max(config.receive_min, min(logical_radial))
172     possible_range = (
173         (possible_low, config.receive_max)
174         if possible_low <= config.receive_max + tolerance
175         else None
176     )
177     positive_samples = tuple(cell for cell in sample_cells if cell.weight_m2 > 0.0)
178     if not positive_samples:
179         status = DomainStatus.DEGENERATE if possible_range is not None else DomainStatus.
180         EMPTY
181         return StateDomain(
182             status, (), tuple(support_points), possible_range, None, 0.0,
183             angle_cells, radial_cells
184         )
185
186     area_low = max(config.receive_min, min(cell.distance_from_first for cell in
187         positive_samples))
188     area_range = (
189         (area_low, config.receive_max)
190         if area_low <= config.receive_max + tolerance
191         else None
192     )
193     if possible_range is None or area_range is None:
194         return StateDomain(
195             DomainStatus.EMPTY, (), tuple(support_points), possible_range, area_range,
196             0.0, angle_cells, radial_cells
197         )
198     return StateDomain(
199         DomainStatus.REGULAR,
200         positive_samples,

```

```

199         tuple(support_points),
200         possible_range,
201         area_range,
202         sum(cell.weight_m2 for cell in positive_samples),
203         angle_cells,
204         radial_cells,
205     )
206
207
208 def radius_grid(domain: StateDomain, count: int) -> tuple[float, ...]:
209     if count < 2:
210         raise ValueError("接收半径节点数必须至少为 2")
211     if domain.area_radius_range is None:
212         return ()
213     lo, hi = domain.area_radius_range
214     if math.isclose(lo, hi):
215         return (lo,)
216     return tuple(lo + (hi - lo) * index / (count - 1) for index in range(count))
217
218
219 def candidate_grid(
220     domain: StateDomain,
221     first: FirstObservation,
222     config: Q2Config,
223     *,
224     step_m: float,
225     max_candidates: int = 400,
226 ) -> tuple[Point, ...]:
227
228     if not math.isfinite(step_m) or step_m <= 0.0:
229         raise ValueError("step_m 必须是正有限数")
230     if max_candidates <= 0:
231         raise ValueError("max_candidates 必须为正数")
232     points = list(domain.support_points) or [cell.point for cell in domain.samples]
233     if not points:
234         return ()
235     min_x = math.floor((min(point[0] for point in points) - config.receive_max) / step_m) *
236         step_m
237     max_x = math.ceil((max(point[0] for point in points) + config.receive_max) / step_m) *
238         step_m
239     min_y = math.floor((min(point[1] for point in points) - config.receive_max) / step_m) *
240         step_m
241     max_y = math.ceil((max(point[1] for point in points) + config.receive_max) / step_m) *
242         step_m
243     nx = math.floor((max_x - min_x) / step_m) + 1
244     ny = math.floor((max_y - min_y) / step_m) + 1
245     if nx * ny > max_candidates:
246         required = math.sqrt((max_x - min_x) * (max_y - min_y) / max_candidates)
247         raise ValueError(
248             f"候选栅格将包含 {nx * ny} 个点; "
249             f"请将 --candidate-step 增大到约 {required:.1f} m, 或提高数量上限"
250         )
251     candidates: list[Point] = []

```

```

248     for ix in range(nx):
249         x = min_x + ix * step_m
250         for iy in range(ny):
251             point = (x, min_y + iy * step_m)
252             if distance(point, first.point) > 1e-9:
253                 candidates.append(point)
254     return tuple(candidates)
255
256
257 __all__ = [
258     "DomainStatus",
259     "FirstObservation",
260     "Q2Config",
261     "SourceCell",
262     "StateDomain",
263     "build_state_domain",
264     "candidate_grid",
265     "radius_grid",
266 ]

```

Listing 8 run_q2_fast.py

```

1  from __future__ import annotations
2
3  import argparse
4  from datetime import datetime
5  import json
6  from pathlib import Path
7  from uuid import uuid4
8
9  from q2_fast import FastOptions, solve_fast_q2
10 from q2_state import FirstObservation, Q2Config
11
12
13 def parse_args() -> argparse.Namespace:
14     parser = argparse.ArgumentParser(
15         description="Q2 快速保守下一测点选择器"
16     )
17     parser.add_argument("--first-x", type=float, default=0.0)
18     parser.add_argument("--first-y", type=float, default=0.0)
19     parser.add_argument("--first-bearing", type=float, default=0.0)
20     parser.add_argument("--circle-sides", type=int, default=48)
21     parser.add_argument("--coarse-sources", type=int, default=24)
22     parser.add_argument("--final-sources", type=int, default=64)
23     parser.add_argument("--local-rounds", type=int, default=4)
24     parser.add_argument("--output", type=Path)
25     parser.add_argument("--no-write", action="store_true")
26     return parser.parse_args()
27
28
29 def default_output_path() -> Path:
30     root = Path(__file__).resolve().parents[1] / "results"
31     stamp = datetime.now().strftime("%Y%m%d_%H%M%S")

```

```

32     return root / f"q2_fast_{stamp}_{uuid4().hex[:8]}.json"
33
34
35 def main() -> int:
36     args = parse_args()
37     first = FirstObservation((args.first_x, args.first_y), args.first_bearing)
38     options = FastOptions(
39         circle_sides=args.circle_sides,
40         coarse_source_count=args.coarse_sources,
41         final_source_count=args.final_sources,
42         local_rounds=args.local_rounds,
43     )
44     result = solve_fast_q2(first, Q2Config(), options)
45     output = None if args.no_write else (args.output or default_output_path()).resolve()
46     result["output"] = str(output) if output is not None else None
47     if output is not None:
48         output.parent.mkdir(parents=True, exist_ok=True)
49         output.write_text(
50             json.dumps(result, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
51         )
52     print(json.dumps(result, ensure_ascii=False, indent=2))
53     return 0
54
55
56 if __name__ == "__main__":
57     raise SystemExit(main())

```

Listing 9 run_q2_three_tasks.py

```

1  from __future__ import annotations
2
3  import json
4  import math
5  from pathlib import Path
6  import time
7  import numpy as np
8  import matplotlib
9  import matplotlib.pyplot as plt
10 from scipy.ndimage import label
11
12 from q2_fast import (
13     build_position_outer_polygon,
14     conservative_safety_margin,
15     evaluate_candidate,
16     solve_fast_q2,
17     _representative_sources,
18     _error_grid,
19     FastOptions,
20 )
21 from q2_state import FirstObservation, Q2Config, build_state_domain
22 from run_q2_validation import SCENARIOS
23
24 plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'DejaVu Sans', 'Arial']

```

```

25 plt.rcParams['axes.unicode_minus'] = False
26 plt.rcParams['mathtext.fontset'] = 'cm'
27 plt.rcParams['pdf.fonttype'] = 42
28 plt.rcParams['svg.fonttype'] = 'none'
29
30 OUTPUT_DIR = Path(__file__).resolve().parent.parent / "results"
31 OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
32
33
34 def run_task1_candidate_region_scan(
35     step_m: float = 25.0,
36     coarse_sources: int = 24,
37     coarse_errors: int = 3,
38     save_plot: bool = True,
39 ) -> dict[str, object]:
40     """任务 1: 对圆心场景的保守安全区域 U 进行栅格扫描。"""
41     print("Running Task 1: Near-optimal candidate region grid scan")
42     first = FirstObservation((0.0, 0.0), 0.0)
43     config = Q2Config()
44     opts = FastOptions()
45
46     poly = build_position_outer_polygon(first, config, opts.circle_sides)
47     domain = build_state_domain(first, config)
48     sources = _representative_sources(domain, coarse_sources)
49     errors = _error_grid(config.error_deg, coarse_errors)
50
51     radius = config.receive_min
52     x_min_raw = max(vertex[0] - radius for vertex in poly)
53     x_max_raw = min(vertex[0] + radius for vertex in poly)
54     y_min_raw = max(vertex[1] - radius for vertex in poly)
55     y_max_raw = min(vertex[1] + radius for vertex in poly)
56     x_min = math.ceil(x_min_raw / step_m) * step_m
57     x_max = math.floor(x_max_raw / step_m) * step_m
58     y_min = math.ceil(y_min_raw / step_m) * step_m
59     y_max = math.floor(y_max_raw / step_m) * step_m
60
61     xs = np.arange(x_min, x_max + step_m * 0.5, step_m)
62     ys = np.arange(y_min, y_max + step_m * 0.5, step_m)
63     X, Y = np.meshgrid(xs, ys)
64
65     Z = np.full(X.shape, np.nan)
66     safe_mask = np.zeros(X.shape, dtype=bool)
67
68     safe_points = []
69     t0 = time.perf_counter()
70     for i in range(X.shape[0]):
71         for j in range(X.shape[1]):
72             pt = (float(X[i, j]), float(Y[i, j]))
73             margin = conservative_safety_margin(pt, poly, config.receive_min)
74             if margin >= 0:
75                 safe_mask[i, j] = True
76                 score = evaluate_candidate(pt, "grid_scan", first, config, poly, sources,
77                                         errors)

```

```

77         Z[i, j] = score.worst_posterior_diameter_m
78         safe_points.append((pt, score.worst_posterior_diameter_m))
79
80     elapsed_scan = time.perf_counter() - t0
81     safe_count = len(safe_points)
82     j_star = float(np.nanmin(Z))
83     best_idx = np.unravel_index(np.nanargmin(Z), Z.shape)
84     best_pt = (float(X[best_idx]), float(Y[best_idx]))
85
86     print(f"    Grid points in U: {safe_count}, min diameter J* = {j_star:.2f} m at {
87         best_pt}, scan time: {elapsed_scan:.2f}s")
88
89     # 计算 tau = 1%、3%、5% 时的指标。
90     cell_area_m2 = step_m * step_m
91     tau_results = {}
92     structure = np.ones((3, 3), dtype=int)
93
94     for tau in [0.01, 0.03, 0.05]:
95         threshold = (1.0 + tau) * j_star
96         in_tau = safe_mask & (Z <= threshold)
97         cell_count = int(np.sum(in_tau))
98         area_m2 = cell_count * cell_area_m2
99         area_km2 = area_m2 / 1e6
100
101         labeled_arr, num_features = label(in_tau, structure=structure)
102
103         tau_xs = X[in_tau]
104         tau_ys = Y[in_tau]
105         if len(tau_xs) > 0:
106             bbox = {
107                 "x_min": float(np.min(tau_xs)),
108                 "x_max": float(np.max(tau_xs)),
109                 "y_min": float(np.min(tau_ys)),
110                 "y_max": float(np.max(tau_ys)),
111             }
112         else:
113             bbox = None
114
115         tau_results[f"tau_{int(tau*100)}pct"] = {
116             "tau": tau,
117             "threshold_diameter_m": float(threshold),
118             "cell_count": cell_count,
119             "area_m2": float(area_m2),
120             "area_km2": float(area_km2),
121             "connected_components": int(num_features),
122             "bounding_box": bbox,
123         }
124
125         print(f"    tau = {int(tau*100)}%: Area = {area_km2:.4f} km2, {num_features}
126             connected components, BBox X=[{bbox['x_min']:.0f}, {bbox['x_max']:.0f}], Y=[{bbox['
127             y_min']:.0f}, {bbox['y_max']:.0f}]"

```

```

127     fig, axes = plt.subplots(
128         2, 1, figsize=(7.2, 5.4), dpi=300, sharex=True,
129         constrained_layout=True, facecolor="white"
130     )
131     levels = np.linspace(j_star, j_star * 1.08, 17)
132     colors = ["#1e40af", "#047857", "#b45309"]
133     styles = ["-", "--", ":"]
134     panel_limits = [(360, 660), (-660, -360)]
135     recommended = [(839.64, 538.75), (839.64, -538.75)]
136     for ax, ylim, point in zip(axes, panel_limits, recommended):
137         ax.set_facecolor("white")
138         cf = ax.contourf(X, Y, Z, levels=levels, cmap="YlOrRd_r", alpha=0.9)
139         for idx, tau in enumerate([0.01, 0.03, 0.05]):
140             threshold = (1.0 + tau) * j_star
141             ax.contour(
142                 X, Y, Z, levels=[threshold], colors=[colors[idx]],
143                 linewidths=1.6, linestyles=styles[idx]
144             )
145         ax.plot(
146             point[0], point[1], marker='*', color='#dc2626',
147             linestyle='none', markersize=11, zorder=10
148         )
149         ax.set_xlim(720, 930)
150         ax.set_ylim(*ylim)
151         ax.set_ylabel('Y 坐标 (m)', fontsize=8)
152         ax.tick_params(labelsize=7)
153         ax.grid(True, linestyle=':', alpha=0.45, color='#cbd5e1')
154
155     axes[-1].set_xlabel('X 坐标 (m)', fontsize=8)
156     legend_handles = []
157     for idx, tau in enumerate([0.01, 0.03, 0.05]):
158         threshold = (1.0 + tau) * j_star
159         handle, = axes[0].plot(
160             [], [], color=colors[idx], lw=1.6, linestyle=styles[idx],
161             label=f"${mathcal{C}}_{{int(tau*100)}\\%}$ ($\\leq {threshold:.1f}$ m)"
162         )
163         legend_handles.append(handle)
164     star, = axes[0].plot(
165         [], [], marker='*', color='#dc2626', linestyle='none', markersize=9,
166         label='推荐测点'
167     )
168     legend_handles.append(star)
169     axes[0].legend(
170         handles=legend_handles, loc='upper right', frameon=False,
171         fontsize=7, ncol=2
172     )
173     cbar = fig.colorbar(cf, ax=axes, shrink=0.92, pad=0.02)
174     cbar.set_label(r"最坏后验直径  $\widehat{J}(\mathbf{p})$  (m)", fontsize=8)
175     cbar.ax.tick_params(labelsize=7)
176     fig.suptitle(
177         r'近优候选域  $\mathcal{C}_\tau$  的双侧结构与目标等值线',
178         fontsize=10, fontweight='bold'
179     )

```

```

180
181     plot_path = OUTPUT_DIR / "fig_q2_candidate_contour.png"
182     fig.savefig(plot_path, dpi=300)
183     fig.savefig(OUTPUT_DIR / "fig_q2_candidate_contour.pdf")
184     fig.savefig(OUTPUT_DIR / "fig_q2_candidate_contour.svg")
185     fig.savefig(OUTPUT_DIR / "fig_q2_candidate_contour.tiff", dpi=600)
186     plt.close(fig)
187     print(f"    Saved contour plot to: {plot_path}")
188
189     return {
190         "step_m": step_m,
191         "j_star_m": j_star,
192         "best_grid_point_m": best_pt,
193         "evaluated_safe_points": safe_count,
194         "grid_scan_bounds_m": {
195             "x_min": float(x_min), "x_max": float(x_max),
196             "y_min": float(y_min), "y_max": float(y_max),
197         },
198         "elapsed_seconds": elapsed_scan,
199         "tau_metrics": tau_results,
200         "plot_path": str(plot_path) if plot_path else None,
201     }
202
203
204 def run_task2_benchmark(repetitions: int = 20) -> dict[str, object]:
205     """任务 2: 在全部六个验证场景中对 solve_fast_q2 进行 20 次基准测试。"""
206     print(f"Running Task 2: Multi-run benchmark ({repetitions} repetitions per scenario)")
207     results = {}
208     for sc in SCENARIOS:
209         times = []
210         best_point = None
211         worst_diam = None
212         for r in range(repetitions):
213             res = solve_fast_q2(sc.first)
214             times.append(res["elapsed_seconds"])
215             if r == 0:
216                 rec = res["recommendations"][0]
217                 best_point = rec["point_m"]
218                 worst_diam = rec["worst_posterior_diameter_m"]
219
220         arr = np.array(times)
221         stats = {
222             "scenario": sc.name,
223             "repetitions": repetitions,
224             "mean_s": float(np.mean(arr)),
225             "std_s": float(np.std(arr)),
226             "median_s": float(np.median(arr)),
227             "min_s": float(np.min(arr)),
228             "max_s": float(np.max(arr)),
229             "p95_s": float(np.percentile(arr, 95)),
230             "best_point_m": best_point,
231             "worst_posterior_diameter_m": worst_diam,
232         }

```

```

233     results[sc.name] = stats
234     print(f"    Scenario '{sc.name}': Mean={stats['mean_s']:.3f}s, Std={stats['std_s']:.3f}s, Median={stats['median_s']:.3f}s, Max={stats['max_s']:.3f}s")
235
236     return results
237
238
239 def run_task3_safety_buffer_sensitivity() -> dict[str, object]:
240     """任务 3: 物理安全缓冲参数 (b > 0) 的敏感性分析."""
241
242     center_first = FirstObservation((0.0, 0.0), 0.0)
243     center_buffers = [0.0, 1.0, 2.0, 5.0, 10.0, 20.0]
244     center_results = []
245
246     for b in center_buffers:
247         res = solve_fast_q2(center_first, safety_buffer_m=b)
248         if res["result_status"] == "FAST_NUMERICAL_SOLUTION":
249             rec = res["recommendations"][0]
250             true_margin = rec["conservative_safety_margin_m"] + b
251             center_results.append({
252                 "buffer_b_m": b,
253                 "status": "FEASIBLE",
254                 "point_m": rec["point_m"],
255                 "movement_m": rec["movement_m"],
256                 "true_safety_margin_m": float(true_margin),
257                 "margin_slack_above_buffer_m": float(rec["conservative_safety_margin_m"]),
258                 "worst_posterior_diameter_m": rec["worst_posterior_diameter_m"],
259             })
260             print(f"    Center (b={b}m): pt=({rec['point_m'][0]:.1f}, {rec['point_m'][1]:.1f}), move={rec['movement_m']:.1f}m, true_margin={true_margin:.2f}m, D2={rec['worst_posterior_diameter_m']:.2f}m")
261         else:
262             center_results.append({
263                 "buffer_b_m": b,
264                 "status": res["result_status"],
265                 "point_m": None,
266                 "movement_m": None,
267                 "true_safety_margin_m": None,
268                 "margin_slack_above_buffer_m": None,
269                 "worst_posterior_diameter_m": None,
270             })
271             print(f"    Center (b={b}m): INFEASIBLE (U is empty)")
272
273     outside_first = FirstObservation((2000.0, 0.0), 180.0)
274     outside_buffers = [0.0, 1.0, 10.0, 100.0, 300.0, 340.0, 349.0, 350.0]
275     outside_results = []
276
277     for b in outside_buffers:
278         res = solve_fast_q2(outside_first, safety_buffer_m=b)
279         if res["result_status"] == "FAST_NUMERICAL_SOLUTION":
280             rec = res["recommendations"][0]
281             true_margin = rec["conservative_safety_margin_m"] + b
282             outside_results.append({

```

```

283         "buffer_b_m": b,
284         "status": "FEASIBLE",
285         "point_m": rec["point_m"],
286         "movement_m": rec["movement_m"],
287         "true_safety_margin_m": float(true_margin),
288         "margin_slack_above_buffer_m": float(rec["conservative_safety_margin_m"]),
289         "worst_posterior_diameter_m": rec["worst_posterior_diameter_m"],
290     })
291     print(f"    Outside (b={b}m): pt=({rec['point_m'][0]:.1f}, {rec['point_m']
' ][1]:.1f}), move={rec['movement_m']:.1f}m, true_margin={true_margin:.2f}m, D2={rec['
worst_posterior_diameter_m']:.2f}m")
292     else:
293         outside_results.append({
294             "buffer_b_m": b,
295             "status": res["result_status"],
296             "point_m": None,
297             "movement_m": None,
298             "true_safety_margin_m": None,
299             "margin_slack_above_buffer_m": None,
300             "worst_posterior_diameter_m": None,
301         })
302     print(f"    Outside (b={b}m): INFEASIBLE (U is empty)")
303
304     return {
305         "center_scenario": center_results,
306         "outside_inward_scenario": outside_results,
307     }
308
309
310 def main():
311     print("Starting Q2 Deep Enhancement Suite (Tasks 1, 2, 3)")
312
313     task1 = run_task1_candidate_region_scan(step_m=25.0, save_plot=True)
314     task2 = run_task2_benchmark(repetitions=20)
315     task3 = run_task3_safety_buffer_sensitivity()
316
317     full_output = {
318         "timestamp": time.strftime("%Y-%m-%d %H:%M:%S"),
319         "task1_candidate_regions": task1,
320         "task2_runtime_benchmark": task2,
321         "task3_safety_buffer_sensitivity": task3,
322     }
323
324     json_path = OUTPUT_DIR / "q2_deep_enhancement_results.json"
325     with open(json_path, "w", encoding="utf-8") as f:
326         json.dump(full_output, f, ensure_ascii=False, indent=2)
327     print(f"\nAll results successfully exported to: {json_path}")
328
329
330 if __name__ == "__main__":
331     main()

```

Listing 10 run_q2_validation.py

```

1  """用于 Q2 第四阶段验证的多场景批量收敛运行程序。"""
2
3  from __future__ import annotations
4
5  import argparse
6  from dataclasses import dataclass
7  from datetime import datetime
8  import json
9  from pathlib import Path
10 import time
11 import uuid
12
13 from q2_regions import ResolutionLevel, solve_numerical_regions
14 from q2_state import FirstObservation, Q2Config
15
16
17 DEFAULT_RESULTS_DIR = Path(__file__).resolve().parent.parent / "results"
18
19
20 @dataclass(frozen=True)
21 class Scenario:
22     name: str
23     first: FirstObservation
24
25
26 SCENARIOS = (
27     Scenario("center", FirstObservation((0.0, 0.0), 0.0)),
28     Scenario("offset", FirstObservation((300.0, 500.0), 125.0)),
29     Scenario("edge_inward", FirstObservation((1600.0, 0.0), 180.0)),
30     Scenario("edge_tangent", FirstObservation((1600.0, 0.0), 90.0)),
31     Scenario("outside_inward", FirstObservation((2000.0, 0.0), 180.0)),
32     Scenario("wraparound", FirstObservation((0.0, 0.0), 359.5)),
33 )
34
35
36 def _levels(profile: str) -> tuple[ResolutionLevel, ...]:
37     if profile == "quick":
38         return (
39             ResolutionLevel(1000.0, 9, 8, 5, 2.0),
40             ResolutionLevel(500.0, 13, 12, 7, 1.0),
41             ResolutionLevel(250.0, 17, 16, 9, 0.5),
42         )
43     return (
44         ResolutionLevel(500.0, 25, 24, 9, 1.0),
45         ResolutionLevel(250.0, 37, 36, 13, 0.5),
46         ResolutionLevel(125.0, 49, 48, 17, 0.25),
47     )
48
49
50 def _parser() -> argparse.ArgumentParser:
51     parser = argparse.ArgumentParser(description="运行 Q2 多场景收敛验证")

```

```

52 parser.add_argument("--profile", choices=("quick", "formal"), default="quick")
53 parser.add_argument("--scenario", action="append", choices=tuple(item.name for item in
SCENARIOS))
54 parser.add_argument("--output-dir", type=Path, default=DEFAULT_RESULTS_DIR)
55 parser.add_argument("--max-candidates", type=int, default=2200)
56 return parser
57
58
59 def main() -> None:
60     args = _parser().parse_args()
61     selected = [item for item in SCENARIOS if not args.scenario or item.name in args.
scenario]
62     run_id = f"q2_validation_{datetime.now():%Y%m%d_%H%M%S}_{str(uuid.uuid4())[:8]}"
63     run_dir = args.output_dir / run_id
64     run_dir.mkdir(parents=True, exist_ok=False)
65     levels = _levels(args.profile)
66     config = Q2Config()
67     index: dict[str, object] = {
68         "run_id": run_id,
69         "profile": args.profile,
70         "evidence": "DIAGNOSTIC" if args.profile == "quick" else "
STAGE4_NUMERICAL_VALIDATION",
71         "levels": [level.to_dict() for level in levels],
72         "scenarios": [],
73     }
74     started = time.perf_counter()
75     for scenario in selected:
76         result = solve_numerical_regions(
77             scenario.first, config, levels, max_candidates=args.max_candidates
78         )
79         result["scenario"] = {
80             "name": scenario.name,
81             "first_point": list(scenario.first.point),
82             "first_bearing_deg": scenario.first.bearing_deg,
83         }
84         path = run_dir / f"{scenario.name}.json"
85         path.write_text(json.dumps(result, ensure_ascii=False, indent=2, allow_nan=False),
encoding="utf-8")
86         index["scenarios"].append({
87             "name": scenario.name,
88             "file": path.name,
89             "status": result["result_status"],
90             "converged": result.get("convergence_assessment", {}).get("converged", False),
91             "convergence": result.get("convergence", []),
92         })
93     index["elapsed_seconds"] = time.perf_counter() - started
94     index_path = run_dir / "index.json"
95     index_path.write_text(json.dumps(index, ensure_ascii=False, indent=2, allow_nan=False),
encoding="utf-8")
96     print(json.dumps({"run_id": run_id, "profile": args.profile, "scenario_count": len(
selected),
97         "elapsed_seconds": index["elapsed_seconds"], "index": str(index_path)
},

```

```

98         ensure_ascii=False, indent=2))
99
100
101 if __name__ == "__main__":
102     main()

```

C 问题三程序代码

Listing 11 q3_strategy.py

```

1 from __future__ import annotations
2
3 from dataclasses import dataclass, field
4 from enum import Enum
5 import math
6 from pathlib import Path
7 import sys
8 from typing import Sequence
9
10 _PROJECT_ROOT = Path(__file__).resolve().parents[2]
11 _SHARED_CODE = _PROJECT_ROOT / "code"
12 _Q1_CODE = _PROJECT_ROOT / "q1" / "code"
13 _Q2_CODE = _PROJECT_ROOT / "q2" / "code"
14 for _code_dir in (_SHARED_CODE, _Q1_CODE, _Q2_CODE):
15     if str(_code_dir) not in sys.path:
16         sys.path.insert(0, str(_code_dir))
17
18 from radio_sim.domain.models import ( # noqa: E402
19     ClearResult,
20     ClearStatus,
21     MeasureResult,
22     MeasureStatus,
23     Position,
24     StrategyResult,
25 )
26 from radio_sim.environment.base import Environment, EnvironmentStateError # noqa: E402
27 from q2_fast import ( # noqa: E402
28     CandidateScore,
29     build_position_outer_polygon,
30     conservative_safety_margin,
31     evaluate_candidate,
32     generate_sparse_candidates,
33     minimum_enclosing_circle,
34     posterior_polygon,
35 )
36 from q2_state import FirstObservation, Q2Config # noqa: E402
37
38 Point = tuple[float, float]
39
40
41 class ChannelStatus(str, Enum):
42     UNKNOWN = "unknown"

```

```

43     ALIVE = "alive"
44     CLEARED = "cleared"
45     ABSENT_CERTIFIED = "absent_certified"
46
47
48     class AbsenceReason(str, Enum):
49         COVER_COMPLETE_NO_SIGNAL = "cover_complete_no_signal"
50         GLOBAL_COUNT_UPPER_BOUND = "global_count_upper_bound"
51
52
53     class RegionHealth(str, Enum):
54         EMPTY = "empty"
55         HEALTHY_CURRENT = "healthy_current"
56         SAFE_STALE = "safe_stale"
57         INVALID = "invalid"
58
59
60     @dataclass(frozen=True)
61     class ObservationEvent:
62         event_id: int
63         purpose: str
64         point: Point
65         status: MeasureStatus
66         angle_deg: float | None
67         virtual_time_s: float
68
69
70     @dataclass(frozen=True)
71     class ClearEvent:
72         event_id: int
73         purpose: str
74         point: Point
75         status: ClearStatus
76         virtual_time_s: float
77
78
79     @dataclass(frozen=True)
80     class LifecycleEvent:
81         event_id: int
82         old_status: ChannelStatus
83         new_status: ChannelStatus
84         reason: str
85
86
87     @dataclass(frozen=True)
88     class CompletionCertificate:
89         valid: bool
90         state_version: int
91         built_from_version: int
92         history_consistent: bool
93         pending_request_empty: bool
94         cleared_channels: tuple[int, ...]
95         absent_channels: tuple[int, ...]

```

```

96     per_channel_reasons: tuple[tuple[int, str], ...]
97     per_channel_evidence: tuple[
98         tuple[int, tuple[int, ...], tuple[Point, ...], tuple[int, ...]], ...
99     ]
100     global_reason: str | None
101
102     def to_record(self) -> dict[str, object]:
103         return {
104             "valid": self.valid,
105             "state_version": self.state_version,
106             "built_from_version": self.built_from_version,
107             "history_consistent": self.history_consistent,
108             "pending_request_empty": self.pending_request_empty,
109             "cleared_channels": list(self.cleared_channels),
110             "absent_channels": list(self.absent_channels),
111             "per_channel_reasons": {
112                 str(channel): reason for channel, reason in self.per_channel_reasons
113             },
114             "per_channel_evidence": {
115                 str(channel): {
116                     "event_ids": list(event_ids),
117                     "search_points": [list(point) for point in search_points],
118                     "confirmed_channels": list(confirmed_channels),
119                 }
120                 for channel, event_ids, search_points, confirmed_channels
121                 in self.per_channel_evidence
122             },
123             "global_reason": self.global_reason,
124         }
125
126
127 @dataclass
128 class ChannelTrack:
129     channel: int
130     status: ChannelStatus = ChannelStatus.UNKNOWN
131     polygon: list[Point] = field(default_factory=list)
132     first_point: Point | None = None
133     first_bearing_deg: float | None = None
134     last_point: Point | None = None
135     last_bearing_deg: float | None = None
136     active_measurements: int = 0
137     no_signal_during_localization: int = 0
138     observations: list[ObservationEvent] = field(default_factory=list)
139     clear_attempts: list[ClearEvent] = field(default_factory=list)
140     lifecycle_events: list[LifecycleEvent] = field(default_factory=list)
141     repeated_location_anchors: dict[Point, int] = field(default_factory=dict)
142     absence_reason: AbsenceReason | None = None
143     absence_evidence_event_ids: tuple[int, ...] = ()
144     absence_confirmed_channels: tuple[int, ...] = ()
145     region_health: RegionHealth = RegionHealth.EMPTY
146     state_version: int = 0
147     region_built_from_version: int = 0
148

```

```

149
150 @dataclass(frozen=True)
151 class Q3StrategyOptions:
152     bearing_error_deg: float = 1.005
153     safety_buffer_m: float = 5.0
154     max_active_measurements: int = 4
155     q2_circle_sides: int = 48
156     q2_source_samples: int = 30
157     q2_error_nodes: int = 3
158     q2_seed_count: int = 3
159     q2_local_rounds: int = 3
160     clear_radius_m: float = 20.0
161     clear_cell_side_m: float = 26.0
162     guaranteed_clear_radius_m: float = 19.5
163
164     def validate(self) -> None:
165         if not 1.0 <= self.bearing_error_deg < 90.0:
166             raise ValueError("bearing_error_deg 必须覆盖官方规定的 1° 误差")
167         if not 0.0 <= self.safety_buffer_m < 995.0:
168             raise ValueError("safety_buffer_m 必须位于 [0, 995) 内")
169         if self.max_active_measurements < 0:
170             raise ValueError("max_active_measurements 必须非负")
171         if self.q2_circle_sides < 8:
172             raise ValueError("q2_circle_sides 必须至少为 8")
173         if self.q2_source_samples <= 0 or self.q2_error_nodes <= 0:
174             raise ValueError("Q2 采样数量必须为正数")
175         if self.q2_seed_count <= 0 or self.q2_local_rounds < 0:
176             raise ValueError("Q2 局部搜索设置无效")
177         if not 0.0 < self.clear_cell_side_m < self.clear_radius_m * math.sqrt(2.0):
178             raise ValueError("清除单元的半对角线必须小于清除半径")
179         if not 0.0 < self.guaranteed_clear_radius_m <= self.clear_radius_m:
180             raise ValueError("保证清除半径无效")
181
182
183 SEARCH_RING_RADIUS_M = 1140.0
184 _SEARCH_RING_HEIGHT_M = SEARCH_RING_RADIUS_M * math.sqrt(3.0) / 2.0
185 SEARCH_ROUTE: tuple[Point, ...] = (
186     (0.0, 0.0),
187     (SEARCH_RING_RADIUS_M, 0.0),
188     (SEARCH_RING_RADIUS_M / 2.0, _SEARCH_RING_HEIGHT_M),
189     (-SEARCH_RING_RADIUS_M / 2.0, _SEARCH_RING_HEIGHT_M),
190     (-SEARCH_RING_RADIUS_M, 0.0),
191     (-SEARCH_RING_RADIUS_M / 2.0, -_SEARCH_RING_HEIGHT_M),
192     (SEARCH_RING_RADIUS_M / 2.0, -_SEARCH_RING_HEIGHT_M),
193 )
194
195
196 def search_cover_bound_m() -> float:
197     """计算圆心加正六边形覆盖在外边界上的解析最坏距离。"""
198     return math.sqrt(
199         1800.0**2
200         + SEARCH_RING_RADIUS_M**2
201         - 2.0 * 1800.0 * SEARCH_RING_RADIUS_M * math.cos(math.pi / 6.0)

```

```

202     )
203
204
205 def _point(position: Position) -> Point:
206     return (position.x, position.y)
207
208
209 def _position(point: Point) -> Position:
210     return Position(point[0], point[1])
211
212
213 def _distance(first: Point, second: Point) -> float:
214     return math.hypot(first[0] - second[0], first[1] - second[1])
215
216
217 def _deduplicate(points: Sequence[Point], tolerance: float = 1e-7) -> list[Point]:
218     result: list[Point] = []
219     for point in points:
220         if all(_distance(point, other) > tolerance for other in result):
221             result.append(point)
222     return result
223
224
225 def _fallback_initial_polygon(
226     point: Point, bearing_deg: float, error_deg: float, range_m: float = 1500.0
227 ) -> list[Point]:
228     """仅在正式外包交集退化时使用的保守三角形。"""
229     theta = math.radians(bearing_deg)
230     epsilon = math.radians(error_deg)
231     outer_ray_m = range_m / math.cos(epsilon)
232     return [
233         point,
234         (
235             point[0] + outer_ray_m * math.cos(theta - epsilon),
236             point[1] + outer_ray_m * math.sin(theta - epsilon),
237         ),
238         (
239             point[0] + outer_ray_m * math.cos(theta + epsilon),
240             point[1] + outer_ray_m * math.sin(theta + epsilon),
241         ),
242     ]
243
244
245 def _representative_polygon_points(
246     polygon: Sequence[Point], limit: int
247 ) -> list[Point]:
248     if not polygon:
249         return []
250     centroid = (
251         sum(point[0] for point in polygon) / len(polygon),
252         sum(point[1] for point in polygon) / len(polygon),
253     )
254     pool: list[Point] = list(polygon) + [centroid]

```

```

255     for index, first in enumerate(polygon):
256         second = polygon[(index + 1) % len(polygon)]
257         for fraction in (0.25, 0.5, 0.75):
258             pool.append(
259                 (
260                     first[0] + fraction * (second[0] - first[0]),
261                     first[1] + fraction * (second[1] - first[1]),
262                 )
263             )
264         pool.append(((first[0] + centroid[0]) / 2.0, (first[1] + centroid[1]) / 2.0))
265     pool = _deduplicate(pool)
266     if len(pool) <= limit:
267         return pool
268
269     selected = [max(pool, key=lambda item: _distance(item, centroid))]
270     while len(selected) < limit:
271         selected.append(
272             max(
273                 pool,
274                 key=lambda item: min(_distance(item, chosen) for chosen in selected),
275             )
276         )
277     return _deduplicate(selected)[:limit]
278
279
280 def _error_grid(error_deg: float, count: int) -> list[float]:
281     if count <= 1:
282         return [0.0]
283     return [
284         -error_deg + 2.0 * error_deg * index / (count - 1)
285         for index in range(count)
286     ]
287
288
289 def _score_key(
290     score: CandidateScore,
291     current: Point,
292     target: Point,
293 ) -> tuple[float, float, float, float]:
294     """惩罚过宽的后验区域"""
295     route_m = _distance(current, score.point) + _distance(score.point, target)
296     excess_diameter_m = max(0.0, score.worst_posterior_diameter_m - 39.0)
297     time_aware_cost_m = route_m + 0.85 * excess_diameter_m
298     return (
299         time_aware_cost_m,
300         score.worst_posterior_diameter_m,
301         route_m,
302         -score.conservative_safety_margin_m,
303     )
304
305
306 def _channel_order(channels: Sequence[int], current_channel: int) -> list[int]:
307     """优先扫描当前信道，再按单调顺序扫描"""

```

```

308     unique = sorted(set(channels))
309     if current_channel not in unique:
310         return unique
311     return [current_channel] + [channel for channel in unique if channel != current_channel
312 ]
313
314 class GuaranteedRollingQ3Strategy:
315     def __init__(self, options: Q3StrategyOptions | None = None) -> None:
316         self.options = options or Q3StrategyOptions()
317         self.options.validate()
318         self._tracks = {channel: ChannelTrack(channel) for channel in range(1, 21)}
319         self._q2_config = Q2Config(
320             center=(0.0, 0.0),
321             target_radius=1800.0,
322             error_deg=self.options.bearing_error_deg,
323             near_radius=5.0,
324             receive_min=1000.0 - self.options.safety_buffer_m,
325             receive_max=1500.0,
326         )
327         self._search_points_visited = 0
328         self._fallback_clear_attempts = 0
329         self._guaranteed_clear_attempts = 0
330         self._state_version = 0
331         self._polygon_rebuild_count = 0
332         self._polygon_rebuild_failure_count = 0
333         self._last_completion_certificate: CompletionCertificate | None = None
334
335     @property
336     def completion_certificate(self) -> CompletionCertificate | None:
337         return self._last_completion_certificate
338
339     def _next_state_version(self) -> int:
340         self._state_version += 1
341         return self._state_version
342
343     @staticmethod
344     def _has_pending_request(env: Environment) -> bool:
345         return bool(getattr(env, "has_pending_request", False))
346
347     def _call_with_pending_recovery(self, env: Environment, action: object, *args: object)
348     -> object:
349         try:
350             return action(*args) # type: ignore[operator]
351         except EnvironmentStateError:
352             if not self._has_pending_request(env):
353                 raise
354             recover = getattr(env, "recover_pending", None)
355             if not callable(recover):
356                 raise
357             return recover()
358
359     def _record_observation(

```

```

359         self,
360         track: ChannelTrack,
361         point: Point,
362         result: MeasureResult,
363         purpose: str,
364     ) -> ObservationEvent:
365         event = ObservationEvent(
366             event_id=self._next_state_version(),
367             purpose=purpose,
368             point=point,
369             status=result.status,
370             angle_deg=result.angle_deg,
371             virtual_time_s=result.virtual_time_s,
372         )
373         track.observations.append(event)
374         anchor = (round(point[0], 7), round(point[1], 7))
375         track.repeated_location_anchors.setdefault(anchor, event.event_id)
376         track.state_version = event.event_id
377         return event
378
379     def _record_clear(
380         self,
381         track: ChannelTrack,
382         point: Point,
383         result: ClearResult,
384         purpose: str,
385     ) -> ClearEvent:
386         event = ClearEvent(
387             event_id=self._next_state_version(),
388             purpose=purpose,
389             point=point,
390             status=result.status,
391             virtual_time_s=result.virtual_time_s,
392         )
393         track.clear_attempts.append(event)
394         track.state_version = event.event_id
395         return event
396
397     def _transition(
398         self,
399         track: ChannelTrack,
400         new_status: ChannelStatus,
401         reason: str,
402     ) -> None:
403         if track.status is new_status:
404             return
405         old_status = track.status
406         event_id = self._next_state_version()
407         track.status = new_status
408         track.state_version = event_id
409         track.lifecycle_events.append(
410             LifecycleEvent(event_id, old_status, new_status, reason)
411         )

```

```

412
413     def _measure_action(
414         self,
415         env: Environment,
416         track: ChannelTrack,
417         point: Point,
418         purpose: str,
419     ) -> MeasureResult:
420         result = self._call_with_pending_recovery(
421             env, env.measure, _position(point), track.channel
422         )
423         if not isinstance(result, MeasureResult):
424             raise EnvironmentStateError("测量请求返回了错误的结果类型")
425         self._record_observation(track, point, result, purpose)
426         return result
427
428     def _clear_action(
429         self,
430         env: Environment,
431         track: ChannelTrack,
432         point: Point,
433         purpose: str,
434     ) -> ClearResult:
435         result = self._call_with_pending_recovery(
436             env, env.clear, _position(point), track.channel
437         )
438         if not isinstance(result, ClearResult):
439             raise EnvironmentStateError("清除请求返回了错误的结果类型")
440         self._record_clear(track, point, result, purpose)
441         return result
442
443     def _exit_action(self, env: Environment) -> None:
444         self._call_with_pending_recovery(env, env.exit)
445
446     def run(self, env: Environment) -> StrategyResult:
447         try:
448             self._call_with_pending_recovery(env, env.enter)
449             self._rolling_search_and_clear(env)
450             certificate = self._completion_certificate(env)
451             self._last_completion_certificate = certificate
452             if certificate.valid and env.state.is_active:
453                 self._exit_action(env)
454             elif env.state.is_active and not self._has_pending_request(env):
455                 self._exit_action(env)
456             note = (
457                 f"visited_search_points={self._search_points_visited};"
458                 f"fallback_clear_attempts={self._fallback_clear_attempts};"
459                 f"guaranteed_clear_attempts={self._guaranteed_clear_attempts};"
460                 f"polygon_rebuilds={self._polygon_rebuild_count};"
461                 f"polygon_rebuild_failures={self._polygon_rebuild_failure_count};"
462                 f"state_version={self._state_version};"
463                 f"certificate={certificate.valid};"
464                 f"certificate_reason={certificate.global_reason}"

```

```

465         )
466         return StrategyResult(
467             strategy_name=type(self).__name__,
468             completed=certificate.valid,
469             note=note,
470         )
471     except EnvironmentStateError as exc:
472         pending = self._has_pending_request(env)
473         if env.state.is_active and not pending:
474             try:
475                 self._exit_action(env)
476             except EnvironmentStateError:
477                 pass
478         certificate = self._completion_certificate(env)
479         self._last_completion_certificate = certificate
480         return StrategyResult(
481             strategy_name=type(self).__name__,
482             completed=False,
483             note=(
484                 "environment ended before the completion certificate: "
485                 f"{type(exc).__name__}: {exc};"
486                 f"pending_request={pending};"
487                 f"state_version={self._state_version}"
488             ),
489         )
490
491     def _confirmed_count(self) -> int:
492         return sum(
493             track.status in (ChannelStatus.ALIVE, ChannelStatus.CLEARED)
494             for track in self._tracks.values()
495         )
496
497     def _required_absence_cover_points(self) -> Sequence[Point]:
498         return SEARCH_ROUTE
499
500     def _absence_evidence_valid(self, track: ChannelTrack) -> bool:
501         if track.absence_reason is AbsenceReason.GLOBAL_COUNT_UPPER_BOUND:
502             confirmed = set(track.absence_confirmed_channels)
503             return len(confirmed) >= 16 and all(
504                 channel in self._tracks
505                 and any(
506                     event.status in (MeasureStatus.DIRECTION, MeasureStatus.NEAR)
507                     for event in self._tracks[channel].observations
508                 )
509                 for channel in confirmed
510             )
511         if track.absence_reason is AbsenceReason.COVER_COMPLETE_NO_SIGNAL:
512             events = {
513                 event.event_id: event
514                 for event in track.observations
515                 if event.purpose == "search" and event.status is MeasureStatus.NO_SIGNAL
516             }
517             evidence = [

```

```

518         events[event_id]
519         for event_id in track.absence_evidence_event_ids
520             if event_id in events
521     ]
522     if len(evidence) != len(track.absence_evidence_event_ids):
523         return False
524     observed_points = {
525         (round(event.point[0], 7), round(event.point[1], 7))
526         for event in evidence
527     }
528     required_points = {
529         (round(point[0], 7), round(point[1], 7))
530         for point in self._required_absence_cover_points()
531     }
532     return bool(evidence) and required_points.issubset(observed_points)
533     return False
534
535 def _completion_certificate(self, env: Environment | None = None) ->
CompletionCertificate:
536     cleared = tuple(
537         channel
538         for channel, track in self._tracks.items()
539         if track.status is ChannelStatus.CLEARED
540     )
541     absent = tuple(
542         channel
543         for channel, track in self._tracks.items()
544         if track.status is ChannelStatus.ABSENT_CERTIFIED
545     )
546     history_consistent = all(
547         (
548             track.status is not ChannelStatus.CLEARED
549             or any(
550                 event.status is ClearStatus.SUCCESS
551                 for event in track.clear_attempts
552             )
553         )
554         and (
555             track.status is not ChannelStatus.ABSENT_CERTIFIED
556             or self._absence_evidence_valid(track)
557         )
558         for track in self._tracks.values()
559     )
560     pending_empty = env is None or not self._has_pending_request(env)
561     all_resolved = all(
562         track.status in (ChannelStatus.CLEARED, ChannelStatus.ABSENT_CERTIFIED)
563         for track in self._tracks.values()
564     )
565     if len(cleared) >= 16:
566         global_reason = "sixteen_cleared_by_count_bound"
567     elif all_resolved:
568         global_reason = "all_channels_resolved"
569     else:

```

```

570         global_reason = None
571         valid = bool(global_reason) and history_consistent and pending_empty
572         per_channel_reasons = tuple(
573             (
574                 channel,
575                 "clear_success"
576                 if track.status is ChannelStatus.CLEARED
577                 else track.absence_reason.value
578                 if track.absence_reason is not None
579                 else track.status.value,
580             )
581             for channel, track in self._tracks.items()
582         )
583         per_channel_evidence = tuple(
584             (
585                 channel,
586                 tuple(
587                     event.event_id
588                     for event in track.clear_attempts
589                     if event.status is ClearStatus.SUCCESS
590                 )
591                 if track.status is ChannelStatus.CLEARED
592                 else track.absence_evidence_event_ids,
593                 tuple(
594                     event.point
595                     for event in track.observations
596                     if event.event_id in track.absence_evidence_event_ids
597                 ),
598                 track.absence_confirmed_channels,
599             )
600             for channel, track in self._tracks.items()
601             if track.status in (ChannelStatus.CLEARED, ChannelStatus.ABSENT_CERTIFIED)
602         )
603         return CompletionCertificate(
604             valid=valid,
605             state_version=self._state_version,
606             built_from_version=self._state_version,
607             history_consistent=history_consistent,
608             pending_request_empty=pending_empty,
609             cleared_channels=cleared,
610             absent_channels=absent,
611             per_channel_reasons=per_channel_reasons,
612             per_channel_evidence=per_channel_evidence,
613             global_reason=global_reason,
614         )
615
616     def _rolling_search_and_clear(self, env: Environment) -> None:
617         """贪心安排强制覆盖点和已发现干扰源。"""
618         unvisited = list(SEARCH_ROUTE)
619         while unvisited or any(
620             track.status is ChannelStatus.ALIVE for track in self._tracks.values()
621         ):
622             if self._confirmed_count() >= 16:

```

```

623         unvisited.clear()
624         self._mark_unknown_absent()
625
626         current = _point(env.current_position)
627         tasks: list[tuple[float, int, str, int]] = []
628         for index, search_point in enumerate(unvisited):
629             tasks.append((_distance(current, search_point), 1, "search", index))
630         for channel, track in self._tracks.items():
631             if track.status is ChannelStatus.ALIVE:
632                 center = minimum_enclosing_circle(track.polygon).center
633                 tasks.append((_distance(current, center), 0, "source", channel))
634         if not tasks:
635             break
636
637         _, _, task_type, identifier = min(tasks)
638         if task_type == "source":
639             self._localize_and_clear(env, self._tracks[identifier])
640             continue
641
642         search_point = unvisited.pop(identifier)
643         unknown_channels = [
644             channel
645             for channel, track in self._tracks.items()
646             if track.status is ChannelStatus.UNKNOWN
647         ]
648         for channel in _channel_order(unknown_channels, env.current_channel):
649             track = self._tracks[channel]
650             result = self._measure_action(env, track, search_point, "search")
651             self._handle_measurement(
652                 env,
653                 track,
654                 search_point,
655                 result,
656             )
657             if self._confirmed_count() >= 16:
658                 break
659             self._search_points_visited += 1
660             if not unvisited or self._confirmed_count() >= 16:
661                 self._mark_unknown_absent()
662
663     def _set_absent_certified(
664         self,
665         track: ChannelTrack,
666         reason: AbsenceReason,
667         *,
668         evidence_event_ids: Sequence[int] = (),
669         confirmed_channels: Sequence[int] = (),
670     ) -> None:
671         track.absence_reason = reason
672         track.absence_evidence_event_ids = tuple(evidence_event_ids)
673         track.absence_confirmed_channels = tuple(confirmed_channels)
674         self._transition(track, ChannelStatus.ABSENT_CERTIFIED, reason.value)
675

```

```

676 def _mark_unknown_absent(self) -> None:
677     confirmed_channels = tuple(
678         channel
679         for channel, track in self._tracks.items()
680         if track.status in (ChannelStatus.ALIVE, ChannelStatus.CLEARED)
681     )
682     count_bound = len(confirmed_channels) >= 16
683     required_points = {
684         (round(point[0], 7), round(point[1], 7)) for point in SEARCH_ROUTE
685     }
686     for track in self._tracks.values():
687         if track.status is not ChannelStatus.UNKNOWN:
688             continue
689         if count_bound:
690             self._set_absent_certified(
691                 track,
692                 AbsenceReason.GLOBAL_COUNT_UPPER_BOUND,
693                 confirmed_channels=confirmed_channels,
694             )
695             continue
696         search_events = [
697             event
698             for event in track.observations
699             if event.purpose == "search" and event.status is MeasureStatus.NO_SIGNAL
700         ]
701         observed_points = {
702             (round(event.point[0], 7), round(event.point[1], 7))
703             for event in search_events
704         }
705         if required_points.issubset(observed_points):
706             self._set_absent_certified(
707                 track,
708                 AbsenceReason.COVER_COMPLETE_NO_SIGNAL,
709                 evidence_event_ids=[event.event_id for event in search_events],
710             )
711
712 def _discovery_phase(self, env: Environment) -> None:
713     for search_point in SEARCH_ROUTE:
714         if self._confirmed_count() >= 16:
715             break
716         unknown_channels = [
717             channel
718             for channel, track in self._tracks.items()
719             if track.status is ChannelStatus.UNKNOWN
720         ]
721         if not unknown_channels:
722             break
723         for channel in _channel_order(unknown_channels, env.current_channel):
724             track = self._tracks[channel]
725             result = self._measure_action(env, track, search_point, "search")
726             self._handle_measurement(env, track, search_point, result)
727             if self._confirmed_count() >= 16:
728                 break

```

```

729         self._search_points_visited += 1
730
731         if self._confirmed_count() >= 16 or self._search_points_visited == len(SEARCH_ROUTE
732     ):
733
734         self._mark_unknown_absent()
735
736     def _handle_measurement(
737         self,
738         env: Environment,
739         track: ChannelTrack,
740         point: Point,
741         result: MeasureResult,
742     ) -> None:
743         status = result.status
744         if status is MeasureStatus.NO_SIGNAL:
745             return
746         if status is MeasureStatus.NEAR:
747             self._transition(track, ChannelStatus.ALIVE, "near_observation")
748             clear_result = self._clear_action(env, track, point, "near_immediate")
749             if clear_result.status is ClearStatus.SUCCESS:
750                 self._transition(track, ChannelStatus.CLEARED, "near_clear_success")
751             else:
752                 track.region_health = RegionHealth.INVALID
753                 raise EnvironmentStateError(
754                     "near 观测后在同一点清除失败；官方约定矛盾"
755                 )
756             return
757
758         assert status is MeasureStatus.DIRECTION
759         assert result.angle_deg is not None
760         self._transition(track, ChannelStatus.ALIVE, "direction_observation")
761         if not track.polygon:
762             first = FirstObservation(point, result.angle_deg)
763             try:
764                 polygon = build_position_outer_polygon(
765                     first, self._q2_config, self.options.q2_circle_sides
766                 )
767             except ValueError:
768                 polygon = _fallback_initial_polygon(
769                     point, result.angle_deg, self.options.bearing_error_deg
770                 )
771             if len(polygon) < 3:
772                 polygon = _fallback_initial_polygon(
773                     point, result.angle_deg, self.options.bearing_error_deg
774                 )
775             track.polygon = polygon
776             track.region_health = RegionHealth.HEALTHY_CURRENT
777             track.region_built_from_version = track.state_version
778             track.first_point = point
779             track.first_bearing_deg = result.angle_deg
780         else:
781             old_polygon = list(track.polygon)
782             clipped = posterior_polygon(

```

```

781         track.polygon,
782         point,
783         result.angle_deg,
784         self.options.bearing_error_deg,
785     )
786     if len(clipped) >= 3:
787         track.polygon = clipped
788         track.region_health = RegionHealth.HEALTHY_CURRENT
789         track.region_built_from_version = track.state_version
790     else:
791         self._polygon_rebuild_count += 1
792         rebuilt = self._rebuild_polygon_from_evidence(track)
793         if rebuilt is not None:
794             track.polygon = rebuilt
795             track.region_health = RegionHealth.HEALTHY_CURRENT
796             track.region_built_from_version = track.state_version
797         elif old_polygon:
798             self._polygon_rebuild_failure_count += 1
799             track.polygon = old_polygon
800             track.region_health = RegionHealth.SAFE_STALE
801         else:
802             self._polygon_rebuild_failure_count += 1
803             track.region_health = RegionHealth.INVALID
804     track.last_point = point
805     track.last_bearing_deg = result.angle_deg
806
807     def _rebuild_polygon_from_evidence(self, track: ChannelTrack) -> list[Point] | None:
808         direction_events = [
809             event
810             for event in track.observations
811             if event.status is MeasureStatus.DIRECTION and event.angle_deg is not None
812         ]
813         if not direction_events:
814             return None
815
816         unique: list[ObservationEvent] = []
817         by_anchor: dict[Point, ObservationEvent] = {}
818         for event in direction_events:
819             anchor = (round(event.point[0], 7), round(event.point[1], 7))
820             prior = by_anchor.get(anchor)
821             if prior is not None:
822                 difference = abs(
823                     (float(event.angle_deg) - float(prior.angle_deg) + 180.0)
824                     % 360.0
825                     - 180.0
826                 )
827                 if difference > 1e-7:
828                     return None
829                 continue
830             by_anchor[anchor] = event
831             unique.append(event)
832
833         first_event = unique[0]

```

```

834     first = FirstObservation(first_event.point, float(first_event.angle_deg))
835     try:
836         polygon = build_position_outer_polygon(
837             first, self._q2_config, self.options.q2_circle_sides
838         )
839     except ValueError:
840         polygon = _fallback_initial_polygon(
841             first_event.point,
842             float(first_event.angle_deg),
843             self.options.bearing_error_deg,
844         )
845     if len(polygon) < 3:
846         polygon = _fallback_initial_polygon(
847             first_event.point,
848             float(first_event.angle_deg),
849             self.options.bearing_error_deg,
850         )
851     for event in unique[1:]:
852         polygon = posterior_polygon(
853             polygon,
854             event.point,
855             float(event.angle_deg),
856             self.options.bearing_error_deg,
857         )
858         if len(polygon) < 3:
859             return None
860     return polygon
861
862 def _localization_phase(self, env: Environment) -> None:
863     while True:
864         alive = [
865             track
866             for track in self._tracks.values()
867             if track.status is ChannelStatus.ALIVE
868         ]
869         if not alive:
870             return
871         current = _point(env.current_position)
872         track = min(
873             alive,
874             key=lambda item: _distance(
875                 current, minimum_enclosing_circle(item.polygon).center
876             ),
877         )
878         self._localize_and_clear(env, track)
879
880 def _localize_and_clear(self, env: Environment, track: ChannelTrack) -> None:
881     if track.region_health is RegionHealth.INVALID or not track.polygon:
882         raise EnvironmentStateError(
883             f"信道 {track.channel} 没有经过认证的位置外包"
884         )
885     for _ in range(self.options.max_active_measurements):
886         circle = minimum_enclosing_circle(track.polygon)

```

```

887         if circle.radius <= self.options.guaranteed_clear_radius_m:
888             if self._try_guaranteed_clear(env, track, circle.center):
889                 return
890             break
891
892         candidate = self._choose_q2_candidate(env, track)
893         if candidate is None:
894             break
895         result = self._measure_action(
896             env, track, candidate, "active_localization"
897         )
898         track.active_measurements += 1
899         if result.status is MeasureStatus.NO_SIGNAL:
900             track.no_signal_during_localization += 1
901             break
902         self._handle_measurement(env, track, candidate, result)
903         if track.status is ChannelStatus.CLEARED:
904             return
905
906         if track.status is not ChannelStatus.CLEARED:
907             circle = minimum_enclosing_circle(track.polygon)
908             if circle.radius <= self.options.clear_radius_m:
909                 if self._try_guaranteed_clear(env, track, circle.center):
910                     return
911             self._clear_by_grid(env, track)
912
913     def _try_guaranteed_clear(
914         self, env: Environment, track: ChannelTrack, point: Point
915     ) -> bool:
916         self._guaranteed_clear_attempts += 1
917         result = self._clear_action(env, track, point, "guaranteed_center")
918         if result.status is ClearStatus.SUCCESS:
919             self._transition(track, ChannelStatus.CLEARED, "guaranteed_clear_success")
920             return True
921         return False
922
923     def _choose_q2_candidate(
924         self, env: Environment, track: ChannelTrack
925     ) -> Point | None:
926         if track.region_health is not RegionHealth.HEALTHY_CURRENT:
927             return None
928         if track.last_bearing_deg is None:
929             return None
930         current = _point(env.current_position)
931         reference = FirstObservation(current, track.last_bearing_deg)
932         candidates, circle = generate_sparse_candidates(
933             reference,
934             self._q2_config,
935             track.polygon,
936         )
937         if not candidates:
938             return None
939

```

```

940     sources = _representative_polygon_points(
941         track.polygon, self.options.q2_source_samples
942     )
943     errors = _error_grid(
944         self.options.bearing_error_deg, self.options.q2_error_nodes
945     )
946     scores = [
947         evaluate_candidate(
948             candidate,
949             "q3_sparse",
950             reference,
951             self._q2_config,
952             track.polygon,
953             sources,
954             errors,
955         )
956         for candidate in candidates
957     ]
958     scores.sort(key=lambda score: _score_key(score, current, circle.center))
959
960     slack = max(0.0, self._q2_config.receive_min - circle.radius)
961     step = min(150.0, max(12.0, 0.4 * slack))
962     directions = [
963         (math.cos(index * math.pi / 4.0), math.sin(index * math.pi / 4.0))
964         for index in range(8)
965     ]
966     best_scores: list[CandidateScore] = []
967     for seed in scores[: self.options.q2_seed_count]:
968         best = seed
969         local_step = step
970         for _ in range(self.options.q2_local_rounds):
971             for dx, dy in directions:
972                 point = (
973                     best.point[0] + local_step * dx,
974                     best.point[1] + local_step * dy,
975                 )
976                 if (
977                     conservative_safety_margin(
978                         point, track.polygon, self._q2_config.receive_min
979                     )
980                     < -1e-7
981                 ):
982                     continue
983                 score = evaluate_candidate(
984                     point,
985                     "q3_local",
986                     reference,
987                     self._q2_config,
988                     track.polygon,
989                     sources,
990                     errors,
991                 )
992                 if _score_key(score, current, circle.center) < _score_key(best, current

```

```

, circle.center):
993         best = score
994         local_step *= 0.5
995         best_scores.append(best)
996
997     best_scores.extend(scores[:3])
998     best = min(best_scores, key=lambda score: _score_key(score, current, circle.center)
999 )
1000
1001     return best.point
1002
1003 def _clear_by_grid(self, env: Environment, track: ChannelTrack) -> None:
1004     if not track.polygon:
1005         return
1006     if track.region_health is RegionHealth.INVALID:
1007         raise EnvironmentStateError(
1008             f"信道 {track.channel} 不能使用未经认证的清除区域"
1009         )
1010     origin = track.first_point or track.polygon[0]
1011     bearing = math.radians(track.first_bearing_deg or 0.0)
1012     ux, uy = math.cos(bearing), math.sin(bearing)
1013     vx, vy = -uy, ux
1014
1015     local = [
1016         (
1017             (point[0] - origin[0]) * ux + (point[1] - origin[1]) * uy,
1018             (point[0] - origin[0]) * vx + (point[1] - origin[1]) * vy,
1019         )
1020         for point in track.polygon
1021     ]
1022     min_u = min(point[0] for point in local)
1023     max_u = max(point[0] for point in local)
1024     min_v = min(point[1] for point in local)
1025     max_v = max(point[1] for point in local)
1026     step = self.options.clear_cell_side_m
1027     columns = max(1, math.ceil((max_u - min_u) / step))
1028     rows = max(1, math.ceil((max_v - min_v) / step))
1029
1030     for row in range(rows):
1031         column_order = range(columns) if row % 2 == 0 else range(columns - 1, -1, -1)
1032         for column in column_order:
1033             u = min_u + (column + 0.5) * step
1034             v = min_v + (row + 0.5) * step
1035             point = (
1036                 origin[0] + u * ux + v * vx,
1037                 origin[1] + u * uy + v * vy,
1038             )
1039             self._fallback_clear_attempts += 1
1040             result = self._clear_action(env, track, point, "grid_fallback")
1041             if result.status is ClearStatus.SUCCESS:
1042                 self._transition(track, ChannelStatus.CLEARED, "grid_clear_success")
1043                 return

```

```

1044 __all__ = [
1045     "AbsenceReason",
1046     "ChannelStatus",
1047     "ChannelTrack",
1048     "ClearEvent",
1049     "CompletionCertificate",
1050     "GuaranteedRollingQ3Strategy",
1051     "LifecycleEvent",
1052     "ObservationEvent",
1053     "Q3StrategyOptions",
1054     "RegionHealth",
1055     "SEARCH_ROUTE",
1056     "_fallback_initial_polygon",
1057     "search_cover_bound_m",
1058 ]

```

Listing 12 run_q3_official.py

```

1 from __future__ import annotations
2
3 import argparse
4 from datetime import datetime
5 import json
6 from pathlib import Path
7 import sys
8 from time import perf_counter
9
10 _PROJECT_ROOT = Path(__file__).resolve().parents[2]
11 _SHARED_CODE = _PROJECT_ROOT / "code"
12 if str(_SHARED_CODE) not in sys.path:
13     sys.path.insert(0, str(_SHARED_CODE))
14
15 from radio_sim.environment.base import EnvironmentStateError # noqa: E402
16 from radio_sim.environment.official import OfficialEnvironment # noqa: E402
17 from q3_strategy import GuaranteedRollingQ3Strategy, Q3StrategyOptions # noqa: E402
18
19
20 def available_path(path: Path) -> Path:
21     if not path.exists():
22         return path
23     stamp = datetime.now().strftime("%Y%m%d_%H%M%S_%f")
24     return path.with_name(f"{path.stem}_{stamp}{path.suffix}")
25
26
27 def parse_args() -> argparse.Namespace:
28     parser = argparse.ArgumentParser(description=__doc__)
29     parser.add_argument("--robot-id", required=True, help="当前登录模拟器的参赛队号")
30     parser.add_argument("--base-url", default="http://127.0.0.1:2026")
31     parser.add_argument("--timeout", type=float, default=5.0)
32     parser.add_argument("--max-retries", type=int, default=2)
33     parser.add_argument("--retry-backoff", type=float, default=0.2)
34     parser.add_argument("--exit-reserve", type=float, default=15.0)
35     parser.add_argument("--safety-buffer", type=float, default=5.0)

```

```

36 parser.add_argument("--max-active-measurements", type=int, default=4)
37 parser.add_argument("--log", type=Path, help="逐请求 JSONL 日志路径")
38 parser.add_argument("--output", type=Path, help="运行摘要 JSON 路径")
39 return parser.parse_args()
40
41
42 def main() -> int:
43     args = parse_args()
44     stamp = datetime.now().strftime("%Y%m%d_%H%M%S_%f")
45     results_dir = Path(__file__).resolve().parents[1] / "results"
46     log_path = available_path(
47         (args.log or results_dir / f"q3_official_{stamp}.jsonl").resolve()
48     )
49     output_path = available_path(
50         (args.output or results_dir / f"q3_official_{stamp}.json").resolve()
51     )
52
53     env = OfficialEnvironment(
54         robot_id=args.robot_id,
55         base_url=args.base_url,
56         timeout_s=args.timeout,
57         max_retries=args.max_retries,
58         retry_backoff_s=args.retry_backoff,
59         exit_reserve_s=args.exit_reserve,
60         log_path=log_path,
61     )
62     strategy = GuaranteedRollingQ3Strategy(
63         Q3StrategyOptions(
64             safety_buffer_m=args.safety_buffer,
65             max_active_measurements=args.max_active_measurements,
66         )
67     )
68
69     started = perf_counter()
70     result = strategy.run(env)
71     cleanup_error: str | None = None
72     if env.state.is_active:
73         if env.has_pending_request:
74             cleanup_error = (
75                 "unresolved request retained; no new /exit was sent: "
76                 f"{env.pending_request}"
77             )
78         else:
79             try:
80                 env.exit()
81             except EnvironmentStateError as exc:
82                 cleanup_error = f"{type(exc).__name__}: {exc}"
83     wall_time_s = perf_counter() - started
84
85     state = env.state
86     summary = {
87         "result_status": "COMPLETE" if result.completed else "INCOMPLETE",
88         "strategy_name": result.strategy_name,

```

```

89     "strategy_completed": result.completed,
90     "strategy_note": result.note,
91     "successful_clear_count": env.successful_clear_count,
92     "measure_count": env.measure_count,
93     "clear_count": env.clear_count,
94     "unique_request_count": env.request_count,
95     "http_attempt_count": env.http_attempt_count,
96     "virtual_time_s": state.virtual_time_s,
97     "wall_time_s": wall_time_s,
98     "entered_remaining_real_duration_s": env.entered_remaining_real_duration_s,
99     "remaining_real_duration_s": env.remaining_real_duration_s,
100    "final_position": {"x": state.current_position.x, "y": state.current_position.y},
101    "final_measurement_channel": state.current_channel,
102    "environment_finished": state.is_finished,
103    "cleanup_error": cleanup_error,
104    "pending_request": env.pending_request,
105    "completion_certificate": (
106        strategy.completion_certificate.to_record()
107        if strategy.completion_certificate is not None
108        else None
109    ),
110    "request_log": str(log_path),
111    "output": str(output_path),
112 }
113 output_path.parent.mkdir(parents=True, exist_ok=True)
114 output_path.write_text(
115     json.dumps(summary, ensure_ascii=False, indent=2) + "\n",
116     encoding="utf-8",
117 )
118 print(json.dumps(summary, ensure_ascii=False, indent=2))
119 return 0 if result.completed and cleanup_error is None else 2
120
121
122 if __name__ == "__main__":
123     raise SystemExit(main())

```

Listing 13 run_q3_batch.py

```

1 from __future__ import annotations
2
3 import argparse
4 import csv
5 from datetime import datetime
6 import json
7 from pathlib import Path
8 import sys
9
10 _PROJECT_ROOT = Path(__file__).resolve().parents[2]
11 _SHARED_CODE = _PROJECT_ROOT / "code"
12 if str(_SHARED_CODE) not in sys.path:
13     sys.path.insert(0, str(_SHARED_CODE))
14
15 from radio_sim.evaluation import run_batch # noqa: E402

```

```

16 from q3_strategy import GuaranteedRollingQ3Strategy, Q3StrategyOptions # noqa: E402
17
18
19 def default_output_path(start_seed: int, count: int) -> Path:
20     root = Path(__file__).resolve().parents[1] / "results"
21     stamp = datetime.now().strftime("%Y%m%d_%H%M%S_%f")
22     return root / f"q3_batch_seed{start_seed}_n{count}_{stamp}.csv"
23
24
25 def available_path(path: Path) -> Path:
26     if not path.exists():
27         return path
28     stamp = datetime.now().strftime("%Y%m%d_%H%M%S_%f")
29     return path.with_name(f"{path.stem}_{stamp}{path.suffix}")
30
31
32 def parse_args() -> argparse.Namespace:
33     parser = argparse.ArgumentParser(description=__doc__)
34     parser.add_argument("--start-seed", type=int, default=0)
35     parser.add_argument("--count", type=int, default=10)
36     parser.add_argument("--safety-buffer", type=float, default=5.0)
37     parser.add_argument("--max-active-measurements", type=int, default=4)
38     parser.add_argument("--output", type=Path)
39     parser.add_argument("--no-write", action="store_true")
40     return parser.parse_args()
41
42
43 def main() -> int:
44     args = parse_args()
45     if args.count <= 0:
46         raise SystemExit("--count 必须为正数")
47
48     strategy = GuaranteedRollingQ3Strategy(
49         Q3StrategyOptions(
50             safety_buffer_m=args.safety_buffer,
51             max_active_measurements=args.max_active_measurements,
52         )
53     )
54     results = run_batch(
55         strategy=strategy,
56         seeds=range(args.start_seed, args.start_seed + args.count),
57         problem="Q3",
58     )
59     records = [result.to_record() for result in results]
60
61     output: Path | None = None
62     if not args.no_write:
63         output = available_path(
64             (args.output or default_output_path(args.start_seed, args.count)).resolve()
65         )
66         output.parent.mkdir(parents=True, exist_ok=True)
67         with output.open("w", newline="", encoding="utf-8-sig") as handle:
68             writer = csv.DictWriter(handle, fieldnames=list(records[0]))

```

```

69         writer.writeheader()
70         writer.writerows(records)
71
72     all_cleared_count = sum(result.all_cleared for result in results)
73     summary = {
74         "problem": "Q3",
75         "run_count": len(results),
76         "seed_start": args.start_seed,
77         "seed_end": args.start_seed + args.count - 1,
78         "all_cleared_count": all_cleared_count,
79         "all_cleared_rate": all_cleared_count / len(results),
80         "strategy_completed_rate": sum(result.strategy_completed for result in results) /
len(results),
81         "mean_virtual_time_s": sum(result.virtual_time_s for result in results) / len(
results),
82         "mean_average_localize_clear_time_s": sum(
83             result.average_localize_clear_time_s or 0.0 for result in results
84         ) / len(results),
85         "max_virtual_time_s": max(result.virtual_time_s for result in results),
86         "mean_wall_time_s": sum(result.wall_time_s for result in results) / len(results),
87         "mean_failed_clear_count": sum(result.failed_clear_count for result in results) /
len(results),
88         "output": str(output) if output is not None else None,
89     }
90     print(json.dumps(summary, ensure_ascii=False, indent=2))
91     return 0
92
93
94 if __name__ == "__main__":
95     raise SystemExit(main())

```

Listing 14 run_q3.py

```

1 from __future__ import annotations
2
3 import argparse
4 from datetime import datetime
5 import json
6 from pathlib import Path
7 import sys
8
9 _PROJECT_ROOT = Path(__file__).resolve().parents[2]
10 _SHARED_CODE = _PROJECT_ROOT / "code"
11 if str(_SHARED_CODE) not in sys.path:
12     sys.path.insert(0, str(_SHARED_CODE))
13
14 from radio_sim.evaluation import run_single # noqa: E402
15 from q3_strategy import GuaranteedRollingQ3Strategy, Q3StrategyOptions # noqa: E402
16
17
18 def default_output_path(seed: int) -> Path:
19     root = Path(__file__).resolve().parents[1] / "results"
20     stamp = datetime.now().strftime("%Y%m%d_%H%M%S_%f")

```

```

21     return root / f"q3_single_seed{seed}_{stamp}.json"
22
23
24 def available_path(path: Path) -> Path:
25     if not path.exists():
26         return path
27     stamp = datetime.now().strftime("%Y%m%d_%H%M%S%f")
28     return path.with_name(f"{path.stem}_{stamp}{path.suffix}")
29
30
31 def parse_args() -> argparse.Namespace:
32     parser = argparse.ArgumentParser(description=__doc__)
33     parser.add_argument("--seed", type=int, default=12345)
34     parser.add_argument("--safety-buffer", type=float, default=5.0)
35     parser.add_argument("--max-active-measurements", type=int, default=4)
36     parser.add_argument("--output", type=Path)
37     parser.add_argument("--no-write", action="store_true")
38     return parser.parse_args()
39
40
41 def main() -> int:
42     args = parse_args()
43     options = Q3StrategyOptions(
44         safety_buffer_m=args.safety_buffer,
45         max_active_measurements=args.max_active_measurements,
46     )
47     metrics = run_single(
48         strategy=GuaranteedRollingQ3Strategy(options),
49         seed=args.seed,
50         problem="Q3",
51     )
52     record = metrics.to_record()
53
54     output: Path | None = None
55     if not args.no_write:
56         output = available_path((args.output or default_output_path(args.seed)).resolve())
57         output.parent.mkdir(parents=True, exist_ok=True)
58         output.write_text(
59             json.dumps(record, ensure_ascii=False, indent=2) + "\n",
60             encoding="utf-8",
61         )
62
63     payload = dict(record)
64     payload["output"] = str(output) if output is not None else None
65     print(json.dumps(payload, ensure_ascii=False, indent=2))
66     return 0
67
68
69 if __name__ == "__main__":
70     raise SystemExit(main())

```

D 问题四程序代码

Listing 15 q4_strategy.py

```
1 from __future__ import annotations
2
3 from dataclasses import dataclass
4 import math
5 from pathlib import Path
6 import sys
7
8 _PROJECT_ROOT = Path(__file__).resolve().parents[2]
9 _SHARED_CODE = _PROJECT_ROOT / "code"
10 _Q2_CODE = _PROJECT_ROOT / "q2" / "code"
11 _Q3_CODE = _PROJECT_ROOT / "q3" / "code"
12 for _code_dir in (_SHARED_CODE, _Q2_CODE, _Q3_CODE):
13     if str(_code_dir) not in sys.path:
14         sys.path.insert(0, str(_code_dir))
15
16 from q2_fast import ( # noqa: E402
17     CandidateScore,
18     conservative_safety_margin,
19     evaluate_candidate,
20     generate_sparse_candidates,
21     minimum_enclosing_circle,
22 )
23 from q2_state import FirstObservation # noqa: E402
24 from q4_clear_terminal import ( # noqa: E402
25     CertifiedStripPlan,
26     build_certified_strip_plan,
27     estimate_full_clear_time_s,
28 )
29 from q3_strategy import ( # noqa: E402
30     AbsenceReason,
31     ChannelTrack,
32     ChannelStatus,
33     GuaranteedRollingQ3Strategy,
34     Q3StrategyOptions,
35     _channel_order,
36     _distance,
37     _error_grid,
38     _point,
39     _position,
40     _representative_polygon_points,
41     _score_key,
42 )
43 from radio_sim.domain.models import ( # noqa: E402
44     ClearStatus,
45     MeasureResult,
46     MeasureStatus,
47     StrategyResult,
48 )
49 from radio_sim.environment.base import Environment # noqa: E402
```

```

50
51 Point = tuple[float, float]
52
53 DIRECTIONAL_INNER_RADIUS_M = 950.0
54 DIRECTIONAL_OUTER_RADIUS_M = 1870.0
55 DIRECTIONAL_RING_COUNT = 12
56
57
58 @dataclass(frozen=True)
59 class Q4StrategyOptions(Q3StrategyOptions):
60
61     max_active_measurements: int = 2
62     mirrored_reacquisition: bool = True
63     prioritized_clear_grid: bool = True
64     task_insertion_scheduler: bool = False
65     joint_direction_ranking: bool = False
66     adaptive_tail_probe: bool = True
67     adaptive_tail_min_visibility: float = 0.30
68     direction_aware_grid_ordering: bool = False
69     direction_heading_samples: int = 36
70     analytic_clear_terminal: bool = False
71     tail_terminal_linkage: bool = False
72
73     def validate(self) -> None:
74         super().validate()
75         if self.direction_heading_samples < 8:
76             raise ValueError("direction_heading_samples 必须至少为 8")
77         if not 0.0 <= self.adaptive_tail_min_visibility <= 1.0:
78             raise ValueError(
79                 "adaptive_tail_min_visibility 必须位于 [0, 1] 内"
80             )
81         if self.tail_terminal_linkage and not self.analytic_clear_terminal:
82             raise ValueError(
83                 "启用 tail_terminal_linkage 时必须同时启用 analytic_clear_terminal"
84             )
85
86
87     def _polar_point(radius_m: float, angle_deg: float) -> Point:
88         angle = math.radians(angle_deg)
89         return (radius_m * math.cos(angle), radius_m * math.sin(angle))
90
91
92     def _reflect_across_bearing_axis(
93         point: Point,
94         origin: Point,
95         bearing_deg: float,
96     ) -> Point:
97         angle = math.radians(bearing_deg)
98         ux, uy = math.cos(angle), math.sin(angle)
99         dx, dy = point[0] - origin[0], point[1] - origin[1]
100         projection = dx * ux + dy * uy
101         axial_x, axial_y = projection * ux, projection * uy
102         return (

```

```

103         origin[0] + 2.0 * axial_x - dx,
104         origin[1] + 2.0 * axial_y - dy,
105     )
106
107
108 def build_directionally_complete_search_points() -> tuple[Point, ...]:
109     inner = tuple(
110         _polar_point(DIRECTIONAL_INNER_RADIUS_M, 30.0 * index)
111         for index in range(DIRECTIONAL_RING_COUNT)
112     )
113     outer = tuple(
114         _polar_point(DIRECTIONAL_OUTER_RADIUS_M, 30.0 * index + 15.0)
115         for index in range(DIRECTIONAL_RING_COUNT)
116     )
117     return ((0.0, 0.0), *inner, *outer)
118
119
120 DIRECTIONAL_SEARCH_POINTS = build_directionally_complete_search_points()
121
122 DIRECTIONAL_SEARCH_ROUTE_INDICES: tuple[int, ...] = (
123     0, 9, 8, 7, 6, 5, 4, 3, 2, 1, 12, 11, 10,
124     22, 21, 20, 19, 18, 17, 16, 15, 14, 13, 24, 23,
125 )
126
127
128 def directional_search_route_length_m() -> float:
129     points = [
130         DIRECTIONAL_SEARCH_POINTS[index]
131         for index in DIRECTIONAL_SEARCH_ROUTE_INDICES
132     ]
133     return sum(
134         _distance(first, second)
135         for first, second in zip(((0.0, 0.0), *points[:-1]), points)
136     )
137
138
139 def _source_before_search(
140     current: Point,
141     next_search: Point,
142     source_centers: dict[int, Point],
143 ) -> int | None:
144     if not source_centers:
145         return None
146     source_channel, source_first_cost = min(
147         (
148             channel,
149             _distance(current, center) + _distance(center, next_search),
150         )
151         for channel, center in source_centers.items()
152     )
153     search_first_cost = _distance(current, next_search) + min(
154         _distance(next_search, center) for center in source_centers.values()
155     )

```

```

156     if source_first_cost + 1e-9 < search_first_cost:
157         return source_channel
158     return None
159
160
161 def _directional_visibility_fraction(
162     candidate: Point,
163     sources: list[Point],
164     visible_points: list[Point],
165     blind_points: list[Point],
166     heading_count: int,
167     near_radius_m: float = 5.0,
168 ) -> float | None:
169     feasible = 0
170     visible = 0
171     for source in sources:
172         for index in range(heading_count):
173             angle = 2.0 * math.pi * index / heading_count
174             ux, uy = math.cos(angle), math.sin(angle)
175
176             def in_emission_halfplane(point: Point) -> bool:
177                 return (
178                     ux * (point[0] - source[0])
179                     + uy * (point[1] - source[1])
180                     >= -1e-9
181                 )
182
183             if any(not in_emission_halfplane(point) for point in visible_points):
184                 continue
185             if any(in_emission_halfplane(point) for point in blind_points):
186                 continue
187             feasible += 1
188             if (
189                 _distance(candidate, source) <= near_radius_m
190                 or in_emission_halfplane(candidate)
191             ):
192                 visible += 1
193     if feasible == 0:
194         return None
195     return visible / feasible
196
197
198 def _compatible_heading_count(
199     source: Point,
200     visible_points: list[Point],
201     blind_points: list[Point],
202     heading_count: int,
203 ) -> int:
204     compatible = 0
205     for index in range(heading_count):
206         angle = 2.0 * math.pi * index / heading_count
207         ux, uy = math.cos(angle), math.sin(angle)
208

```

```

209     def visible(point: Point) -> bool:
210         return (
211             ux * (point[0] - source[0])
212             + uy * (point[1] - source[1])
213             >= -1e-9
214         )
215
216         if all(visible(point) for point in visible_points) and all(
217             not visible(point) for point in blind_points
218         ):
219             compatible += 1
220     return compatible
221
222
223 def directional_search_triangles() -> tuple[tuple[Point, Point, Point], ...]:
224     """返回有限发现证书使用的 36 个三角形。"""
225     center = DIRECTIONAL_SEARCH_POINTS[0]
226     inner = DIRECTIONAL_SEARCH_POINTS[1:13]
227     outer = DIRECTIONAL_SEARCH_POINTS[13:25]
228     triangles: list[tuple[Point, Point, Point]] = []
229     for index in range(DIRECTIONAL_RING_COUNT):
230         next_index = (index + 1) % DIRECTIONAL_RING_COUNT
231         previous_index = (index - 1) % DIRECTIONAL_RING_COUNT
232         triangles.append((center, inner[index], inner[next_index]))
233         triangles.append((inner[index], outer[previous_index], outer[index]))
234         triangles.append((inner[index], inner[next_index], outer[index]))
235     return tuple(triangles)
236
237
238 def directional_search_certificate() -> dict[str, float | int | bool]:
239     """计算 Q4 发现证明所需的几何量。"""
240     triangles = directional_search_triangles()
241     maximum_edge = max(
242         _distance(first, second)
243         for triangle in triangles
244         for first, second in (
245             (triangle[0], triangle[1]),
246             (triangle[1], triangle[2]),
247             (triangle[2], triangle[0]),
248         )
249     )
250     outer_inradius = DIRECTIONAL_OUTER_RADIUS_M * math.cos(math.pi / 12.0)
251     return {
252         "search_point_count": len(DIRECTIONAL_SEARCH_POINTS),
253         "triangle_count": len(triangles),
254         "maximum_triangle_edge_m": maximum_edge,
255         "outer_polygon_inradius_m": outer_inradius,
256         "strict_receive_margin_m": 1000.0 - maximum_edge,
257         "covers_target_disk": outer_inradius >= 1800.0,
258         "all_triangle_edges_below_receive_min": maximum_edge < 1000.0,
259     }
260
261

```

```

262 class GuaranteedDirectionalQ4Strategy(GuaranteedRollingQ3Strategy):
263     """ Q4 基线策略。 """
264
265     def __init__(self, options: Q4StrategyOptions | None = None) -> None:
266         super().__init__(options or Q4StrategyOptions())
267         self._search_completed_by_channel = {
268             channel: set() for channel in range(1, 21)
269         }
270         self._mirrored_reacquisition_attempts = 0
271         self._mirrored_reacquisition_hits = 0
272         self._source_first_insertions = 0
273         self._search_first_decisions = 0
274         self._joint_direction_rankings = 0
275         self._adaptive_tail_probe_attempts = 0
276         self._adaptive_tail_probe_hits = 0
277         self._direction_aware_grid_uses = 0
278         self._analytic_plan_generated = 0
279         self._analytic_plan_certified = 0
280         self._analytic_plan_selected = 0
281         self._analytic_plan_fallbacks = 0
282         self._analytic_estimated_saving_s = 0.0
283         self._tail_terminal_linkage_evaluations = 0
284         self._tail_terminal_analytic_choices = 0
285         self._tail_terminal_estimated_cost_reduction_s = 0.0
286         self._last_candidate_score_by_channel: dict[int, CandidateScore] = {}
287         self._visible_points_by_channel: dict[int, list[Point]] = {
288             channel: [] for channel in range(1, 21)
289         }
290         self._blind_points_by_channel: dict[int, list[Point]] = {
291             channel: [] for channel in range(1, 21)
292         }
293
294     def run(self, env: Environment) -> StrategyResult:
295         result = super().run(env)
296         note = (
297             f"{{result.note}};"
298             f"mirrored_reacquisition_attempts={{self._mirrored_reacquisition_attempts}};"
299             f"mirrored_reacquisition_hits={{self._mirrored_reacquisition_hits}};"
300             f"source_first_insertions={{self._source_first_insertions}};"
301             f"search_first_decisions={{self._search_first_decisions}};"
302             f"joint_direction_rankings={{self._joint_direction_rankings}};"
303             f"adaptive_tail_probe_attempts={{self._adaptive_tail_probe_attempts}};"
304             f"adaptive_tail_probe_hits={{self._adaptive_tail_probe_hits}};"
305             f"direction_aware_grid_uses={{self._direction_aware_grid_uses}};"
306             f"analytic_plan_generated={{self._analytic_plan_generated}};"
307             f"analytic_plan_certified={{self._analytic_plan_certified}};"
308             f"analytic_plan_selected={{self._analytic_plan_selected}};"
309             f"analytic_plan_fallbacks={{self._analytic_plan_fallbacks}};"
310             f"analytic_estimated_saving_s={{self._analytic_estimated_saving_s:.6f}};"
311             f"tail_terminal_linkage_evaluations="
312             f"{{self._tail_terminal_linkage_evaluations}};"
313             f"tail_terminal_analytic_choices="
314             f"{{self._tail_terminal_analytic_choices}};"

```

```

315         f"tail_terminal_estimated_cost_reduction_s="
316         f"{self._tail_terminal_estimated_cost_reduction_s:.6f}"
317     )
318     return StrategyResult(
319         strategy_name=result.strategy_name,
320         completed=result.completed,
321         note=note,
322     )
323
324 @property
325 def analytic_terminal_diagnostics(self) -> dict[str, bool | int | float]:
326     return {
327         "analytic_clear_terminal_enabled": (
328             self.options.analytic_clear_terminal
329         ),
330         "tail_terminal_linkage_enabled": self.options.tail_terminal_linkage,
331         "analytic_plan_generated": self._analytic_plan_generated,
332         "analytic_plan_certified": self._analytic_plan_certified,
333         "analytic_plan_selected": self._analytic_plan_selected,
334         "analytic_plan_fallbacks": self._analytic_plan_fallbacks,
335         "analytic_estimated_saving_s": self._analytic_estimated_saving_s,
336         "tail_terminal_linkage_evaluations": (
337             self._tail_terminal_linkage_evaluations
338         ),
339         "tail_terminal_analytic_choices": (
340             self._tail_terminal_analytic_choices
341         ),
342         "tail_terminal_estimated_cost_reduction_s": (
343             self._tail_terminal_estimated_cost_reduction_s
344         ),
345     }
346
347 @staticmethod
348 def _append_distinct(points: list[Point], point: Point) -> None:
349     if all(_distance(point, prior) > 1e-7 for prior in points):
350         points.append(point)
351
352 def _record_blind_probe(self, channel: int, point: Point) -> None:
353     self._append_distinct(self._blind_points_by_channel[channel], point)
354
355 def _required_absence_cover_points(self) -> Sequence[Point]:
356     return DIRECTIONAL_SEARCH_POINTS
357
358 def _handle_measurement(
359     self,
360     env: Environment,
361     track: ChannelTrack,
362     point: Point,
363     result: MeasureResult,
364 ) -> None:
365     if result.status is not MeasureStatus.NO_SIGNAL:
366         self._append_distinct(
367             self._visible_points_by_channel[track.channel],

```

```

368         point,
369     )
370     super()._handle_measurement(env, track, point, result)
371
372     def _certify_completed_unknown_channels(self) -> None:
373         required = len(DIRECTIONAL_SEARCH_POINTS)
374         for channel, track in self._tracks.items():
375             if (
376                 track.status is ChannelStatus.UNKNOWN
377                 and len(self._search_completed_by_channel[channel]) == required
378             ):
379                 search_events = [
380                     event
381                     for event in track.observations
382                     if event.purpose == "search"
383                     and event.status is MeasureStatus.NO_SIGNAL
384                 ]
385                 self._set_absent_certified(
386                     track,
387                     AbsenceReason.COVER_COMPLETE_NO_SIGNAL,
388                     evidence_event_ids=[event.event_id for event in search_events],
389                 )
390
391     def _rolling_search_and_clear(self, env: Environment) -> None:
392         """穿插有收益的干扰源处理任务。"""
393         if self.options.task_insertion_scheduler:
394             unvisited = list(DIRECTIONAL_SEARCH_ROUTE_INDICES)
395         else:
396             unvisited = list(range(len(DIRECTIONAL_SEARCH_POINTS)))
397
398         while unvisited or any(
399             track.status is ChannelStatus.ALIVE for track in self._tracks.values()
400         ):
401             if self._confirmed_count() >= 16:
402                 unvisited.clear()
403                 self._mark_unknown_absent()
404
405             current = _point(env.current_position)
406             source_centers = {
407                 channel: minimum_enclosing_circle(track.polygon).center
408                 for channel, track in self._tracks.items()
409                 if track.status is ChannelStatus.ALIVE
410             }
411             task_type: str | None = None
412             identifier: int | None = None
413
414             if self.options.task_insertion_scheduler:
415                 if unvisited:
416                     next_search_index = min(
417                         unvisited,
418                         key=lambda index: _distance(
419                             current,
420                             DIRECTIONAL_SEARCH_POINTS[index],

```

```

421         ),
422     )
423     next_search = DIRECTIONAL_SEARCH_POINTS[next_search_index]
424     source_channel = _source_before_search(
425         current,
426         next_search,
427         source_centers,
428     )
429     if source_channel is not None:
430         task_type, identifier = "source", source_channel
431         self._source_first_insertions += 1
432     else:
433         task_type, identifier = "search", next_search_index
434         if source_centers:
435             self._search_first_decisions += 1
436     elif source_centers:
437         identifier = min(
438             source_centers,
439             key=lambda channel: _distance(
440                 current, source_centers[channel]
441             ),
442         )
443     task_type = "source"
444 else:
445     tasks: list[tuple[float, int, str, int]] = []
446     for index in unvisited:
447         tasks.append(
448             (
449                 _distance(current, DIRECTIONAL_SEARCH_POINTS[index]),
450                 1,
451                 "search",
452                 index,
453             )
454         )
455     for channel, center in source_centers.items():
456         tasks.append(
457             (_distance(current, center), 0, "source", channel)
458         )
459     if tasks:
460         _, _, task_type, identifier = min(tasks)
461
462     if task_type is None or identifier is None:
463         break
464     if task_type == "source":
465         self._localize_and_clear(env, self._tracks[identifier])
466         continue
467
468     unvisited.remove(identifier)
469     search_point = DIRECTIONAL_SEARCH_POINTS[identifier]
470     unknown_channels = [
471         channel
472         for channel, track in self._tracks.items()
473         if track.status is ChannelStatus.UNKNOWN

```

```

474     ]
475     for channel in _channel_order(unknown_channels, env.current_channel):
476         track = self._tracks[channel]
477         result = self._measure_action(env, track, search_point, "search")
478         self._search_completed_by_channel[channel].add(identifier)
479         self._handle_measurement(
480             env,
481             track,
482             search_point,
483             result,
484         )
485         self._certify_completed_unknown_channels()
486         if self._confirmed_count() >= 16:
487             break
488         self._search_points_visited += 1
489
490     self._certify_completed_unknown_channels()
491
492     def _choose_untried_q2_candidate(
493         self,
494         env: Environment,
495         track: ChannelTrack,
496         avoided: list[Point],
497         *,
498         force_joint_direction: bool = False,
499     ) -> Point | None:
500         """结合 Q2 几何信息与 Q4 定向历史选择安全探测点。"""
501         if track.last_point is None or track.last_bearing_deg is None:
502             return None
503         current = _point(env.current_position)
504         reference = FirstObservation(track.last_point, track.last_bearing_deg)
505         candidates, circle = generate_sparse_candidates(
506             reference,
507             self._q2_config,
508             track.polygon,
509         )
510         candidates = [
511             point
512             for point in candidates
513             if all(_distance(point, prior) > 10.0 for prior in avoided)
514         ]
515
516         blind_points = self._blind_points_by_channel[track.channel]
517         joint_requested = (
518             self.options.joint_direction_ranking or force_joint_direction
519         )
520         if joint_requested and blind_points:
521             reflected: list[Point] = []
522             for point in candidates:
523                 mirror = _reflect_across_bearing_axis(
524                     point,
525                     track.last_point,
526                     track.last_bearing_deg,

```

```

527         )
528         if any(_distance(mirror, prior) <= 10.0 for prior in avoided):
529             continue
530         if any(
531             _distance(mirror, prior) <= 1e-7
532             for prior in (*candidates, *reflected)
533         ):
534             continue
535         if (
536             conservative_safety_margin(
537                 mirror,
538                 track.polygon,
539                 self._q2_config.receive_min,
540             )
541             < -1e-7
542         ):
543             continue
544         reflected.append(mirror)
545         candidates.extend(reflected)
546     if not candidates:
547         return None
548
549     sources = _representative_polygon_points(
550         track.polygon, self.options.q2_source_samples
551     )
552     errors = _error_grid(
553         self.options.bearing_error_deg, self.options.q2_error_nodes
554     )
555     visible_points = self._visible_points_by_channel[track.channel]
556     joint_enabled = (
557         joint_requested
558         and len(blind_points) >= 2
559         and _directional_visibility_fraction(
560             circle.center,
561             sources,
562             visible_points,
563             blind_points,
564             self.options.direction_heading_samples,
565             self._q2_config.near_radius,
566         )
567         is not None
568     )
569     if joint_enabled:
570         self._joint_direction_rankings += 1
571
572     def ranking_key(score: CandidateScore) -> tuple[float, ...]:
573         base = _score_key(score, current, circle.center)
574         if not joint_enabled:
575             return base
576         fraction = _directional_visibility_fraction(
577             score.point,
578             sources,
579             visible_points,

```

```

580         blind_points,
581         self.options.direction_heading_samples,
582         self._q2_config.near_radius,
583     )
584     assert fraction is not None
585     return (-fraction, *base)
586
587 scores = [
588     evaluate_candidate(
589         candidate,
590         "q4_sparse",
591         reference,
592         self._q2_config,
593         track.polygon,
594         sources,
595         errors,
596     )
597     for candidate in candidates
598 ]
599 scores.sort(key=ranking_key)
600
601 slack = max(0.0, self._q2_config.receive_min - circle.radius)
602 step = min(150.0, max(12.0, 0.4 * slack))
603 directions = [
604     (math.cos(index * math.pi / 4.0), math.sin(index * math.pi / 4.0))
605     for index in range(8)
606 ]
607 refined: list[CandidateScore] = []
608 for seed in scores[: self.options.q2_seed_count]:
609     best = seed
610     local_step = step
611     for _ in range(self.options.q2_local_rounds):
612         for dx, dy in directions:
613             point = (
614                 best.point[0] + local_step * dx,
615                 best.point[1] + local_step * dy,
616             )
617             if any(_distance(point, prior) <= 10.0 for prior in avoided):
618                 continue
619             if (
620                 conservative_safety_margin(
621                     point, track.polygon, self._q2_config.receive_min
622                 )
623                 < -1e-7
624             ):
625                 continue
626             score = evaluate_candidate(
627                 point,
628                 "q4_local",
629                 reference,
630                 self._q2_config,
631                 track.polygon,
632                 sources,

```

```

633         errors,
634     )
635     if ranking_key(score) < ranking_key(best):
636         best = score
637         local_step *= 0.5
638         refined.append(best)
639
640     refined.extend(scores[:3])
641     best = min(refined, key=ranking_key)
642     self._last_candidate_score_by_channel[track.channel] = best
643     return best.point
644
645 def _mirrored_reacquisition_candidate(
646     self,
647     track: ChannelTrack,
648     failed_candidate: Point,
649     avoided: list[Point],
650 ) -> Point | None:
651     """盲区探测失败后，尝试最近可见示向线的另一侧。"""
652     if track.last_point is None or track.last_bearing_deg is None:
653         return None
654     candidate = _reflect_across_bearing_axis(
655         failed_candidate,
656         track.last_point,
657         track.last_bearing_deg,
658     )
659     if any(_distance(candidate, prior) <= 10.0 for prior in avoided):
660         return None
661     if (
662         conservative_safety_margin(
663             candidate,
664             track.polygon,
665             self._q2_config.receive_min,
666         )
667         < -1e-7
668     ):
669         return None
670     return candidate
671
672 def _localize_and_clear(self, env: Environment, track: ChannelTrack) -> None:
673     avoided: list[Point] = []
674     attempts = 0
675     while attempts < self.options.max_active_measurements:
676         circle = minimum_enclosing_circle(track.polygon)
677         if circle.radius <= self.options.guaranteed_clear_radius_m:
678             if self._try_guaranteed_clear(env, track, circle.center):
679                 return
680             break
681
682         candidate = self._choose_untried_q2_candidate(env, track, avoided)
683         if candidate is None:
684             break
685         result = self._measure_action(

```

```

686         env, track, candidate, "active_localization"
687     )
688     track.active_measurements += 1
689     attempts += 1
690     if result.status is MeasureStatus.NO_SIGNAL:
691         track.no_signal_during_localization += 1
692         avoided.append(candidate)
693         self._record_blind_probe(track.channel, candidate)
694         recovery = (
695             self._mirrored_reacquisition_candidate(
696                 track,
697                 candidate,
698                 avoided,
699             )
700             if self.options.mirrored_reacquisition
701             else None
702         )
703         if recovery is not None and attempts < self.options.max_active_measurements
704             :
705             self._mirrored_reacquisition_attempts += 1
706             recovery_result = self._measure_action(
707                 env,
708                 track,
709                 recovery,
710                 "mirrored_reacquisition",
711             )
712             track.active_measurements += 1
713             attempts += 1
714             if recovery_result.status is MeasureStatus.NO_SIGNAL:
715                 track.no_signal_during_localization += 1
716                 avoided.append(recovery)
717                 self._record_blind_probe(track.channel, recovery)
718             else:
719                 self._mirrored_reacquisition_hits += 1
720                 self._handle_measurement(
721                     env,
722                     track,
723                     recovery,
724                     recovery_result,
725                 )
726                 avoided.clear()
727                 if track.status is ChannelStatus.CLEARED:
728                     return
729                 continue
730             self._handle_measurement(env, track, candidate, result)
731             avoided.clear()
732             if track.status is ChannelStatus.CLEARED:
733                 return
734
735         if track.status is ChannelStatus.CLEARED:
736             return
737         if self.options.adaptive_tail_probe:

```

```

738         self._try_adaptive_tail_probe(env, track, avoided)
739         if track.status is ChannelStatus.CLEARED:
740             return
741         circle = minimum_enclosing_circle(track.polygon)
742         if circle.radius <= self.options.clear_radius_m:
743             if self._try_guaranteed_clear(env, track, circle.center):
744                 return
745         self._clear_by_grid(env, track)
746
747     def _build_clear_grid_rows(
748         self,
749         track: ChannelTrack,
750     ) -> tuple[dict[int, list[Point]], dict[int, float], float]:
751         origin = track.first_point or track.polygon[0]
752         bearing = math.radians(track.first_bearing_deg or 0.0)
753         ux, uy = math.cos(bearing), math.sin(bearing)
754         vx, vy = -uy, ux
755         local = [
756             (
757                 (point[0] - origin[0]) * ux + (point[1] - origin[1]) * uy,
758                 (point[0] - origin[0]) * vx + (point[1] - origin[1]) * vy,
759             )
760             for point in track.polygon
761         ]
762         min_u = min(point[0] for point in local)
763         max_u = max(point[0] for point in local)
764         min_v = min(point[1] for point in local)
765         max_v = max(point[1] for point in local)
766         step = self.options.clear_cell_side_m
767         columns = max(1, math.ceil((max_u - min_u) / step))
768         rows = max(1, math.ceil((max_v - min_v) / step))
769         circle_center = minimum_enclosing_circle(track.polygon).center
770         center_v = (
771             (circle_center[0] - origin[0]) * vx
772             + (circle_center[1] - origin[1]) * vy
773         )
774
775         row_points: dict[int, list[Point]] = {}
776         row_v: dict[int, float] = {}
777         for row in range(rows):
778             low_v = min_v + row * step
779             high_v = low_v + step
780             strip_u: list[float] = []
781             for point in local:
782                 if low_v - 1e-9 <= point[1] <= high_v + 1e-9:
783                     strip_u.append(point[0])
784             for index, first in enumerate(local):
785                 second = local[(index + 1) % len(local)]
786                 dv = second[1] - first[1]
787                 if abs(dv) <= 1e-12:
788                     continue
789             for boundary_v in (low_v, high_v):
790                 fraction = (boundary_v - first[1]) / dv

```

```

791         if -1e-12 <= fraction <= 1.0 + 1e-12:
792             strip_u.append(
793                 first[0] + fraction * (second[0] - first[0])
794             )
795         if not strip_u:
796             continue
797         first_column = max(
798             0,
799             min(columns - 1, math.floor((min(strip_u) - min_u) / step)),
800         )
801         last_column = max(
802             0,
803             min(columns - 1, math.floor((max(strip_u) - min_u) / step)),
804         )
805         points: list[Point] = []
806         for column in range(first_column, last_column + 1):
807             u = min_u + (column + 0.5) * step
808             v = min_v + (row + 0.5) * step
809             points.append(
810                 (
811                     origin[0] + u * ux + v * vx,
812                     origin[1] + u * uy + v * vy,
813                 )
814             )
815             row_points[row] = points
816             row_v[row] = min_v + (row + 0.5) * step
817         return row_points, row_v, center_v
818
819     def _clear_grid_plan(
820         self,
821         track: ChannelTrack,
822         current: Point,
823         *,
824         direction_aware: bool,
825     ) -> tuple[list[Point], bool]:
826         row_points, row_v, center_v = self._build_clear_grid_rows(track)
827         point_weights: dict[Point, int] = {}
828         blind_points = self._blind_points_by_channel[track.channel]
829         visible_points = self._visible_points_by_channel[track.channel]
830         if direction_aware and blind_points:
831             point_weights = {
832                 point: _compatible_heading_count(
833                     point,
834                     visible_points,
835                     blind_points,
836                     self.options.direction_heading_samples,
837                 )
838                 for points in row_points.values()
839                 for point in points
840             }
841         used_direction = bool(point_weights) and max(point_weights.values()) > 0
842         if used_direction:
843             row_order = sorted(

```

```

844         row_points,
845         key=lambda row: (
846             -max(point_weights[point] for point in row_points[row]),
847             -sum(point_weights[point] for point in row_points[row]),
848             abs(row_v[row] - center_v),
849         ),
850     )
851 else:
852     row_order = sorted(
853         row_points,
854         key=lambda row: abs(row_v[row] - center_v),
855     )
856
857 plan: list[Point] = []
858 cursor = current
859 for row in row_order:
860     points = list(row_points[row])
861     reverse = False
862     if used_direction:
863         weights = [point_weights[point] for point in points]
864         best_weight = max(weights)
865         first_best = weights.index(best_weight)
866         last_best = len(weights) - 1 - weights[::-1].index(best_weight)
867         forward_rank = (
868             first_best,
869             _distance(cursor, points[0]),
870         )
871         reverse_rank = (
872             len(points) - 1 - last_best,
873             _distance(cursor, points[-1]),
874         )
875         reverse = reverse_rank < forward_rank
876     else:
877         reverse = (
878             _distance(cursor, points[-1])
879             < _distance(cursor, points[0])
880         )
881     if reverse:
882         points.reverse()
883     plan.extend(points)
884     cursor = points[-1]
885     return plan, used_direction
886
887 @staticmethod
888 def _estimated_grid_time_s(plan: list[Point], current: Point) -> float:
889     return estimate_full_clear_time_s(plan, current)
890
891 def _analytic_clear_plan(
892     self,
893     track: ChannelTrack,
894     current: Point,
895 ) -> CertifiedStripPlan | None:
896     if not track.polygon:

```

```

897         return None
898     certified_radius_m = min(
899         self.options.guaranteed_clear_radius_m,
900         self.options.clear_radius_m - 1e-6,
901     )
902     return build_certified_strip_plan(
903         track.polygon,
904         origin=track.first_point or track.polygon[0],
905         bearing_deg=track.first_bearing_deg or 0.0,
906         current=current,
907         certified_radius_m=certified_radius_m,
908     )
909
910     def _tail_terminal_cost_s(
911         self,
912         track: ChannelTrack,
913         current: Point,
914         grid_plan: list[Point],
915     ) -> float:
916         grid_cost_s = self._estimated_grid_time_s(grid_plan, current)
917         if not self.options.tail_terminal_linkage:
918             return grid_cost_s
919
920         self._tail_terminal_linkage_evaluations += 1
921         analytic = self._analytic_clear_plan(track, current)
922         if analytic is None:
923             return grid_cost_s
924         if grid_plan and analytic.estimated_time_s + 1e-9 >= grid_cost_s:
925             return grid_cost_s
926
927         self._tail_terminal_analytic_choices += 1
928         self._tail_terminal_estimated_cost_reduction_s += max(
929             0.0,
930             grid_cost_s - analytic.estimated_time_s,
931         )
932         return analytic.estimated_time_s
933
934     def _tail_visibility_fraction(
935         self,
936         track: ChannelTrack,
937         candidate: Point,
938     ) -> float | None:
939         sources = _representative_polygon_points(
940             track.polygon,
941             max(self.options.q2_source_samples, 96),
942         )
943         return _directional_visibility_fraction(
944             candidate,
945             sources,
946             self._visible_points_by_channel[track.channel],
947             self._blind_points_by_channel[track.channel],
948             self.options.direction_heading_samples,
949             self._q2_config.near_radius,

```

```

950     )
951
952     def _try_adaptive_tail_probe(
953         self,
954         env: Environment,
955         track: ChannelTrack,
956         avoided: list[Point],
957     ) -> None:
958         """仅当额外探测的几何收益超过栅格代价时，才执行该探测。"""
959         if len(self._blind_points_by_channel[track.channel]) < 2:
960             return
961         current = _point(env.current_position)
962         plan, _ = self._clear_grid_plan(
963             track,
964             current,
965             direction_aware=self.options.direction_aware_grid_ordering,
966         )
967         terminal_cost_s = self._tail_terminal_cost_s(track, current, plan)
968         candidate = self._choose_untried_q2_candidate(
969             env,
970             track,
971             avoided,
972             force_joint_direction=True,
973         )
974         if candidate is None:
975             return
976         visible_fraction = self._tail_visibility_fraction(track, candidate)
977         if visible_fraction is None:
978             return
979         if visible_fraction < self.options.adaptive_tail_min_visibility:
980             return
981         probe_cost_s = _distance(current, candidate) / 5.0 + 5.0
982         current_diameter_m = max(
983             _distance(first, second)
984             for first in track.polygon
985             for second in track.polygon
986         )
987         score = self._last_candidate_score_by_channel.get(track.channel)
988         if score is None or current_diameter_m <= 1e-9:
989             return
990         retained_area_fraction = min(
991             1.0,
992             (score.worst_posterior_diameter_m / current_diameter_m) ** 2,
993         )
994         estimated_saving_s = (
995             visible_fraction
996             * terminal_cost_s
997             * (1.0 - retained_area_fraction)
998         )
999         if probe_cost_s >= estimated_saving_s:
1000             return
1001
1002         self._adaptive_tail_probe_attempts += 1

```

```

1003     result = self._measure_action(env, track, candidate, "adaptive_tail_probe")
1004     track.active_measurements += 1
1005     if result.status is MeasureStatus.NO_SIGNAL:
1006         track.no_signal_during_localization += 1
1007         self._record_blind_probe(track.channel, candidate)
1008         return
1009     self._adaptive_tail_probe_hits += 1
1010     self._handle_measurement(env, track, candidate, result)
1011
1012     def _clear_by_grid(self, env: Environment, track: ChannelTrack) -> None:
1013         if not self.options.prioritized_clear_grid:
1014             super()._clear_by_grid(env, track)
1015             return
1016         if not track.polygon:
1017             return
1018
1019         current = _point(env.current_position)
1020         grid_plan, used_direction = self._clear_grid_plan(
1021             track,
1022             current,
1023             direction_aware=self.options.direction_aware_grid_ordering,
1024         )
1025         selected_plan = grid_plan
1026         selected_analytic = False
1027
1028         if self.options.analytic_clear_terminal:
1029             self._analytic_plan_generated += 1
1030             analytic = self._analytic_clear_plan(track, current)
1031             if analytic is None:
1032                 self._analytic_plan_fallbacks += 1
1033             else:
1034                 self._analytic_plan_certified += 1
1035                 grid_cost_s = self._estimated_grid_time_s(grid_plan, current)
1036                 if (
1037                     not grid_plan
1038                     or analytic.estimated_time_s + 1e-9 < grid_cost_s
1039                 ):
1040                     selected_plan = list(analytic.points)
1041                     selected_analytic = True
1042                     self._analytic_plan_selected += 1
1043                     self._analytic_estimated_saving_s += max(
1044                         0.0, grid_cost_s - analytic.estimated_time_s
1045                     )
1046
1047         if used_direction and not selected_analytic:
1048             self._direction_aware_grid_uses += 1
1049         for point in selected_plan:
1050             self._fallback_clear_attempts += 1
1051             result = self._clear_action(env, track, point, "q4_terminal_grid")
1052             if result.status is ClearStatus.SUCCESS:
1053                 self._transition(track, ChannelStatus.CLEARED, "q4_terminal_clear_success")
1054             return
1055     __all__ = [

```

```

1056     "DIRECTIONAL_INNER_RADIUS_M",
1057     "DIRECTIONAL_OUTER_RADIUS_M",
1058     "DIRECTIONAL_SEARCH_POINTS",
1059     "DIRECTIONAL_SEARCH_ROUTE_INDICES",
1060     "GuaranteedDirectionalQ4Strategy",
1061     "Q4StrategyOptions",
1062     "build_directionally_complete_search_points",
1063     "directional_search_certificate",
1064     "directional_search_route_length_m",
1065     "directional_search_triangles",
1066 ]

```

Listing 16 q4_clear_terminal.py

```

1  from __future__ import annotations
2
3  from dataclasses import dataclass
4  import math
5  from typing import Iterable, Sequence
6
7  Point = tuple[float, float]
8
9
10 @dataclass(frozen=True, slots=True)
11 class LocalCoverCell:
12     low_u: float
13     high_u: float
14     low_v: float
15     high_v: float
16
17     @property
18     def center(self) -> Point:
19         return (
20             0.5 * (self.low_u + self.high_u),
21             0.5 * (self.low_v + self.high_v),
22         )
23
24     @property
25     def covering_radius_m(self) -> float:
26         return math.hypot(
27             0.5 * (self.high_u - self.low_u),
28             0.5 * (self.high_v - self.low_v),
29         )
30
31
32 @dataclass(frozen=True, slots=True)
33 class CertifiedStripPlan:
34     points: tuple[Point, ...]
35     cells: tuple[LocalCoverCell, ...]
36     estimated_time_s: float
37     certified_radius_m: float
38     maximum_cell_radius_m: float
39     longitudinal_step_m: float

```

```

40
41
42 def _distance(first: Point, second: Point) -> float:
43     return math.hypot(first[0] - second[0], first[1] - second[1])
44
45
46 def estimate_full_clear_time_s(points: Sequence[Point], current: Point) -> float:
47     """最坏路线代价"""
48     if not points:
49         return 0.0
50     distance_m = _distance(current, points[0]) + sum(
51         _distance(first, second)
52         for first, second in zip(points, points[1:])
53     )
54     return distance_m / 5.0 + 3.0 * max(0, len(points) - 1) + 5.0
55
56
57 def _clip_at_u(
58     polygon: Sequence[Point], boundary: float, *, keep_greater: bool
59 ) -> list[Point]:
60     if not polygon:
61         return []
62
63     def inside(point: Point) -> bool:
64         if keep_greater:
65             return point[0] >= boundary - 1e-10
66         return point[0] <= boundary + 1e-10
67
68     result: list[Point] = []
69     previous = polygon[-1]
70     previous_inside = inside(previous)
71     for current in polygon:
72         current_inside = inside(current)
73         if current_inside != previous_inside:
74             denominator = current[0] - previous[0]
75             if abs(denominator) > 1e-15:
76                 fraction = (boundary - previous[0]) / denominator
77                 result.append(
78                     (
79                         boundary,
80                         previous[1] + fraction * (current[1] - previous[1]),
81                     )
82                 )
83         if current_inside:
84             result.append(current)
85         previous = current
86         previous_inside = current_inside
87     return result
88
89
90 def _clip_to_strip(
91     polygon: Sequence[Point], low_u: float, high_u: float
92 ) -> list[Point]:

```

```

93     return _clip_at_u(
94         _clip_at_u(polygon, low_u, keep_greater=True),
95         high_u,
96         keep_greater=False,
97     )
98
99
100 def _localize(
101     polygon: Sequence[Point], origin: Point, bearing_deg: float
102 ) -> tuple[list[Point], tuple[float, float, float, float]]:
103     angle = math.radians(bearing_deg)
104     ux, uy = math.cos(angle), math.sin(angle)
105     vx, vy = -uy, ux
106     local = [
107         (
108             (point[0] - origin[0]) * ux + (point[1] - origin[1]) * uy,
109             (point[0] - origin[0]) * vx + (point[1] - origin[1]) * vy,
110         )
111         for point in polygon
112     ]
113     return local, (ux, uy, vx, vy)
114
115
116 def _globalize(
117     point: Point,
118     origin: Point,
119     frame: tuple[float, float, float, float],
120 ) -> Point:
121     ux, uy, vx, vy = frame
122     return (
123         origin[0] + point[0] * ux + point[1] * vx,
124         origin[1] + point[0] * uy + point[1] * vy,
125     )
126
127
128 def _ordered_points(
129     cells_by_strip: Sequence[Sequence[LocalCoverCell]],
130     *,
131     reverse_strips: bool,
132     origin: Point,
133     frame: tuple[float, float, float, float],
134     current: Point,
135 ) -> tuple[Point, ...]:
136     strips: Iterable[Sequence[LocalCoverCell]] = cells_by_strip
137     if reverse_strips:
138         strips = reversed(cells_by_strip)
139     route: list[Point] = []
140     cursor = current
141     for strip in strips:
142         candidates = sorted(
143             (_globalize(cell.center, origin, frame) for cell in strip),
144             key=lambda point: (
145                 (point[0] - origin[0]) * frame[2]

```

```

146         + (point[1] - origin[1]) * frame[3]
147     ),
148 )
149     if not candidates:
150         continue
151     if _distance(cursor, candidates[-1]) < _distance(cursor, candidates[0]):
152         candidates.reverse()
153     route.extend(candidates)
154     cursor = candidates[-1]
155     return tuple(route)
156
157
158 def _candidate_steps(radius_m: float) -> tuple[float, ...]:
159     raw = (
160         0.80 * radius_m,
161         1.00 * radius_m,
162         1.20 * radius_m,
163         26.0,
164         1.40 * radius_m,
165         1.60 * radius_m,
166         1.80 * radius_m,
167     )
168     return tuple(
169         sorted(
170             {
171                 round(step, 10)
172                 for step in raw
173                 if 0.0 < step < 2.0 * radius_m
174             }
175         )
176     )
177
178
179 def build_certified_strip_plan(
180     polygon: Sequence[Point],
181     *,
182     origin: Point,
183     bearing_deg: float,
184     current: Point,
185     certified_radius_m: float = 19.5,
186 ) -> CertifiedStripPlan | None:
187     if not polygon or certified_radius_m <= 0.0:
188         return None
189     if any(not (math.isfinite(x) and math.isfinite(y)) for x, y in polygon):
190         return None
191
192     local, frame = _localize(polygon, origin, bearing_deg)
193     min_u = min(point[0] for point in local)
194     max_u = max(point[0] for point in local)
195     span_u = max_u - min_u
196     best: CertifiedStripPlan | None = None
197     padding_m = 1e-7
198

```

```

199 for requested_step in _candidate_steps(certified_radius_m):
200     strip_count = max(1, math.ceil(span_u / requested_step))
201     step = span_u / strip_count if span_u > 1e-12 else 0.0
202     half_step = 0.5 * step
203     if half_step >= certified_radius_m:
204         continue
205     maximum_band_height = 2.0 * math.sqrt(
206         max(0.0, certified_radius_m**2 - half_step**2)
207     )
208     if maximum_band_height <= 0.0:
209         continue
210
211     cells_by_strip: list[list[LocalCoverCell]] = []
212     valid = True
213     for index in range(strip_count):
214         low_u = min_u + index * step
215         high_u = max_u if index + 1 == strip_count else low_u + step
216         clipped = _clip_to_strip(local, low_u, high_u)
217         if not clipped:
218             cells_by_strip.append([])
219             continue
220         low_v = min(point[1] for point in clipped) - padding_m
221         high_v = max(point[1] for point in clipped) + padding_m
222         width_v = max(0.0, high_v - low_v)
223         band_count = max(1, math.ceil(width_v / maximum_band_height))
224         band_height = width_v / band_count
225         strip_cells: list[LocalCoverCell] = []
226         for band in range(band_count):
227             cell = LocalCoverCell(
228                 low_u=low_u,
229                 high_u=high_u,
230                 low_v=low_v + band * band_height,
231                 high_v=(
232                     high_v
233                     if band + 1 == band_count
234                     else low_v + (band + 1) * band_height
235                 ),
236             )
237             if cell.covering_radius_m > certified_radius_m + 1e-9:
238                 valid = False
239                 break
240             strip_cells.append(cell)
241         if not valid:
242             break
243         cells_by_strip.append(strip_cells)
244     if not valid:
245         continue
246
247     cells = tuple(cell for strip in cells_by_strip for cell in strip)
248     if not cells:
249         continue
250     route_candidates = (
251         _ordered_points(

```

```

252         cells_by_strip,
253         reverse_strips=False,
254         origin=origin,
255         frame=frame,
256         current=current,
257     ),
258     _ordered_points(
259         cells_by_strip,
260         reverse_strips=True,
261         origin=origin,
262         frame=frame,
263         current=current,
264     ),
265 )
266 points = min(
267     route_candidates,
268     key=lambda route: estimate_full_clear_time_s(route, current),
269 )
270 plan = CertifiedStripPlan(
271     points=points,
272     cells=cells,
273     estimated_time_s=estimate_full_clear_time_s(points, current),
274     certified_radius_m=certified_radius_m,
275     maximum_cell_radius_m=max(
276         cell.covering_radius_m for cell in cells
277     ),
278     longitudinal_step_m=step,
279 )
280 if best is None or plan.estimated_time_s < best.estimated_time_s:
281     best = plan
282 return best
283
284
285 __all__ = [
286     "CertifiedStripPlan",
287     "LocalCoverCell",
288     "build_certified_strip_plan",
289     "estimate_full_clear_time_s",
290 ]

```

Listing 17 run_q4_official.py

```

1  """在官方 HTTP 模拟器上运行 Q4 策略。"""
2  from __future__ import annotations
3
4  import argparse
5  from datetime import datetime
6  import json
7  from pathlib import Path
8  import sys
9  from time import perf_counter
10
11 _PROJECT_ROOT = Path(__file__).resolve().parents[2]

```

```

12 _SHARED_CODE = _PROJECT_ROOT / "code"
13 if str(_SHARED_CODE) not in sys.path:
14     sys.path.insert(0, str(_SHARED_CODE))
15
16 from radio_sim.environment.base import EnvironmentStateError # noqa: E402
17 from radio_sim.environment.official import OfficialEnvironment # noqa: E402
18 from q4_strategy import GuaranteedDirectionalQ4Strategy, Q4StrategyOptions # noqa: E402
19
20
21 def available_path(path: Path) -> Path:
22     if not path.exists():
23         return path
24     stamp = datetime.now().strftime("%Y%m%d_%H%M%S_%f")
25     return path.with_name(f"{path.stem}_{stamp}{path.suffix}")
26
27
28 def parse_args() -> argparse.Namespace:
29     parser = argparse.ArgumentParser(description=__doc__)
30     parser.add_argument("--robot-id", required=True, help="当前登录模拟器的参赛队号")
31     parser.add_argument("--base-url", default="http://127.0.0.1:2026")
32     parser.add_argument("--timeout", type=float, default=5.0)
33     parser.add_argument("--max-retries", type=int, default=2)
34     parser.add_argument("--retry-backoff", type=float, default=0.2)
35     parser.add_argument("--exit-reserve", type=float, default=15.0)
36     parser.add_argument("--safety-buffer", type=float, default=5.0)
37     parser.add_argument("--max-active-measurements", type=int, default=2)
38     parser.add_argument(
39         "--analytic-clear-terminal",
40         action=argparse.BooleanOptionalAction,
41         default=False,
42     )
43     parser.add_argument(
44         "--tail-terminal-linkage",
45         action=argparse.BooleanOptionalAction,
46         default=False,
47         help="启用 A2; 同时会启用 A1 解析终端",
48     )
49     parser.add_argument("--log", type=Path, help="逐请求 JSONL 日志路径")
50     parser.add_argument("--output", type=Path, help="运行摘要 JSON 路径")
51     return parser.parse_args()
52
53
54 def main() -> int:
55     args = parse_args()
56     stamp = datetime.now().strftime("%Y%m%d_%H%M%S_%f")
57     results_dir = Path(__file__).resolve().parents[1] / "results"
58     log_path = available_path(
59         (args.log or results_dir / f"q4_official_{stamp}.jsonl").resolve()
60     )
61     output_path = available_path(
62         (args.output or results_dir / f"q4_official_{stamp}.json").resolve()
63     )
64

```

```

65     env = OfficialEnvironment(
66         robot_id=args.robot_id,
67         base_url=args.base_url,
68         timeout_s=args.timeout,
69         max_retries=args.max_retries,
70         retry_backoff_s=args.retry_backoff,
71         exit_reserve_s=args.exit_reserve,
72         log_path=log_path,
73     )
74     strategy = GuaranteedDirectionalQ4Strategy(
75         Q4StrategyOptions(
76             safety_buffer_m=args.safety_buffer,
77             max_active_measurements=args.max_active_measurements,
78             analytic_clear_terminal=(
79                 args.analytic_clear_terminal or args.tail_terminal_linkage
80             ),
81             tail_terminal_linkage=args.tail_terminal_linkage,
82         )
83     )
84
85     started = perf_counter()
86     result = strategy.run(env)
87     cleanup_error: str | None = None
88     if env.state.is_active:
89         if env.has_pending_request:
90             cleanup_error = (
91                 "unresolved request retained; no new /exit was sent: "
92                 f"{env.pending_request}"
93             )
94         else:
95             try:
96                 env.exit()
97             except EnvironmentStateError as exc:
98                 cleanup_error = f"{type(exc).__name__}: {exc}"
99     wall_time_s = perf_counter() - started
100
101     state = env.state
102     summary = {
103         "result_status": "COMPLETE" if result.completed else "INCOMPLETE",
104         "strategy_name": result.strategy_name,
105         "strategy_completed": result.completed,
106         "strategy_note": result.note,
107         **strategy.analytic_terminal_diagnostics,
108         "successful_clear_count": env.successful_clear_count,
109         "measure_count": env.measure_count,
110         "clear_count": env.clear_count,
111         "unique_request_count": env.request_count,
112         "http_attempt_count": env.http_attempt_count,
113         "virtual_time_s": state.virtual_time_s,
114         "average_localize_clear_time_s": (
115             state.virtual_time_s / env.successful_clear_count
116             if env.successful_clear_count
117             else None

```

```

118     ),
119     "wall_time_s": wall_time_s,
120     "entered_remaining_real_duration_s": env.entered_remaining_real_duration_s,
121     "remaining_real_duration_s": env.remaining_real_duration_s,
122     "final_position": {"x": state.current_position.x, "y": state.current_position.y},
123     "final_measurement_channel": state.current_channel,
124     "environment_finished": state.is_finished,
125     "cleanup_error": cleanup_error,
126     "pending_request": env.pending_request,
127     "completion_certificate": (
128         strategy.completion_certificate.to_record()
129         if strategy.completion_certificate is not None
130         else None
131     ),
132     "request_log": str(log_path),
133     "output": str(output_path),
134 }
135 output_path.parent.mkdir(parents=True, exist_ok=True)
136 output_path.write_text(
137     json.dumps(summary, ensure_ascii=False, indent=2) + "\n",
138     encoding="utf-8",
139 )
140 print(json.dumps(summary, ensure_ascii=False, indent=2))
141 return 0 if result.completed and cleanup_error is None else 2
142
143
144 if __name__ == "__main__":
145     raise SystemExit(main())

```

Listing 18 run_q4_batch.py

```

1  """在可复现的本地随机种子区间上批量运行 Q4 策略。"""
2  from __future__ import annotations
3
4  import argparse
5  import csv
6  from datetime import datetime
7  import json
8  from pathlib import Path
9  import sys
10
11  _PROJECT_ROOT = Path(__file__).resolve().parents[2]
12  _SHARED_CODE = _PROJECT_ROOT / "code"
13  if str(_SHARED_CODE) not in sys.path:
14      sys.path.insert(0, str(_SHARED_CODE))
15
16  from q4_strategy import GuaranteedDirectionalQ4Strategy, Q4StrategyOptions # noqa: E402
17  from radio_sim.evaluation import run_single # noqa: E402
18
19
20  def default_output_path(start_seed: int, count: int) -> Path:
21      root = Path(__file__).resolve().parents[1] / "results"
22      stamp = datetime.now().strftime("%Y%m%d_%H%M%S_%f")

```

```

23     return root / f"q4_batch_seed{start_seed}_n{count}_{stamp}.csv"
24
25
26 def available_path(path: Path) -> Path:
27     if not path.exists():
28         return path
29     stamp = datetime.now().strftime("%Y%m%d_%H%M%S_%f")
30     return path.with_name(f"{path.stem}_{stamp}{path.suffix}")
31
32
33 def parse_args() -> argparse.Namespace:
34     parser = argparse.ArgumentParser(description=__doc__)
35     parser.add_argument("--start-seed", type=int, default=0)
36     parser.add_argument("--count", type=int, default=10)
37     parser.add_argument("--safety-buffer", type=float, default=5.0)
38     parser.add_argument("--max-active-measurements", type=int, default=2)
39     parser.add_argument(
40         "--analytic-clear-terminal",
41         action=argparse.BooleanOptionalAction,
42         default=False,
43     )
44     parser.add_argument(
45         "--tail-terminal-linkage",
46         action=argparse.BooleanOptionalAction,
47         default=False,
48         help="启用 A2; 同时会启用 A1 解析终端",
49     )
50     parser.add_argument("--output", type=Path)
51     parser.add_argument("--no-write", action="store_true")
52     parser.add_argument("--task-insertion", action="store_true")
53     parser.add_argument("--joint-direction-ranking", action="store_true")
54     parser.add_argument(
55         "--adaptive-tail-probe",
56         action=argparse.BooleanOptionalAction,
57         default=True,
58     )
59     parser.add_argument(
60         "--adaptive-tail-min-visibility", type=float, default=0.30
61     )
62     parser.add_argument("--direction-aware-grid-ordering", action="store_true")
63     return parser.parse_args()
64
65
66 def main() -> int:
67     args = parse_args()
68     if args.count <= 0:
69         raise SystemExit("--count 必须为正数")
70
71     options = Q4StrategyOptions(
72         safety_buffer_m=args.safety_buffer,
73         max_active_measurements=args.max_active_measurements,
74         analytic_clear_terminal=(
75             args.analytic_clear_terminal or args.tail_terminal_linkage

```

```

76     ),
77     tail_terminal_linkage=args.tail_terminal_linkage,
78     task_insertion_scheduler=args.task_insertion,
79     joint_direction_ranking=args.joint_direction_ranking,
80     adaptive_tail_probe=args.adaptive_tail_probe,
81     adaptive_tail_min_visibility=args.adaptive_tail_min_visibility,
82     direction_aware_grid_ordering=args.direction_aware_grid_ordering,
83 )
84 results = []
85 records = []
86 for seed in range(args.start_seed, args.start_seed + args.count):
87     strategy = GuaranteedDirectionalQ4Strategy(options)
88     result = run_single(strategy=strategy, seed=seed, problem="Q4")
89     record = result.to_record()
90     record.update(strategy.analytic_terminal_diagnostics)
91     results.append(result)
92     records.append(record)
93
94 output: Path | None = None
95 if not args.no_write:
96     output = available_path(
97         (args.output or default_output_path(args.start_seed, args.count)).resolve()
98     )
99     output.parent.mkdir(parents=True, exist_ok=True)
100     with output.open("w", newline="", encoding="utf-8-sig") as handle:
101         writer = csv.DictWriter(handle, fieldnames=list(records[0]))
102         writer.writeheader()
103         writer.writerows(records)
104
105 all_cleared_count = sum(result.all_cleared for result in results)
106 summary = {
107     "problem": "Q4",
108     "run_count": len(results),
109     "seed_start": args.start_seed,
110     "seed_end": args.start_seed + args.count - 1,
111     "all_cleared_count": all_cleared_count,
112     "all_cleared_rate": all_cleared_count / len(results),
113     "strategy_completed_rate": sum(result.strategy_completed for result in results)
114     / len(results),
115     "mean_virtual_time_s": sum(result.virtual_time_s for result in results)
116     / len(results),
117     "mean_average_localize_clear_time_s": sum(
118         result.average_localize_clear_time_s or 0.0 for result in results
119     )
120     / len(results),
121     "max_virtual_time_s": max(result.virtual_time_s for result in results),
122     "mean_wall_time_s": sum(result.wall_time_s for result in results)
123     / len(results),
124     "mean_failed_clear_count": sum(
125         result.failed_clear_count for result in results
126     )
127     / len(results),
128     "analytic_plan_generated_count": sum(

```

```

129         int(record["analytic_plan_generated"]) for record in records
130     ),
131     "analytic_plan_certified_count": sum(
132         int(record["analytic_plan_certified"]) for record in records
133     ),
134     "analytic_plan_selected_count": sum(
135         int(record["analytic_plan_selected"]) for record in records
136     ),
137     "analytic_plan_fallback_count": sum(
138         int(record["analytic_plan_fallbacks"]) for record in records
139     ),
140     "analytic_estimated_saving_s": sum(
141         float(record["analytic_estimated_saving_s"]) for record in records
142     ),
143     "tail_terminal_linkage_evaluation_count": sum(
144         int(record["tail_terminal_linkage_evaluations"])
145         for record in records
146     ),
147     "tail_terminal_analytic_choice_count": sum(
148         int(record["tail_terminal_analytic_choices"])
149         for record in records
150     ),
151     "tail_terminal_estimated_cost_reduction_s": sum(
152         float(record["tail_terminal_estimated_cost_reduction_s"])
153         for record in records
154     ),
155     "output": str(output) if output is not None else None,
156 }
157 print(json.dumps(summary, ensure_ascii=False, indent=2))
158 return 0
159
160
161 if __name__ == "__main__":
162     raise SystemExit(main())

```

Listing 19 run_q4.py

```

1  """在一个可复现的本地场景上运行 Q4 策略。"""
2  from __future__ import annotations
3
4  import argparse
5  from datetime import datetime
6  import json
7  from pathlib import Path
8  import sys
9
10 _PROJECT_ROOT = Path(__file__).resolve().parents[2]
11 _SHARED_CODE = _PROJECT_ROOT / "code"
12 if str(_SHARED_CODE) not in sys.path:
13     sys.path.insert(0, str(_SHARED_CODE))
14
15 from q4_strategy import GuaranteedDirectionalQ4Strategy, Q4StrategyOptions # noqa: E402
16 from radio_sim.evaluation import run_single # noqa: E402

```

```

17
18
19 def default_output_path(seed: int) -> Path:
20     root = Path(__file__).resolve().parents[1] / "results"
21     stamp = datetime.now().strftime("%Y%m%d_%H%M%S%f")
22     return root / f"q4_single_seed{seed}_{stamp}.json"
23
24
25 def available_path(path: Path) -> Path:
26     if not path.exists():
27         return path
28     stamp = datetime.now().strftime("%Y%m%d_%H%M%S%f")
29     return path.with_name(f"{path.stem}_{stamp}{path.suffix}")
30
31
32 def parse_args() -> argparse.Namespace:
33     parser = argparse.ArgumentParser(description=__doc__)
34     parser.add_argument("--seed", type=int, default=12345)
35     parser.add_argument("--safety-buffer", type=float, default=5.0)
36     parser.add_argument("--max-active-measurements", type=int, default=2)
37     parser.add_argument(
38         "--analytic-clear-terminal",
39         action=argparse.BooleanOptionalAction,
40         default=False,
41     )
42     parser.add_argument(
43         "--tail-terminal-linkage",
44         action=argparse.BooleanOptionalAction,
45         default=False,
46         help="启用 A2; 同时会启用 A1 解析终端",
47     )
48     parser.add_argument("--output", type=Path)
49     parser.add_argument("--no-write", action="store_true")
50     return parser.parse_args()
51
52
53 def main() -> int:
54     args = parse_args()
55     options = Q4StrategyOptions(
56         safety_buffer_m=args.safety_buffer,
57         max_active_measurements=args.max_active_measurements,
58         analytic_clear_terminal=(
59             args.analytic_clear_terminal or args.tail_terminal_linkage
60         ),
61         tail_terminal_linkage=args.tail_terminal_linkage,
62     )
63     strategy = GuaranteedDirectionalQ4Strategy(options)
64     metrics = run_single(
65         strategy=strategy,
66         seed=args.seed,
67         problem="Q4",
68     )
69     record = metrics.to_record()

```

```

70     record.update(strategy.analytic_terminal_diagnostics)
71
72     output: Path | None = None
73     if not args.no_write:
74         output = available_path((args.output or default_output_path(args.seed)).resolve())
75         output.parent.mkdir(parents=True, exist_ok=True)
76         output.write_text(
77             json.dumps(record, ensure_ascii=False, indent=2) + "\n",
78             encoding="utf-8",
79         )
80
81     payload = dict(record)
82     payload["output"] = str(output) if output is not None else None
83     print(json.dumps(payload, ensure_ascii=False, indent=2))
84     return 0
85
86
87 if __name__ == "__main__":
88     raise SystemExit(main())

```

E 公共仿真支撑模块代码

Listing 20 domain/models.py

```

1 from __future__ import annotations
2
3 from dataclasses import dataclass
4 from enum import Enum
5 from math import isfinite
6
7
8 class ProblemType(str, Enum):
9     Q3 = "Q3"
10    Q4 = "Q4"
11
12    @classmethod
13    def coerce(cls, value: "ProblemType | str") -> "ProblemType":
14        if isinstance(value, cls):
15            return value
16        try:
17            return cls(str(value).upper())
18        except ValueError as exc:
19            raise ValueError("problem must be 'Q3' or 'Q4'") from exc
20
21
22 class MeasureStatus(str, Enum):
23     NO_SIGNAL = "no_signal"
24     NEAR = "near"
25     DIRECTION = "direction"
26
27
28 class ClearStatus(str, Enum):

```

```

29 SUCCESS = "success"
30 NO_TARGET_IN_RANGE = "no_target_in_range"
31
32
33 @dataclass(frozen=True, slots=True)
34 class Position:
35     x: float
36     y: float
37
38     def __post_init__(self) -> None:
39         x = float(self.x)
40         y = float(self.y)
41         if not isfinite(x) or not isfinite(y):
42             raise ValueError("position coordinates must be finite")
43         object.__setattr__(self, "x", 0.0 if x == 0.0 else x)
44         object.__setattr__(self, "y", 0.0 if y == 0.0 else y)
45
46
47 @dataclass(frozen=True, slots=True)
48 class EnvironmentState:
49     current_position: Position
50     current_channel: int
51     virtual_time_s: float
52     is_active: bool
53     is_finished: bool
54
55
56 @dataclass(frozen=True, slots=True)
57 class MeasureResult:
58     status: MeasureStatus
59     angle_deg: float | None
60     action_time_s: float
61     virtual_time_s: float
62
63
64 @dataclass(frozen=True, slots=True)
65 class ClearResult:
66     status: ClearStatus
67     action_time_s: float
68     virtual_time_s: float
69
70
71 @dataclass(frozen=True, slots=True)
72 class StrategyResult:
73     strategy_name: str
74     completed: bool = True
75     note: str = ""
76
77
78 @dataclass(frozen=True, slots=True)
79 class SimulationMetrics:
80     seed: int
81     problem: ProblemType

```

```

82     strategy_name: str
83     source_count: int
84     cleared_count: int
85     cleared_ratio: float
86     all_cleared: bool
87     virtual_time_s: float
88     average_localize_clear_time_s: float | None
89     total_distance_m: float
90     measure_count: int
91     clear_count: int
92     failed_clear_count: int
93     channel_switch_count: int
94     strategy_completed: bool
95     wall_time_s: float
96
97     def to_record(self) -> dict[str, object]:
98         return {
99             "seed": self.seed,
100             "problem": self.problem.value,
101             "strategy_name": self.strategy_name,
102             "source_count": self.source_count,
103             "cleared_count": self.cleared_count,
104             "cleared_ratio": self.cleared_ratio,
105             "all_cleared": self.all_cleared,
106             "virtual_time_s": self.virtual_time_s,
107             "average_localize_clear_time_s": self.average_localize_clear_time_s,
108             "total_distance_m": self.total_distance_m,
109             "measure_count": self.measure_count,
110             "clear_count": self.clear_count,
111             "failed_clear_count": self.failed_clear_count,
112             "channel_switch_count": self.channel_switch_count,
113             "strategy_completed": self.strategy_completed,
114             "wall_time_s": self.wall_time_s,
115         }

```

Listing 21 environment/base.py

```

1  from __future__ import annotations
2
3  from typing import Protocol, runtime_checkable
4
5  from radio_sim.domain.models import (
6      ClearResult,
7      EnvironmentState,
8      MeasureResult,
9      Position,
10 )
11
12
13  class EnvironmentStateError(RuntimeError):
14      """当动作不符合环境生命周期时抛出"""
15
16

```

```

17 class InvalidActionError(ValueError):
18     """当动作参数超出公开协议约束时抛出"""
19
20
21 @runtime_checkable
22 class Environment(Protocol):
23     @property
24     def state(self) -> EnvironmentState:
25         ...
26
27     @property
28     def current_position(self) -> Position:
29         ...
30
31     @property
32     def current_channel(self) -> int:
33         ...
34
35     @property
36     def virtual_time_s(self) -> float:
37         ...
38
39     def enter(self) -> EnvironmentState:
40         ...
41
42     def measure(self, position: Position, channel: int) -> MeasureResult:
43         ...
44
45     def clear(self, position: Position, channel: int) -> ClearResult:
46         ...
47
48     def exit(self) -> EnvironmentState:
49         ...

```

Listing 22 environment/local.py

```

1 from __future__ import annotations
2
3 from dataclasses import dataclass
4
5 from radio_sim.domain.models import (
6     ClearResult,
7     ClearStatus,
8     EnvironmentState,
9     MeasureResult,
10    MeasureStatus,
11    Position,
12    ProblemType,
13 )
14 from radio_sim.environment.base import EnvironmentStateError, InvalidActionError
15 from radio_sim.scenario.config import ScenarioConfig
16 from radio_sim.scenario.generator import ScenarioGenerator
17 from radio_sim.scenario.models import Scenario, Source, SourceType

```

```

18 from radio_sim.utils.deterministic_noise import measurement_error_deg
19 from radio_sim.utils.geometry import bearing_deg, distance, in_directional_coverage
20
21
22 @dataclass(frozen=True, slots=True)
23 class PhysicsConfig:
24     movement_speed_mps: float = 5.0
25     channel_switch_time_s: float = 1.0
26     measurement_time_s: float = 5.0
27     failed_clear_time_s: float = 3.0
28     successful_clear_time_s: float = 5.0
29     near_distance_m: float = 5.0
30     clear_distance_m: float = 20.0
31     directional_half_angle_deg: float = 90.0
32     coordinate_limit_m: float = 2_000_000.0
33     max_virtual_duration_s: float = 360_000.0
34     angle_decimal_places: int = 2
35
36     def __post_init__(self) -> None:
37         positive_fields = (
38             self.movement_speed_mps,
39             self.measurement_time_s,
40             self.failed_clear_time_s,
41             self.successful_clear_time_s,
42             self.near_distance_m,
43             self.clear_distance_m,
44             self.coordinate_limit_m,
45             self.max_virtual_duration_s,
46         )
47         if any(value <= 0 for value in positive_fields):
48             raise ValueError("physics distances, durations, limits, and speed must be
49 positive")
50         if self.channel_switch_time_s < 0:
51             raise ValueError("channel switch time cannot be negative")
52         if not 0 < self.directional_half_angle_deg <= 180:
53             raise ValueError("directional half angle must be in (0, 180]")
54         if self.angle_decimal_places < 0:
55             raise ValueError("angle_decimal_places cannot be negative")
56
57 @dataclass(frozen=True, slots=True)
58 class _EvaluationSnapshot:
59     total_distance_m: float
60     measure_count: int
61     clear_count: int
62     failed_clear_count: int
63     channel_switch_count: int
64     cleared_count: int
65     source_count: int
66     timed_out: bool
67
68
69 class LocalEnvironment:

```

```

70
71     def __init__(
72         self,
73         scenario: Scenario,
74         physics: PhysicsConfig | None = None,
75     ) -> None:
76         self._seed = scenario.seed
77         self._sources_by_channel = scenario.source_by_channel()
78         self._physics = physics or PhysicsConfig()
79         self._cleared_channels: set[int] = set()
80         self._position = Position(0.0, 0.0)
81         self._channel = 1
82         self._virtual_time_s = 0.0
83         self._active = False
84         self._finished = False
85         self._timed_out = False
86         self._total_distance_m = 0.0
87         self._measure_count = 0
88         self._clear_count = 0
89         self._failed_clear_count = 0
90         self._channel_switch_count = 0
91
92     @classmethod
93     def from_seed(
94         cls,
95         seed: int,
96         problem: ProblemType | str,
97         scenario_config: ScenarioConfig | None = None,
98         physics: PhysicsConfig | None = None,
99     ) -> "LocalEnvironment":
100         problem_type = ProblemType.coerce(problem)
101         config = scenario_config or ScenarioConfig.for_problem(problem_type)
102         if config.problem is not problem_type:
103             config = config.with_problem(problem_type)
104         scenario = ScenarioGenerator().generate(seed=seed, config=config)
105         return cls(scenario=scenario, physics=physics)
106
107     @property
108     def state(self) -> EnvironmentState:
109         return EnvironmentState(
110             current_position=self._position,
111             current_channel=self._channel,
112             virtual_time_s=self._virtual_time_s,
113             is_active=self._active,
114             is_finished=self._finished,
115         )
116
117     @property
118     def current_position(self) -> Position:
119         return self._position
120
121     @property
122     def current_channel(self) -> int:

```

```

123         return self._channel
124
125     @property
126     def virtual_time_s(self) -> float:
127         return self._virtual_time_s
128
129     def _evaluation_snapshot(self) -> _EvaluationSnapshot:
130         return _EvaluationSnapshot(
131             total_distance_m=self._total_distance_m,
132             measure_count=self._measure_count,
133             clear_count=self._clear_count,
134             failed_clear_count=self._failed_clear_count,
135             channel_switch_count=self._channel_switch_count,
136             cleared_count=len(self._cleared_channels),
137             source_count=len(self._sources_by_channel),
138             timed_out=self._timed_out,
139         )
140
141     def enter(self) -> EnvironmentState:
142         if self._active or self._finished:
143             raise EnvironmentStateError("enter may be called exactly once")
144         self._active = True
145         return self.state
146
147     def exit(self) -> EnvironmentState:
148         self._require_active()
149         self._active = False
150         self._finished = True
151         return self.state
152
153     def measure(self, position: Position, channel: int) -> MeasureResult:
154         self._require_active()
155         position, channel = self._validate_action(position, channel)
156
157         move_distance = distance(self._position, position)
158         switched = channel != self._channel
159         action_time = (
160             move_distance / self._physics.movement_speed_mps
161             + self._physics.measurement_time_s
162             + (self._physics.channel_switch_time_s if switched else 0.0)
163         )
164         self._advance_position(position, move_distance, action_time)
165         self._channel = channel
166         self._measure_count += 1
167         if switched:
168             self._channel_switch_count += 1
169
170         source = self._available_source(channel)
171         if source is None or not self._can_receive(source, position):
172             result = MeasureResult(
173                 status=MeasureStatus.NO_SIGNAL,
174                 angle_deg=None,
175                 action_time_s=action_time,

```

```

176         virtual_time_s=self._virtual_time_s,
177     )
178     else:
179         source_distance = distance(position, source.position)
180         if source_distance <= self._physics.near_distance_m:
181             result = MeasureResult(
182                 status=MeasureStatus.NEAR,
183                 angle_deg=None,
184                 action_time_s=action_time,
185                 virtual_time_s=self._virtual_time_s,
186             )
187         else:
188             true_bearing = bearing_deg(position, source.position)
189             noisy_bearing = (
190                 true_bearing
191                 + measurement_error_deg(self._seed, channel, position)
192             ) % 360.0
193             result = MeasureResult(
194                 status=MeasureStatus.DIRECTION,
195                 angle_deg=round(noisy_bearing, self._physics.angle_decimal_places) %
360.0,
196                 action_time_s=action_time,
197                 virtual_time_s=self._virtual_time_s,
198             )
199
200         self._finish_if_timed_out()
201         return result
202
203     def clear(self, position: Position, channel: int) -> ClearResult:
204         self._require_active()
205         position, channel = self._validate_action(position, channel)
206
207         move_distance = distance(self._position, position)
208         source = self._available_source(channel)
209         success = (
210             source is not None
211             and distance(position, source.position) <= self._physics.clear_distance_m
212         )
213         clear_time = (
214             self._physics.successful_clear_time_s
215             if success
216             else self._physics.failed_clear_time_s
217         )
218         action_time = move_distance / self._physics.movement_speed_mps + clear_time
219         self._advance_position(position, move_distance, action_time)
220         self._clear_count += 1
221
222         if success:
223             self._cleared_channels.add(channel)
224             status = ClearStatus.SUCCESS
225         else:
226             self._failed_clear_count += 1
227             status = ClearStatus.NO_TARGET_IN_RANGE

```

```

228
229         result = ClearResult(
230             status=status,
231             action_time_s=action_time,
232             virtual_time_s=self._virtual_time_s,
233         )
234         self._finish_if_timed_out()
235         return result
236
237     def _available_source(self, channel: int) -> Source | None:
238         if channel in self._cleared_channels:
239             return None
240         return self._sources_by_channel.get(channel)
241
242     def _can_receive(self, source: Source, receiver: Position) -> bool:
243         if distance(source.position, receiver) > source.receive_radius_m:
244             return False
245         if source.source_type is SourceType.OMNIDIRECTIONAL:
246             return True
247         assert source.direction_deg is not None
248         return in_directional_coverage(
249             source_position=source.position,
250             source_direction_deg=source.direction_deg,
251             receiver_position=receiver,
252             half_angle_deg=self._physics.directional_half_angle_deg,
253         )
254
255     def _validate_action(self, position: Position, channel: int) -> tuple[Position, int]:
256         if not isinstance(position, Position):
257             raise InvalidActionError("position must be a Position instance")
258         if abs(position.x) > self._physics.coordinate_limit_m or abs(position.y) > self._physics.coordinate_limit_m:
259             raise InvalidActionError("position coordinate exceeds simulator limit")
260         if isinstance(channel, bool) or not isinstance(channel, int) or not 1 <= channel <=
261         20:
262             raise InvalidActionError("channel must be an integer in 1..20")
263         return position, channel
264
265     def _require_active(self) -> None:
266         if not self._active:
267             if self._timed_out:
268                 raise EnvironmentStateError("environment ended at the virtual-time limit")
269                 raise EnvironmentStateError("environment is not active; call enter first")
270
271     def _advance_position(
272         self,
273         position: Position,
274         move_distance: float,
275         action_time_s: float,
276     ) -> None:
277         self._position = position
278         self._total_distance_m += move_distance
279         self._virtual_time_s += action_time_s

```

```

279
280     def _finish_if_timed_out(self) -> None:
281         if self._virtual_time_s >= self._physics.max_virtual_duration_s:
282             self._timed_out = True
283             self._active = False
284             self._finished = True

```

Listing 23 environment/official.py

```

1  """官方 Q3/Q4 模拟器的 HTTP+JSON 适配器。"""
2  from __future__ import annotations
3
4  from collections.abc import Callable, Mapping
5  from dataclasses import dataclass
6  import json
7  import math
8  from pathlib import Path
9  import threading
10 import time
11 from typing import Any
12 from urllib.error import HTTPError, URLError
13 from urllib.request import Request, urlopen
14
15 from radio_sim.domain.models import (
16     ClearResult,
17     ClearStatus,
18     EnvironmentState,
19     MeasureResult,
20     MeasureStatus,
21     Position,
22 )
23 from radio_sim.environment.base import EnvironmentStateError, InvalidActionError
24
25
26 class OfficialEnvironmentError(EnvironmentStateError):
27     """模拟器故障的基类"""
28
29
30 class OfficialTransportError(OfficialEnvironmentError):
31     """重试后仍未获得可信的 HTTP 响应。"""
32
33
34 class OfficialHTTPError(OfficialEnvironmentError):
35     """模拟器返回了非 200 的 HTTP 响应。"""
36
37     def __init__(self, status_code: int, response: object) -> None:
38         self.status_code = status_code
39         self.response = response
40         super().__init__(f"official simulator returned HTTP {status_code}: {response!r}")
41
42
43 class OfficialRejectedError(OfficialEnvironmentError):
44     """模拟器返回 HTTP 200, 但 accepted=false。"""

```

```

45
46     def __init__(self, response: Mapping[str, Any]) -> None:
47         self.response = dict(response)
48         super().__init__(f"official simulator rejected the action: {dict(response)!r}")
49
50
51     class OfficialProtocolError(OfficialEnvironmentError):
52         """响应不符合已公布的 JSON 结构。"""
53
54
55     class OfficialDeadlineError(OfficialEnvironmentError):
56         """已触及时间边界，此时应尝试退出。"""
57
58
59     class OfficialPendingRequestError(OfficialEnvironmentError):
60         """先前请求的结果未知，必须优先重新放该请求。"""
61
62
63     @dataclass(frozen=True, slots=True)
64     class _PendingRequest:
65         action: str
66         path: str
67         payload: dict[str, Any]
68
69
70     def _json_without_duplicate_keys(raw: bytes) -> dict[str, Any]:
71         try:
72             text = raw.decode("utf-8")
73         except UnicodeDecodeError as exc:
74             raise OfficialProtocolError("response is not UTF-8") from exc
75
76         def pairs_hook(pairs: list[tuple[str, Any]]) -> dict[str, Any]:
77             result: dict[str, Any] = {}
78             for key, value in pairs:
79                 if key in result:
80                     raise OfficialProtocolError(f"response contains duplicate key {key!r}")
81                 result[key] = value
82             return result
83
84         try:
85             value = json.loads(text, object_pairs_hook=pairs_hook)
86         except OfficialProtocolError:
87             raise
88         except (json.JSONDecodeError, ValueError) as exc:
89             raise OfficialProtocolError("response body is not valid JSON") from exc
90         if not isinstance(value, dict):
91             raise OfficialProtocolError("response JSON must be an object")
92         return value
93
94
95     def _finite_number(payload: Mapping[str, Any], key: str) -> float:
96         value = payload.get(key)
97         if isinstance(value, bool) or not isinstance(value, (int, float)):

```

```

98         raise OfficialProtocolError(f"response field {key!r} must be a number")
99     result = float(value)
100     if not math.isfinite(result):
101         raise OfficialProtocolError(f"response field {key!r} must be finite")
102     return result
103
104
105 class OfficialEnvironment:
106
107     def __init__(
108         self,
109         robot_id: str,
110         *,
111         base_url: str = "http://127.0.0.1:2026",
112         arena_id: str = "default",
113         timeout_s: float = 5.0,
114         max_retries: int = 2,
115         retry_backoff_s: float = 0.2,
116         exit_reserve_s: float = 15.0,
117         log_path: str | Path | None = None,
118         opener: Callable[..., Any] | None = None,
119         monotonic: Callable[[], float] | None = None,
120         sleeper: Callable[[float], None] | None = None,
121     ) -> None:
122         robot_bytes = robot_id.encode("utf-8")
123         if not 1 <= len(robot_bytes) <= 64:
124             raise ValueError("robot_id UTF-8 length must be in 1..64 bytes")
125         if any(ord(character) < 32 or ord(character) == 127 for character in robot_id):
126             raise ValueError("robot_id cannot contain control characters")
127         if arena_id != "default":
128             raise ValueError("arena_id must be 'default'")
129         if timeout_s <= 0.0:
130             raise ValueError("timeout_s must be positive")
131         if max_retries < 0:
132             raise ValueError("max_retries must be nonnegative")
133         if retry_backoff_s < 0.0 or exit_reserve_s < 0.0:
134             raise ValueError("retry_backoff_s and exit_reserve_s must be nonnegative")
135
136         normalized_url = base_url.rstrip("/")
137         if not normalized_url.startswith(("http://", "https://")):
138             raise ValueError("base_url must start with http:// or https://")
139
140         self._robot_id = robot_id
141         self._arena_id = arena_id
142         self._base_url = normalized_url
143         self._timeout_s = float(timeout_s)
144         self._max_retries = int(max_retries)
145         self._retry_backoff_s = float(retry_backoff_s)
146         self._exit_reserve_s = float(exit_reserve_s)
147         self._log_path = Path(log_path) if log_path is not None else None
148         self._opener = opener or urlopen
149         self._monotonic = monotonic or time.monotonic
150         self._sleeper = sleeper or time.sleep

```

```

151     self._action_lock = threading.Lock()
152
153     self._position = Position(0.0, 0.0)
154     self._channel = 1
155     self._virtual_time_s = 0.0
156     self._active = False
157     self._finished = False
158     self._request_counter = 0
159     self._http_attempt_count = 0
160     self._measure_count = 0
161     self._clear_count = 0
162     self._successful_clear_count = 0
163     self._entered_remaining_real_duration_s: float | None = None
164     self._real_deadline: float | None = None
165     self._pending_request: _PendingRequest | None = None
166
167     @property
168     def state(self) -> EnvironmentState:
169         return EnvironmentState(
170             current_position=self._position,
171             current_channel=self._channel,
172             virtual_time_s=self._virtual_time_s,
173             is_active=self._active,
174             is_finished=self._finished,
175         )
176
177     @property
178     def current_position(self) -> Position:
179         return self._position
180
181     @property
182     def current_channel(self) -> int:
183         return self._channel
184
185     @property
186     def virtual_time_s(self) -> float:
187         return self._virtual_time_s
188
189     @property
190     def entered_remaining_real_duration_s(self) -> float | None:
191         return self._entered_remaining_real_duration_s
192
193     @property
194     def remaining_real_duration_s(self) -> float | None:
195         if self._real_deadline is None:
196             return None
197         return max(0.0, self._real_deadline - self._monotonic())
198
199     @property
200     def request_count(self) -> int:
201         return self._request_counter
202
203     @property

```

```

204 def http_attempt_count(self) -> int:
205     return self._http_attempt_count
206
207 @property
208 def has_pending_request(self) -> bool:
209     return self._pending_request is not None
210
211 @property
212 def pending_request(self) -> dict[str, object] | None:
213     pending = self._pending_request
214     if pending is None:
215         return None
216     return {
217         "action": pending.action,
218         "path": pending.path,
219         "request_id": pending.payload["request_id"],
220     }
221
222 @property
223 def measure_count(self) -> int:
224     return self._measure_count
225
226 @property
227 def clear_count(self) -> int:
228     return self._clear_count
229
230 @property
231 def successful_clear_count(self) -> int:
232     return self._successful_clear_count
233
234 def enter(self) -> EnvironmentState:
235     if self._active or self._finished:
236         raise EnvironmentStateError("enter may be called exactly once")
237     self._require_no_pending_request()
238     with self._exclusive_action():
239         payload = self._base_payload("enter")
240         return self._execute_new_request("enter", "/enter", payload)
241
242 def measure(self, position: Position, channel: int) -> MeasureResult:
243     self._require_active()
244     self._require_no_pending_request()
245     self._require_real_time_for_action()
246     position, channel = self._validate_action(position, channel)
247     with self._exclusive_action():
248         payload = self._action_payload("measure", position, channel)
249         return self._execute_new_request("measure", "/measure", payload)
250
251 def clear(self, position: Position, channel: int) -> ClearResult:
252     self._require_active()
253     self._require_no_pending_request()
254     self._require_real_time_for_action()
255     position, channel = self._validate_action(position, channel)
256     with self._exclusive_action():

```

```

257         payload = self._action_payload("clear", position, channel)
258         return self._execute_new_request("clear", "/clear", payload)
259
260     def exit(self) -> EnvironmentState:
261         self._require_active()
262         self._require_no_pending_request()
263         with self._exclusive_action():
264             payload = self._base_payload("exit")
265             return self._execute_new_request("exit", "/exit", payload)
266
267     def recover_pending(self) -> EnvironmentState | MeasureResult | ClearResult:
268         """原样重放未决请求，不创建新的请求 ID。"""
269         if self._pending_request is None:
270             raise OfficialPendingRequestError("there is no pending request to recover")
271         with self._exclusive_action():
272             return self._complete_pending_request()
273
274     def _execute_new_request(
275         self,
276         action: str,
277         path: str,
278         payload: Mapping[str, Any],
279     ) -> EnvironmentState | MeasureResult | ClearResult:
280         self._pending_request = _PendingRequest(action, path, dict(payload))
281         return self._complete_pending_request()
282
283     def _complete_pending_request(self) -> EnvironmentState | MeasureResult | ClearResult:
284         pending = self._pending_request
285         if pending is None:
286             raise OfficialPendingRequestError("there is no pending request to complete")
287         try:
288             response = self._post(pending.path, pending.payload)
289             result = self._apply_response(pending, response)
290         except (OfficialHTTPError, OfficialRejectedError):
291             self._pending_request = None
292             raise
293         except (OfficialTransportError, OfficialProtocolError):
294             raise
295         self._pending_request = None
296         return result
297
298     def _apply_response(
299         self,
300         pending: _PendingRequest,
301         response: Mapping[str, Any],
302     ) -> EnvironmentState | MeasureResult | ClearResult:
303         action = pending.action
304         if action == "enter":
305             virtual_time = self._accepted_virtual_time(response)
306             max_virtual = _finite_number(response, "max_virtual_duration_s")
307             max_real = _finite_number(response, "max_real_duration_s")
308             remaining = _finite_number(response, "remaining_real_duration_s")
309             if max_virtual <= 0.0 or max_real <= 0.0 or not 0.0 <= remaining <= max_real:

```

```

310         raise OfficialProtocolError("invalid duration values in /enter response")
311     self._position = Position(0.0, 0.0)
312     self._channel = 1
313     self._virtual_time_s = virtual_time
314     self._entered_remaining_real_duration_s = remaining
315     self._real_deadline = self._monotonic() + remaining
316     self._active = True
317     return self.state
318
319     if action == "exit":
320         virtual_time = self._accepted_virtual_time(response)
321         if response.get("exit_reason") != "user_exit":
322             raise OfficialProtocolError("accepted /exit must return user_exit")
323         self._virtual_time_s = virtual_time
324         self._active = False
325         self._finished = True
326         return self.state
327
328     raw_position = pending.payload["position"]
329     assert isinstance(raw_position, Mapping)
330     position = Position(float(raw_position["x"]), float(raw_position["y"]))
331     channel = int(pending.payload["channel"])
332     new_virtual_time = self._accepted_virtual_time(response)
333
334     if action == "measure":
335         raw_status = response.get("measure_result")
336         try:
337             status = MeasureStatus(raw_status)
338         except (TypeError, ValueError) as exc:
339             raise OfficialProtocolError(
340                 f"invalid measure_result {raw_status!r}"
341             ) from exc
342         angle: float | None = None
343         if status is MeasureStatus.DIRECTION:
344             angle = _finite_number(response, "svd_deg")
345             if not 0.0 <= angle < 360.0:
346                 raise OfficialProtocolError("svd_deg must lie in [0, 360)")
347         elif "svd_deg" in response:
348             raise OfficialProtocolError("svd_deg is only valid for direction responses")
349
350     )
351     action_time = self._time_increment(new_virtual_time)
352     self._position = position
353     self._channel = channel
354     self._virtual_time_s = new_virtual_time
355     self._measure_count += 1
356     return MeasureResult(status, angle, action_time, new_virtual_time)
357
358     if action == "clear":
359         raw_status = response.get("clear_result")
360         try:
361             status = ClearStatus(raw_status)
362         except (TypeError, ValueError) as exc:
363             raise OfficialProtocolError(f"invalid clear_result {raw_status!r}") from

```

```

exc
362         action_time = self._time_increment(new_virtual_time)
363         self._position = position
364         self._virtual_time_s = new_virtual_time
365         self._clear_count += 1
366         if status is ClearStatus.SUCCESS:
367             self._successful_clear_count += 1
368         return ClearResult(status, action_time, new_virtual_time)
369
370         raise OfficialProtocolError(f"unknown pending action {action!r}")
371
372     def _base_payload(self, action: str) -> dict[str, Any]:
373         return {
374             "arena_id": self._arena_id,
375             "robot_id": self._robot_id,
376             "request_id": self._next_request_id(action),
377         }
378
379     def _action_payload(
380         self, action: str, position: Position, channel: int
381     ) -> dict[str, Any]:
382         payload = self._base_payload(action)
383         payload["position"] = {"x": position.x, "y": position.y}
384         payload["channel"] = channel
385         return payload
386
387     def _next_request_id(self, action: str) -> str:
388         self._request_counter += 1
389         request_id = f"{action}-{self._request_counter:06d}"
390         if len(request_id.encode("utf-8")) > 128:
391             raise RuntimeError("generated request_id exceeds protocol limit")
392         return request_id
393
394     def _post(self, path: str, payload: Mapping[str, Any]) -> dict[str, Any]:
395         body = json.dumps(
396             dict(payload),
397             ensure_ascii=False,
398             allow_nan=False,
399             separators=(",", ":"),
400         ).encode("utf-8")
401         if len(body) > 65536:
402             raise InvalidActionError("request body exceeds 65536 bytes")
403
404         last_error: BaseException | None = None
405         for attempt in range(self._max_retries + 1):
406             self._http_attempt_count += 1
407             request = Request(
408                 self._base_url + path,
409                 data=body,
410                 headers={"Content-Type": "application/json"},
411                 method="POST",
412             )
413             self._write_log(

```

```

414         {
415             "event": "request",
416             "attempt": attempt + 1,
417             "path": path,
418             "payload": dict(payload),
419         }
420     )
421     try:
422         with self._opener(request, timeout=self._timeout_s) as http_response:
423             status_code = int(getattr(http_response, "status", 200))
424             raw_response = http_response.read()
425     except HTTPError as exc:
426         status_code = int(exc.code)
427         raw_response = exc.read()
428         response = self._safe_parse_error(raw_response)
429         self._write_log(
430             {
431                 "event": "http_error",
432                 "attempt": attempt + 1,
433                 "path": path,
434                 "status_code": status_code,
435                 "response": response,
436             }
437         )
438         if status_code in (429, 500) and attempt < self._max_retries:
439             self._retry_pause(attempt)
440             continue
441         raise OfficialHTTPError(status_code, response) from exc
442     except (URLError, TimeoutError, OSError) as exc:
443         last_error = exc
444         self._write_log(
445             {
446                 "event": "transport_error",
447                 "attempt": attempt + 1,
448                 "path": path,
449                 "error": f"{type(exc).__name__}: {exc}",
450             }
451         )
452         if attempt < self._max_retries:
453             self._retry_pause(attempt)
454             continue
455         break
456
457     response = _json_without_duplicate_keys(raw_response)
458     self._write_log(
459         {
460             "event": "response",
461             "attempt": attempt + 1,
462             "path": path,
463             "status_code": status_code,
464             "response": response,
465         }
466     )

```

```

467         if status_code != 200:
468             raise OfficialHTTPError(status_code, response)
469         self._validate_envelope(response)
470         if response["accepted"] is not True:
471             raise OfficialRejectedError(response)
472         return response
473
474     self._write_log(
475         {
476             "event": "pending_request",
477             "path": path,
478             "request_id": payload["request_id"],
479             "attempt_count": self._max_retries + 1,
480         }
481     )
482     raise OfficialTransportError(
483         f"no response for {path} after {self._max_retries + 1} attempts; "
484         f"the same request_id was reused: {payload['request_id']!r}"
485     ) from last_error
486
487     def _validate_envelope(self, response: Mapping[str, Any]) -> None:
488         if type(response.get("accepted")) is not bool:
489             raise OfficialProtocolError("accepted must be a JSON boolean")
490         _finite_number(response, "real_timestamp_ms")
491         _finite_number(response, "virtual_time_s")
492
493     def _accepted_virtual_time(self, response: Mapping[str, Any]) -> float:
494         self._validate_envelope(response)
495         value = _finite_number(response, "virtual_time_s")
496         if value + 1e-9 < self._virtual_time_s:
497             raise OfficialProtocolError("accepted virtual_time_s moved backwards")
498         return value
499
500     def _time_increment(self, new_virtual_time: float) -> float:
501         increment = new_virtual_time - self._virtual_time_s
502         if increment < -1e-9:
503             raise OfficialProtocolError("virtual_time_s moved backwards")
504         return max(0.0, increment)
505
506     def _require_active(self) -> None:
507         if not self._active:
508             raise EnvironmentStateError("environment is not active; call enter first")
509
510     def _require_no_pending_request(self) -> None:
511         pending = self._pending_request
512         if pending is not None:
513             raise OfficialPendingRequestError(
514                 "request outcome is unresolved; call recover_pending() before sending "
515                 f"a new action (pending {pending.action} id={pending.payload['request_id']!r})"
516             )
517
518     def _require_real_time_for_action(self) -> None:

```

```

519     remaining = self.remaining_real_duration_s
520     if remaining is not None and remaining <= self._exit_reserve_s:
521         raise OfficialDeadlineError(
522             f"only {remaining:.3f}s remain; preserving time for /exit"
523         )
524
525     def _validate_action(self, position: Position, channel: int) -> tuple[Position, int]:
526         if not isinstance(position, Position):
527             raise InvalidActionError("position must be a Position instance")
528         if abs(position.x) > 2_000_000.0 or abs(position.y) > 2_000_000.0:
529             raise InvalidActionError("position coordinate exceeds simulator limit")
530         if isinstance(channel, bool) or not isinstance(channel, int) or not 1 <= channel <=
20:
531             raise InvalidActionError("channel must be an integer in 1..20")
532         return position, channel
533
534     def _retry_pause(self, attempt: int) -> None:
535         delay = self._retry_backoff_s * (2**attempt)
536         remaining = self.remaining_real_duration_s
537         if remaining is not None:
538             delay = min(delay, max(0.0, remaining - self._exit_reserve_s))
539         if delay > 0.0:
540             self._sleeper(delay)
541
542     def _safe_parse_error(self, raw: bytes) -> object:
543         try:
544             return _json_without_duplicate_keys(raw)
545         except OfficialProtocolError:
546             return raw.decode("utf-8", errors="replace")
547
548     def _write_log(self, record: Mapping[str, Any]) -> None:
549         if self._log_path is None:
550             return
551         self._log_path.parent.mkdir(parents=True, exist_ok=True)
552         enriched = {
553             "local_timestamp_ms": int(time.time() * 1000),
554             **dict(record),
555         }
556         with self._log_path.open("a", encoding="utf-8", newline="\n") as handle:
557             handle.write(json.dumps(enriched, ensure_ascii=False, allow_nan=False))
558             handle.write("\n")
559
560     class _ActionContext:
561         def __init__(self, lock: threading.Lock) -> None:
562             self._lock = lock
563
564         def __enter__(self) -> None:
565             if not self._lock.acquire(blocking=False):
566                 raise OfficialEnvironmentError(
567                     "different actions may not be sent concurrently"
568                 )
569
570         def __exit__(self, exc_type: object, exc: object, traceback: object) -> None:

```

```

571         self._lock.release()
572
573     def _exclusive_action(self) -> _ActionContext:
574         return self._ActionContext(self._action_lock)
575
576
577 __all__ = [
578     "OfficialDeadlineError",
579     "OfficialEnvironment",
580     "OfficialEnvironmentError",
581     "OfficialHTTPError",
582     "OfficialPendingRequestError",
583     "OfficialProtocolError",
584     "OfficialRejectedError",
585     "OfficialTransportError",
586 ]

```

Listing 24 evaluation/metrics.py

```

1  from __future__ import annotations
2
3  from radio_sim.domain.models import ProblemType, SimulationMetrics, StrategyResult
4  from radio_sim.environment.local import LocalEnvironment
5
6
7  def build_metrics(
8      env: LocalEnvironment,
9      seed: int,
10     problem: ProblemType,
11     strategy_result: StrategyResult,
12     wall_time_s: float,
13 ) -> SimulationMetrics:
14     diagnostics = env._evaluation_snapshot()
15     cleared_ratio = diagnostics.cleared_count / diagnostics.source_count
16     average_time = (
17         env.virtual_time_s / diagnostics.cleared_count
18         if diagnostics.cleared_count
19         else None
20     )
21     return SimulationMetrics(
22         seed=seed,
23         problem=problem,
24         strategy_name=strategy_result.strategy_name,
25         source_count=diagnostics.source_count,
26         cleared_count=diagnostics.cleared_count,
27         cleared_ratio=cleared_ratio,
28         all_cleared=diagnostics.cleared_count == diagnostics.source_count,
29         virtual_time_s=env.virtual_time_s,
30         average_localize_clear_time_s=average_time,
31         total_distance_m=diagnostics.total_distance_m,
32         measure_count=diagnostics.measure_count,
33         clear_count=diagnostics.clear_count,
34         failed_clear_count=diagnostics.failed_clear_count,

```

```

35     channel_switch_count=diagnostics.channel_switch_count,
36     strategy_completed=strategy_result.completed,
37     wall_time_s=wall_time_s,
38 )

```

Listing 25 evaluation/runner.py

```

1  from __future__ import annotations
2
3  import copy
4  from collections.abc import Iterable
5  from time import perf_counter
6
7  from radio_sim.domain.models import ProblemType, SimulationMetrics
8  from radio_sim.environment.local import LocalEnvironment, PhysicsConfig
9  from radio_sim.evaluation.metrics import build_metrics
10 from radio_sim.scenario.config import ScenarioConfig
11 from radio_sim.strategy.base import Strategy
12
13
14 def run_single(
15     strategy: Strategy,
16     seed: int,
17     problem: ProblemType | str,
18     scenario_config: ScenarioConfig | None = None,
19     physics: PhysicsConfig | None = None,
20 ) -> SimulationMetrics:
21     problem_type = ProblemType.coerce(problem)
22     env = LocalEnvironment.from_seed(
23         seed=seed,
24         problem=problem_type,
25         scenario_config=scenario_config,
26         physics=physics,
27     )
28     started = perf_counter()
29     strategy_result = strategy.run(env)
30     wall_time_s = perf_counter() - started
31     return build_metrics(env, seed, problem_type, strategy_result, wall_time_s)
32
33
34 def run_batch(
35     strategy: Strategy,
36     seeds: Iterable[int],
37     problem: ProblemType | str,
38     scenario_config: ScenarioConfig | None = None,
39     physics: PhysicsConfig | None = None,
40 ) -> list[SimulationMetrics]:
41     results: list[SimulationMetrics] = []
42     for seed in seeds:
43         strategy_instance = copy.deepcopy(strategy)
44         results.append(
45             run_single(
46                 strategy=strategy_instance,

```

```

47         seed=seed,
48         problem=problem,
49         scenario_config=scenario_config,
50         physics=physics,
51     )
52 )
53 return results

```

Listing 26 scenario/config.py

```

1
2 from __future__ import annotations
3
4 from dataclasses import dataclass, replace
5
6 from radio_sim.domain.models import ProblemType
7
8
9 @dataclass(frozen=True, slots=True)
10 class ScenarioConfig:
11     problem: ProblemType = ProblemType.Q3
12     mode: str = "normal"
13
14     # 题目事实
15     source_count_min: int = 10
16     source_count_max: int = 16
17     channel_min: int = 1
18     channel_max: int = 20
19     arena_radius_m: float = 1800.0
20     receive_radius_min_m: float = 1000.0
21     receive_radius_max_m: float = 1500.0
22
23     # 仿真假设:
24     source_count_distribution: str = "discrete_uniform"
25     channel_distribution: str = "uniform_without_replacement"
26     position_distribution: str = "uniform_disk_area"
27     receive_radius_distribution: str = "continuous_uniform"
28     q4_directional_count_distribution: str = "discrete_uniform_nonempty_mix"
29     direction_distribution: str = "continuous_uniform"
30
31     def __post_init__(self) -> None:
32         object.__setattr__(self, "problem", ProblemType.coerce(self.problem))
33         if self.mode != "normal":
34             raise ValueError("only scenario mode 'normal' is implemented")
35         if not 1 <= self.source_count_min <= self.source_count_max <= 20:
36             raise ValueError("source count range must satisfy 1 <= min <= max <= 20")
37         if not 1 <= self.channel_min <= self.channel_max <= 20:
38             raise ValueError("channel range must lie within 1..20")
39         channel_slots = self.channel_max - self.channel_min + 1
40         if self.source_count_max > channel_slots:
41             raise ValueError("source count exceeds available unique channels")
42         if self.arena_radius_m <= 0:
43             raise ValueError("arena_radius_m must be positive")

```

```

44         if not 0 < self.receive_radius_min_m <= self.receive_radius_max_m:
45             raise ValueError("invalid receive radius range")
46
47     @classmethod
48     def for_problem(cls, problem: ProblemType | str, **changes: object) -> "ScenarioConfig"
49     :
50         return cls(problem=ProblemType.coerce(problem), **changes)
51
52     def with_problem(self, problem: ProblemType | str) -> "ScenarioConfig":
53         return replace(self, problem=ProblemType.coerce(problem))

```

Listing 27 scenario/generator.py

```

1  from __future__ import annotations
2
3  from math import cos, pi, sin, sqrt
4  from random import Random
5
6  from radio_sim.domain.models import Position
7  from radio_sim.scenario.config import ScenarioConfig
8  from radio_sim.scenario.models import Scenario, Source, SourceType
9
10
11  class ScenarioGenerator:
12      def generate(self, seed: int, config: ScenarioConfig) -> Scenario:
13          rng = Random(seed)
14          source_count = rng.randint(config.source_count_min, config.source_count_max)
15          channels = rng.sample(
16              range(config.channel_min, config.channel_max + 1),
17              source_count,
18          )
19          source_types = self._source_types(rng, source_count, config)
20
21          sources: list[Source] = []
22          for channel, source_type in zip(channels, source_types, strict=True):
23              radius = config.arena_radius_m * sqrt(rng.random())
24              theta = 2.0 * pi * rng.random()
25              position = Position(radius * cos(theta), radius * sin(theta))
26              receive_radius = rng.uniform(
27                  config.receive_radius_min_m,
28                  config.receive_radius_max_m,
29              )
30              direction = rng.uniform(0.0, 360.0) if source_type is SourceType.DIRECTIONAL
31          else None
32          sources.append(
33              Source(
34                  channel=channel,
35                  position=position,
36                  receive_radius_m=receive_radius,
37                  source_type=source_type,
38                  direction_deg=direction,
39              )

```

```

40         return Scenario(seed=seed, problem=config.problem, sources=tuple(sources))
41
42
43     @staticmethod
44     def _source_types(
45         rng: Random,
46         source_count: int,
47         config: ScenarioConfig,
48     ) -> list[SourceType]:
49         if config.problem.value == "Q3":
50             return [SourceType.OMNIDIRECTIONAL] * source_count
51
52         directional_count = rng.randint(1, source_count - 1)
53         result = [SourceType.DIRECTIONAL] * directional_count
54         result.extend([SourceType.OMNIDIRECTIONAL] * (source_count - directional_count))
55         rng.shuffle(result)
56         return result

```

Listing 28 scenario/models.py

```

1  from __future__ import annotations
2
3  from dataclasses import dataclass
4  from enum import Enum
5
6  from radio_sim.domain.models import Position, ProblemType
7
8
9  class SourceType(str, Enum):
10     OMNIDIRECTIONAL = "omnidirectional"
11     DIRECTIONAL = "directional"
12
13
14  @dataclass(frozen=True, slots=True)
15  class Source:
16     channel: int
17     position: Position
18     receive_radius_m: float
19     source_type: SourceType
20     direction_deg: float | None = None
21
22     def __post_init__(self) -> None:
23         if not 1 <= self.channel <= 20:
24             raise ValueError("source channel must be in 1..20")
25         if self.receive_radius_m <= 0:
26             raise ValueError("receive radius must be positive")
27         if self.source_type is SourceType.DIRECTIONAL:
28             if self.direction_deg is None:
29                 raise ValueError("directional source requires direction_deg")
30             object.__setattr__(self, "direction_deg", self.direction_deg % 360.0)
31         elif self.direction_deg is not None:
32             raise ValueError("omnidirectional source must not have direction_deg")
33

```

```

34
35 @dataclass(frozen=True, slots=True)
36 class Scenario:
37     seed: int
38     problem: ProblemType
39     sources: tuple[Source, ...]
40
41     def __post_init__(self) -> None:
42         channels = [source.channel for source in self.sources]
43         if len(channels) != len(set(channels)):
44             raise ValueError("scenario source channels must be unique")
45
46     def source_by_channel(self) -> dict[int, Source]:
47         return {source.channel: source for source in self.sources}

```

Listing 29 strategy/base.py

```

1 from __future__ import annotations
2
3 from typing import Protocol
4
5 from radio_sim.domain.models import StrategyResult
6 from radio_sim.environment.base import Environment
7
8
9 class Strategy(Protocol):
10     def run(self, env: Environment) -> StrategyResult:
11         ...

```

Listing 30 strategy/smoke_test.py

```

1 from __future__ import annotations
2
3 from radio_sim.domain.models import Position, StrategyResult
4 from radio_sim.environment.base import Environment
5
6
7 class SmokeTestStrategy:
8     def run(self, env: Environment) -> StrategyResult:
9         env.enter()
10         env.measure(Position(300.0, 400.0), 1)
11         env.measure(Position(300.0, 400.0), 2)
12         env.clear(Position(300.0, 0.0), 3)
13         env.measure(Position(300.0, 0.0), 2)
14         env.exit()
15         return StrategyResult(
16             strategy_name=type(self).__name__,
17             completed=True,
18             note="environment lifecycle smoke test only",
19         )

```

Listing 31 utils/deterministic_noise.py

```

1 from __future__ import annotations
2
3 import hashlib
4 import struct
5
6 from radio_sim.domain.models import Position
7
8
9 def measurement_error_deg(seed: int, channel: int, position: Position) -> float:
10     """将（种子、信道、精确数值坐标）确定性映射到 [-1, 1]。
11
12     在允许区间内采用均匀映射属于仿真假设。稳定哈希只是一种实现机制，
13     用于满足官方关于同一地点重复测量误差保持不变的要求。
14     """
15     payload = bytearray()
16     payload.extend(str(int(seed)).encode("ascii"))
17     payload.extend(b"|")
18     payload.extend(str(int(channel)).encode("ascii"))
19     payload.extend(b"|")
20     payload.extend(struct.pack("!d", position.x))
21     payload.extend(struct.pack("!d", position.y))
22     digest = hashlib.sha256(payload).digest()
23     integer = int.from_bytes(digest[:8], byteorder="big", signed=False)
24     unit = integer / ((1 << 64) - 1)
25     return 2.0 * unit - 1.0

```

Listing 32 utils/geometry.py

```

1 from __future__ import annotations
2
3 from math import atan2, degrees, hypot
4
5 from radio_sim.domain.models import Position
6
7 _EPSILON = 1e-9
8
9
10 def distance(a: Position, b: Position) -> float:
11     return hypot(b.x - a.x, b.y - a.y)
12
13
14 def bearing_deg(origin: Position, target: Position) -> float:
15     if distance(origin, target) <= _EPSILON:
16         raise ValueError("bearing is undefined for coincident points")
17     return degrees(atan2(target.y - origin.y, target.x - origin.x)) % 360.0
18
19
20 def circular_difference_deg(a: float, b: float) -> float:
21     """返回位于 [-180, 180) 的带符号最短角差 a-b。"""
22     return (a - b + 180.0) % 360.0 - 180.0
23
24

```

```

25 def in_directional_coverage(
26     source_position: Position,
27     source_direction_deg: float,
28     receiver_position: Position,
29     half_angle_deg: float = 90.0,
30 ) -> bool:
31     if distance(source_position, receiver_position) <= _EPSILON:
32         return True
33     receiver_bearing = bearing_deg(source_position, receiver_position)
34     return abs(circular_difference_deg(receiver_bearing, source_direction_deg)) <= (
35         half_angle_deg + _EPSILON
36     )

```

Listing 33 scripts/run_batch.py

```

1 from __future__ import annotations
2
3 import argparse
4 import csv
5 import json
6 from pathlib import Path
7
8 from radio_sim.evaluation import run_batch
9 from radio_sim.strategy import SmokeTestStrategy
10
11
12 def main() -> None:
13     parser = argparse.ArgumentParser(description=__doc__)
14     parser.add_argument("--start-seed", type=int, default=0)
15     parser.add_argument("--count", type=int, default=100)
16     parser.add_argument("--problem", choices=("Q3", "Q4"), default="Q3")
17     parser.add_argument("--output", type=Path)
18     args = parser.parse_args()
19     if args.count <= 0:
20         parser.error("--count must be positive")
21
22     results = run_batch(
23         strategy=SmokeTestStrategy(),
24         seeds=range(args.start_seed, args.start_seed + args.count),
25         problem=args.problem,
26     )
27     records = [result.to_record() for result in results]
28
29     if args.output is not None:
30         args.output.parent.mkdir(parents=True, exist_ok=True)
31         with args.output.open("w", newline="", encoding="utf-8-sig") as handle:
32             writer = csv.DictWriter(handle, fieldnames=list(records[0]))
33             writer.writeheader()
34             writer.writerows(records)
35
36     summary = {
37         "problem": args.problem,
38         "run_count": len(results),

```

```

39     "seed_start": args.start_seed,
40     "seed_end": args.start_seed + args.count - 1,
41     "mean_cleared_ratio": sum(item.cleared_ratio for item in results) / len(results),
42     "mean_virtual_time_s": sum(item.virtual_time_s for item in results) / len(results),
43     "output": str(args.output) if args.output is not None else None,
44 }
45 print(json.dumps(summary, ensure_ascii=False, indent=2))
46
47
48 if __name__ == "__main__":
49     main()

```

Listing 34 scripts/run_single.py

```

1 from __future__ import annotations
2
3 import argparse
4 import json
5
6 from radio_sim.evaluation import run_single
7 from radio_sim.strategy import SmokeTestStrategy
8
9
10 def main() -> None:
11     parser = argparse.ArgumentParser(description=__doc__)
12     parser.add_argument("--seed", type=int, default=12345)
13     parser.add_argument("--problem", choices=("Q3", "Q4"), default="Q3")
14     args = parser.parse_args()
15
16     metrics = run_single(
17         strategy=SmokeTestStrategy(),
18         seed=args.seed,
19         problem=args.problem,
20     )
21     print(json.dumps(metrics.to_record(), ensure_ascii=False, indent=2))
22
23
24 if __name__ == "__main__":
25     main()

```

F 官方测试运行日志

Listing 35 formal-p3-1-2T4W-AQ2B-BFJ9-DETS.jlog

```

1 {
2   "official_log_envelope": {
3     "package_type": "formal_behavior_log",
4     "envelope_version": 1,
5     "payload_schema_version": "behavior-payload-v1",
6     "team_no": "202619001270",
7     "problem_no": 3,
8     "formal_index": 1,

```

```

9      "case_code": "2T4W-AQ2B-BFJ9-DETS",
10     "created_at_utc": "2026-09-13T06:30:48.147Z",
11     "client_version": "1.1.0",
12     "build_id": "b625dc14aa816fdaf35b6f1984dcf7b2",
13     "build_manifest_sha256": "2
14     a6739bb5f3ecaaab1741d2a2c9c3b01432e0dd704a4224aa99339827302a927",
15     "compression": "gzip",
16     "content_encryption": "aes-256-gcm-chunked"
17 },
18 "execution_summary": {
19     "result_status": "COMPLETE",
20     "strategy_name": "GuaranteedRollingQ3Strategy",
21     "strategy_completed": true,
22     "successful_clear_count": 11,
23     "measure_count": 103,
24     "clear_count": 11,
25     "unique_request_count": 116,
26     "virtual_time_s": 2740.959003,
27     "wall_time_s": 3.1797740000001795,
28     "environment_finished": true,
29     "cleanup_error": null,
30     "cleared_channels": [
31         2,
32         4,
33         5,
34         7,
35         12,
36         13,
37         14,
38         15,
39         16,
40         18,
41         20
42     ],
43     "absent_channels": [
44         1,
45         3,
46         6,
47         8,
48         9,
49         10,
50         11,
51         17,
52         19
53     ]
54 }

```

Listing 36 formal-p3-2-VC2X-UWRN-NBUP-2X6V.jlog

```

1 {
2   "official_log_envelope": {
3     "package_type": "formal_behavior_log",

```

```

4     "envelope_version": 1,
5     "payload_schema_version": "behavior-payload-v1",
6     "team_no": "202619001270",
7     "problem_no": 3,
8     "formal_index": 2,
9     "case_code": "VC2X-UWRN-NBUP-2X6V",
10    "created_at_utc": "2026-09-13T06:35:26.372Z",
11    "client_version": "1.1.0",
12    "build_id": "b625dc14aa816fdaf35b6f1984dcf7b2",
13    "build_manifest_sha256": "2
a6739bb5f3ecaaab1741d2a2c9c3b01432e0dd704a4224aa99339827302a927",
14    "compression": "gzip",
15    "content_encryption": "aes-256-gcm-chunked"
16 },
17 "execution_summary": {
18     "result_status": "COMPLETE",
19     "strategy_name": "GuaranteedRollingQ3Strategy",
20     "strategy_completed": true,
21     "successful_clear_count": 14,
22     "measure_count": 90,
23     "clear_count": 14,
24     "unique_request_count": 106,
25     "virtual_time_s": 3378.625854,
26     "wall_time_s": 4.602846099999624,
27     "environment_finished": true,
28     "cleanup_error": null,
29     "cleared_channels": [
30         1,
31         2,
32         3,
33         4,
34         5,
35         8,
36         9,
37         11,
38         12,
39         15,
40         17,
41         18,
42         19,
43         20
44     ],
45     "absent_channels": [
46         6,
47         7,
48         10,
49         13,
50         14,
51         16
52     ]
53 }
54 }

```

Listing 37 formal-p3-3-M9X2-6D5X-E84D-5AEH.jlog

```
1 {
2   "official_log_envelope": {
3     "package_type": "formal_behavior_log",
4     "envelope_version": 1,
5     "payload_schema_version": "behavior-payload-v1",
6     "team_no": "202619001270",
7     "problem_no": 3,
8     "formal_index": 3,
9     "case_code": "M9X2-6D5X-E84D-5AEH",
10    "created_at_utc": "2026-09-13T06:39:35.814Z",
11    "client_version": "1.1.0",
12    "build_id": "b625dc14aa816fdaf35b6f1984dcf7b2",
13    "build_manifest_sha256": "2
14    a6739bb5f3ecaaab1741d2a2c9c3b01432e0dd704a4224aa993339827302a927",
15    "compression": "gzip",
16    "content_encryption": "aes-256-gcm-chunked"
17  },
18  "execution_summary": {
19    "result_status": "COMPLETE",
20    "strategy_name": "GuaranteedRollingQ3Strategy",
21    "strategy_completed": true,
22    "successful_clear_count": 14,
23    "measure_count": 98,
24    "clear_count": 14,
25    "unique_request_count": 114,
26    "virtual_time_s": 3367.690795,
27    "wall_time_s": 4.861793000000034,
28    "environment_finished": true,
29    "cleanup_error": null,
30    "cleared_channels": [
31      2,
32      3,
33      5,
34      6,
35      8,
36      9,
37      10,
38      11,
39      12,
40      15,
41      17,
42      18,
43      19,
44      20
45    ],
46    "absent_channels": [
47      1,
48      4,
49      7,
50      13,
51      14,
```

```

51     16
52   ]
53 }
54 }

```

Listing 38 formal-p4-1-4Z33-SF8B-2CGX-7UYM.jlog

```

1 {
2   "official_log_envelope": {
3     "package_type": "formal_behavior_log",
4     "envelope_version": 1,
5     "payload_schema_version": "behavior-payload-v1",
6     "team_no": "202619001270",
7     "problem_no": 4,
8     "formal_index": 1,
9     "case_code": "4Z33-SF8B-2CGX-7UYM",
10    "created_at_utc": "2026-09-13T06:47:40.315Z",
11    "client_version": "1.1.0",
12    "build_id": "b625dc14aa816fdaf35b6f1984dcf7b2",
13    "build_manifest_sha256": "2
a6739bb5f3ecaaab1741d2a2c9c3b01432e0dd704a4224aa99339827302a927",
14    "compression": "gzip",
15    "content_encryption": "aes-256-gcm-chunked"
16  },
17  "execution_summary": {
18    "result_status": "COMPLETE",
19    "strategy_name": "GuaranteedDirectionalQ4Strategy",
20    "strategy_completed": true,
21    "successful_clear_count": 12,
22    "measure_count": 292,
23    "clear_count": 12,
24    "unique_request_count": 306,
25    "virtual_time_s": 6710.566225,
26    "wall_time_s": 5.5463644999999815,
27    "environment_finished": true,
28    "cleanup_error": null,
29    "cleared_channels": [
30      2,
31      3,
32      6,
33      8,
34      9,
35      11,
36      13,
37      15,
38      16,
39      17,
40      18,
41      20
42    ],
43    "absent_channels": [
44      1,
45      4,

```

```

46     5,
47     7,
48     10,
49     12,
50     14,
51     19
52 ]
53 }
54 }

```

Listing 39 formal-p4-2-MAF3-M9C7-BEAZ-D6V9.jlog

```

1 {
2   "official_log_envelope": {
3     "package_type": "formal_behavior_log",
4     "envelope_version": 1,
5     "payload_schema_version": "behavior-payload-v1",
6     "team_no": "202619001270",
7     "problem_no": 4,
8     "formal_index": 2,
9     "case_code": "MAF3-M9C7-BEAZ-D6V9",
10    "created_at_utc": "2026-09-13T06:50:07.173Z",
11    "client_version": "1.1.0",
12    "build_id": "b625dc14aa816fdaf35b6f1984dcf7b2",
13    "build_manifest_sha256": "2
a6739bb5f3ecaaab1741d2a2c9c3b01432e0dd704a4224aa99339827302a927",
14    "compression": "gzip",
15    "content_encryption": "aes-256-gcm-chunked"
16  },
17  "execution_summary": {
18    "result_status": "COMPLETE",
19    "strategy_name": "GuaranteedDirectionalQ4Strategy",
20    "strategy_completed": true,
21    "successful_clear_count": 12,
22    "measure_count": 291,
23    "clear_count": 12,
24    "unique_request_count": 305,
25    "virtual_time_s": 7033.751908,
26    "wall_time_s": 6.099793800000043,
27    "environment_finished": true,
28    "cleanup_error": null,
29    "cleared_channels": [
30      2,
31      3,
32      6,
33      8,
34      11,
35      12,
36      13,
37      14,
38      15,
39      17,
40      19,

```

```

41     20
42 ],
43 "absent_channels": [
44     1,
45     4,
46     5,
47     7,
48     9,
49     10,
50     16,
51     18
52 ]
53 }
54 }

```

Listing 40 formal-p4-3-KKFA-B32Q-ZKTZ-ETUB.jlog

```

1 {
2   "official_log_envelope": {
3     "package_type": "formal_behavior_log",
4     "envelope_version": 1,
5     "payload_schema_version": "behavior-payload-v1",
6     "team_no": "202619001270",
7     "problem_no": 4,
8     "formal_index": 3,
9     "case_code": "KKFA-B32Q-ZKTZ-ETUB",
10    "created_at_utc": "2026-09-13T06:52:42.427Z",
11    "client_version": "1.1.0",
12    "build_id": "b625dc14aa816fdaf35b6f1984dcf7b2",
13    "build_manifest_sha256": "2
a6739bb5f3ecaaab1741d2a2c9c3b01432e0dd704a4224aa99339827302a927",
14    "compression": "gzip",
15    "content_encryption": "aes-256-gcm-chunked"
16  },
17  "execution_summary": {
18    "result_status": "COMPLETE",
19    "strategy_name": "GuaranteedDirectionalQ4Strategy",
20    "strategy_completed": true,
21    "successful_clear_count": 14,
22    "measure_count": 243,
23    "clear_count": 24,
24    "unique_request_count": 269,
25    "virtual_time_s": 6535.328162,
26    "wall_time_s": 6.045969900000273,
27    "environment_finished": true,
28    "cleanup_error": null,
29    "cleared_channels": [
30      1,
31      2,
32      3,
33      5,
34      7,
35      8,

```

```
36     9,  
37     10,  
38     11,  
39     14,  
40     15,  
41     18,  
42     19,  
43     20  
44 ],  
45 "absent_channels": [  
46     4,  
47     6,  
48     12,  
49     13,  
50     16,  
51     17  
52 ]  
53 }  
54 }
```